Politecnico di Torino

Master of Science in Computer Engineering

Department of Control and Computer Engineering (DAUIN)

Database and Data Mining Group (DBDMG)

Large Language Models for Software Engineering

Theoretical foundations, methodologies, and practical architectures

Comprehensive master's course study guide

Authored for graduate students, researchers, and practitioners

Course Instructors:
Prof. Flavio Giobergia
Prof. Riccardo Coppola
Politecnico di Torino

Academic Year:
2025 / 2026

Target Program:
M.Sc. Computer Engineering

Compiled and Typeset with LaTeX and `tcolorbox`
Torino, Italy — Academic Year 2025/2026

Contents

Preface iii

Notation and conventions v

I Foundations of large language models 1

1 Introduction to language models 2
- 1.1 The sequence prediction problem and probabilistic language modeling 2
- 1.2 Markov assumptions and n-gram language models 4
- 1.3 Transition count matrix construction walkthrough 6
- 1.4 Autoregressive estimation and generation 8
- 1.5 Fundamental limitations of statistical n-gram models: sparsity and zero-frequency collapse 10
- 1.6 Statistical smoothing and discounting algorithms 12
- 1.7 Evaluation metrics: cross-entropy and perplexity 16
- 1.8 Historical evolution of language models 20

2 Deep learning foundations for NLP 25
- 2.1 From biological neurons to the artificial perceptron 25
- 2.2 Stacking linear layers and the proof of linear stacking collapse 28
- 2.3 Taxonomy of neural activation functions 32
- 2.4 Architectural task formulations and software engineering case studies 35
- 2.5 Loss function formulations and optimization landscapes 36
- 2.6 Computational graphs and exact adjoint backpropagation 39
- 2.7 The sequential computation impasse: bridge to recurrent neural networks 43

3 Recurrent neural networks and sequence modeling 45
- 3.1 Sequential data and constraints of feed-forward neural networks 45
- 3.2 The recurrent neural network cell and weight sharing 46
- 3.3 Backpropagation through time and the proof of vanishing gradients 48
- 3.4 Case correction demonstration: Sherlock Holmes training dynamics 51
- 3.5 Gated architectures: LSTM and GRU 54
- 3.6 Sequence-to-sequence (Seq2Seq) and encoder-decoder pipelines 57
- 3.7 Persistent bottlenecks of gated RNNs and the road to transformers 60

4 Distributed word representations and Word2Vec 65
- 4.1 The representation problem: why continuous embeddings are mandatory 65
- 4.2 The baseline approach: local one-hot encodings and why they fail 66
- 4.3 Distributed representations and the distributional hypothesis 70
- 4.4 The embedding training loop and semantic geometry evolution 71
- 4.5 Word2Vec continuous bag-of-words (CBOW) 74

4.6 Word2Vec continuous skip-gram and the softmax bottleneck 76
4.7 Scaling softmax: hierarchical softmax and negative sampling 78
4.8 Limitations of Word2Vec and the out-of-vocabulary (OOV) barrier 83
4.9 FastText: subword character n-gram embeddings 84
4.10 Emergent vector space geometry: clustering, metrics and relational analogies . . 86
4.11 Modern subword tokenization and input pipelines for foundation models 91

Preface

Pedagogical vision

The emergence of Large Language Models (LLMs) represents a fundamental paradigm shift in computer science and software engineering. Moving beyond rigid syntax-directed rules, modern software engineering increasingly leverages deep probabilistic models capable of synthesizing code, repairing complex bugs, formalizing requirements, and orchestrating multi-agent systems.

This study guide accompanies the Master of Science course *Large Language Models for Software Engineering* at Politecnico di Torino, taught by Prof. Flavio Giobergia and Prof. Riccardo Coppola. The curriculum is intentionally structured into two complementary pillars:

1. **Part I: Foundations of large language models** — Traces the mathematical and architectural lineage of language modeling, beginning with n-gram Markov chains, proceeding through feedforward networks, distributed word embeddings (Word2Vec, FastText), recurrent networks and vanishing gradient proofs, and culminating in the self-attention Transformer architecture, pre-training scaling laws, instruction tuning (RLHF, DPO), and parameter-efficient fine-tuning (LoRA, QLoRA).

2. **Part II: Large language models for software engineering** — Bridges foundational model capabilities with the Software Development Life Cycle (SDLC), exploring prompt engineering patterns, autonomous agent frameworks (ReAct, multi-agent collaboration), automated test generation, software maintenance and refactoring, rigorous evaluation methodologies (Pass@k, CodeBLEU), and security, intellectual property, and ethical implications.

Visual callout structure

To maximize learning effectiveness and exam readiness, the guide embeds three specialized pedagogical environments:

Exam insights: Recurrent exam patterns and pitfalls

Highlighted in terracotta/amber, these boxes isolate specific questions frequently asked in oral and written examinations, point out common mathematical misconceptions (e.g., mixing logarithm bases when computing perplexity), and underscore key grading criteria.

Intuition & analogy: Geometric and conceptual analogies

Highlighted in royal purple, these boxes provide conceptual, visual, and intuitive interpretations of complex mathematical machinery (e.g., attention as a soft differentiable lookup table, high-dimensional vector space geometry).

External deep dive: Seminal research and authoritative multimedia

Highlighted in pine teal, these boxes contextualize textbook theory with seminal academic papers [1–3] and breakdowns by leading researchers and educators.

Examination framework and course organization

As detailed by the instructors in the introductory lecture, assessment in this course is divided into two distinct components:

1. **Written Examination (18 Points)**:
    - Duration: 75 minutes.
    - Format: Typically around 12 questions (combining closed-ended multiple-choice items and open-ended technical/mathematical derivations).
    - Scope: Comprehensive coverage across all foundational and applied software engineering topics.
    - Threshold: A minimum score of 10/18 is strictly required to achieve sufficiency.
2. **Practical Course Project (14 Points)**:
    - Implementation: Development of an applied LLM-for-software-engineering system using Python and PyTorch.
    - Modality: Undertaken individually or in small teams.
    - Scoring: Total points across the written test (18) and project (14) sum to 32, with scores exceeding 30 earning *30 e Lode* (honors).
3. **Lecture and Lab Schedule**:
    - Monday: 08:30 – 11:30 (Room 8) — Core theoretical foundations.
    - Wednesday: 11:30 – 13:00 (Room R1) — Practical workshops, interactive coding, and PyTorch laboratories.

How to use this study guide

Each chapter is fully self-contained and begins with precise learning objectives, followed by exhaustive mathematical derivations, algorithmic walkthroughs, and practical software engineering implications. Students are encouraged to independently verify all mathematical derivations and implement the core algorithms to develop deep mechanical empathy for large language models.

Notation and conventions

Throughout this study guide, we adopt consistent mathematical, algorithmic, and typographic conventions aligned with standard deep learning and software engineering literature.

Linear algebra and tensor operations

Symbol	Type	Description
x, y, z	Scalar	Real-valued scalars or scalar components ($x \in \mathbb{R}$).
$\mathbf{x}, \mathbf{y}, \mathbf{h}$	Vector	Column vectors ($\mathbf{x} \in \mathbb{R}^d$). Lowercase bold font.
$\mathbf{x}^\top$	Transpose	Transpose of vector $\mathbf{x}$ (row vector $\mathbf{x}^\top \in \mathbb{R}^{1\times d}$).
$\mathbf{A}, \mathbf{W}, \mathbf{X}$	Matrix	Real matrices ($\mathbf{A} \in \mathbb{R}^{m\times n}$). Uppercase bold font.
$\mathbf{A}_{i,j}$ or a_{ij}	Scalar	Entry at row i and column j of matrix $\mathbf{A}$.
$\mathbf{I}$ or $\mathbf{I}_d$	Matrix	Identity matrix of dimension $d \times d$.
$\|\mathbf{x}\|_2$ or $\|\mathbf{x}\|$	Scalar	Euclidean (L_2) norm of vector $\mathbf{x}$, $\sqrt{\sum_i x_i^2}$.
$\|\mathbf{A}\|_F$	Scalar	Frobenius norm of matrix $\mathbf{A}$, $\sqrt{\sum_{i,j} A_{i,j}^2}$.
$\langle \mathbf{u}, \mathbf{v} \rangle$ or $\mathbf{u}^\top \mathbf{v}$	Scalar	Standard dot (inner) product between vectors $\mathbf{u}$ and $\mathbf{v}$.
$\odot$	Operator	Element-wise (Hadamard) product between tensors of identical shape.
$\otimes$	Operator	Outer product or Kronecker tensor product.

Table 1: Linear algebra and tensor notation summary.

Probability and information theory

Neural networks, transformers, and software engineering

Symbol	Type	Description
$\mathbb{P}(A)$	Scalar	Probability of event A ($\mathbb{P} : \mathcal{F} \to [0, 1]$).
$\mathbb{P}(X = x)$	Scalar	Probability mass function for discrete random variable X.
$\mathbb{P}(Y \mid X)$	Distribution	Conditional probability distribution of Y given X.
$\mathbb{E}_{x\sim P}[f(x)]$	Scalar	Expected value of function $f(x)$ under distribution P.
$\mathbb{V}\text{ar}(X)$	Scalar	Variance of random variable X, $\mathbb{E}[(X - \mathbb{E}[X])^2]$.
$\mathcal{H}(P)$	Scalar	Shannon entropy of distribution P, $-\sum_x P(x) \log P(x)$.
$\mathcal{H}(P, Q)$	Scalar	Cross-entropy between target distribution P and model Q.
$D_{\mathrm{KL}}(P \parallel Q)$	Scalar	Kullback–Leibler (KL) divergence from Q to P.
$\mathrm{PPL}(W)$	Scalar	Perplexity of word sequence W under a language model.

Table 2: Probability and information theoretic notation.

Symbol	Type	Description
$\mathcal{V}$	Set	Discrete vocabulary of tokens; $\lvert\mathcal{V}\rvert = V$ denotes vocabulary size.
T or T	Integer	Sequence length (number of input or output tokens).
d_{model}	Integer	Hidden embedding dimensionality of the Transformer model.
d_{ff}	Integer	Dimensionality of the intermediate Feed-Forward Network layer.
d_k, d_v	Integer	Dimensionality of query/key and value projections per head.
h	Integer	Number of parallel attention heads ($h \cdot d_k = d_{\text{model}}$).
$\sigma(z)$	Function	Standard logistic sigmoid function: $\sigma(z) = \frac{1}{1+e^{-z}}$.
$\mathrm{softmax}(\mathbf{z})$	Function	Softmax normalized exponential vector operator.
$\mathrm{ReLU}(z)$	Function	Rectified Linear Unit activation: $\max(0, z)$.
$\mathrm{GELU}(z)$	Function	Gaussian Error Linear Unit activation.
$\mathcal{L}$ or $\mathcal{L}$	Scalar	Scalar objective / loss function minimized during optimization.
Pass@k	Metric	Probability that at least one of k generated code samples passes unit tests.
CodeBLEU	Metric	Multi-aspect code evaluation metric combining BLEU, AST, and data flow.

Table 3: Model architecture parameters and SE metrics.

Part I

Foundations of large language models

CHAPTER 1

Introduction to language models

1.1 The sequence prediction problem and probabilistic language modeling

At its core, a language model formalizes the intuitive mechanism by which humans evaluate the fluency, plausibility, and likelihood of word sequences. In Natural Language Processing (NLP) and modern AI-assisted Software Engineering, we model human language and source code not through rigid, brittle rule systems, but as a discrete stochastic process governed by probability distributions.

Let $\mathcal{V} = \{w_1, w_2, \ldots, w_V\}$ denote a finite vocabulary of $V = |\mathcal{V}|$ discrete tokens (e.g., words, subwords, or punctuation symbols). An arbitrary natural language sentence, code snippet, or document of length N is represented as an ordered sequence of discrete random variables $W = (w_1, w_2, \ldots, w_N) \in \mathcal{V}^N$.

Definition 1.1: *Statistical language model*

A *statistical language model* is a formal probability distribution $\mathbb{P}$ defined over the set of all possible discrete sequences $\mathcal{S} = \bigcup_{N=1}^{\infty} \mathcal{V}^N$. For any specific sequence $W = (w_1, \ldots, w_N)$, the language model assigns a scalar probability $\mathbb{P}(W) \in [0, 1]$ satisfying the Kolmogorov probability axioms:

$$\mathbb{P}(W) \geq 0 \quad \forall W \in \mathcal{S}, \quad \text{and} \quad \sum_{S \in \mathcal{S}} \mathbb{P}(S) = 1. \tag{1.1}$$

The model maps the syntactic and semantic conventions of human language into real values on the unit interval:

- Fluent, grammatically coherent sentences receive high probability mass:

$$\mathbb{P}(\text{“this”}, \text{“is”}, \text{“a”}, \text{“reasonable”}, \text{“sentence”}) = 0.1$$

- Syntactically valid but semantically unusual or rare sentences receive noticeably lower probability:

$$\mathbb{P}(\text{“this”}, \text{“is”}, \text{“a”}, \text{“purple”}, \text{“sentence”}) = 0.01$$

- Ungrammatical, corrupted, or disjoint sequences receive negligible probability:

$$\mathbb{P}(\text{“this”}, \text{“are”}, \text{“a”}, \text{“reasonable”}, \text{“sentence”}) = 0.001$$

Evaluating the joint probability $\mathbb{P}(w_1, \ldots, w_N)$ directly over the combinatorial space $\mathcal{V}^N$ is intractable because observing every conceivable multi-word sequence in a training dataset

is impossible. To structure this estimation problem, we invoke the fundamental *chain rule of probability*.

Definition 1.2: *Chain rule decomposition of language models*

For any sequence of random variables $(w_1, w_2, \ldots, w_N)$, the joint probability factors identically and unconditionally into an ordered product of conditional next-token probabilities:

$$\begin{aligned}\mathbb{P}(w_1, w_2, \ldots, w_N) &= \mathbb{P}(w_1) \cdot \mathbb{P}(w_2 \mid w_1) \cdot \mathbb{P}(w_3 \mid w_1, w_2) \cdots \mathbb{P}(w_N \mid w_1, \ldots, w_{N-1}) \\ &= \prod_{t=1}^{N} \mathbb{P}(w_t \mid w_1, \ldots, w_{t-1}) = \prod_{t=1}^{N} \mathbb{P}(w_t \mid w_{<t}),\end{aligned} \tag{1.2}$$

where $w_{<t} = (w_1, \ldots, w_{t-1})$ represents the prefix history preceding timestep t. For the initial token $t = 1$, the conditioning prefix is the empty sequence $\emptyset$, giving $\mathbb{P}(w_1 \mid w_{<1}) = \mathbb{P}(w_1)$.

Applying Equation (1.2) to our running example yields the concrete factorization:

$$\begin{aligned}\mathbb{P}(\text{this, is, a, reasonable, sentence}) = \mathbb{P}(&\text{this}) \\ &\cdot \mathbb{P}(\text{is} \mid \text{this}) \\ &\cdot \mathbb{P}(\text{a} \mid \text{this, is}) \\ &\cdot \mathbb{P}(\text{reasonable} \mid \text{this, is, a}) \\ &\cdot \mathbb{P}(\text{sentence} \mid \text{this, is, a, reasonable}).\end{aligned}$$

Reframing language modeling as conditional prediction. This mathematical decomposition marks a pivotal conceptual shift in Natural Language Processing. Instead of attempting to score complete, static documents simultaneously, the modeling focus shifts entirely to sequential, conditional next-word estimation:

Given history $w_{<t}$, estimate the categorical probability distribution over all words in $\mathcal{V}$: $\mathbb{P}(w_t = w \mid w_{<t})$.

This formulation aligns directly with the classic psychological *cloze test* (fill-in-the-blank):

"This is a reasonable _____*"* $\implies$ $\mathbb{P}(\text{sentence} \mid \text{This is a reasonable}) = ?$

Every modern autoregressive language model, including GPT-4 and Claude, relies fundamentally on this conditional next-token formulation.

Intuition & analogy: The core thesis: next-word prediction as full language modeling

In the opening lecture of the Politecnico di Torino course, the fundamental essence of modern language modeling was crystallized:

> *"If we can predict the next word, then we have estimated this conditional probability. And then we can plug this into the chain rule, and estimate the probability of any arbitrary sentence. This is what a language model does: nothing more, nothing less. If you understand this idea, you understand the core mechanism driving the entire course, from 1980s Markov chains to modern trillion-parameter LLMs."*

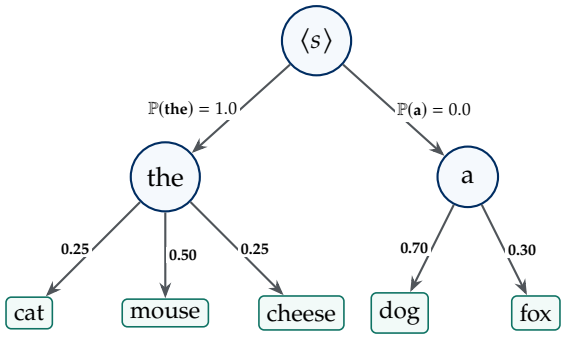


Figure 1.1: Visualizing autoregressive sequence generation as path traversal across a probability branching tree. At the root symbol $\langle s \rangle$, outgoing edges partition total probability mass across vocabulary tokens. At each node $w_{<t}$, children represent the conditional distribution $\mathbb{P}(w_t \mid w_{<t})$. The law of total probability guarantees that branch probabilities sum to unity at each decision point.

Intuition & analogy: The branching probability tree

Sequence generation can be visualized geometrically as walking down a directed decision tree (Figure 1.1). At the root node $\langle s \rangle$, the model confronts the entire vocabulary $\mathcal{V}$. Choosing a token w_1 traverses an edge with probability $\mathbb{P}(w_1)$ and lands at a new conditioning state. From this state, another set of branches opens, weighted by $\mathbb{P}(w_2 \mid w_1)$. The joint probability of an entire sentence is simply the product of edge weights along that unique trajectory. Generating text consists of sampling a path from the root to a designated terminal token $\langle /s \rangle$.

1.2 Markov assumptions and n-gram language models

Although Equation (1.2) is mathematically exact and preserves all syntactic and semantic dependencies, computing $\mathbb{P}(w_t \mid w_{<t})$ directly in empirical applications presents an insurmountable combinatorial bottleneck. As the sequence index t grows, the conditioning history $w_{<t} = (w_1, \ldots, w_{t-1})$ expands monotonically. Because each constituent token is drawn from a vocabulary $\mathcal{V}$ of size $V = |\mathcal{V}|$, the space of potential prefix histories preceding token t is the Cartesian product space $\mathcal{V}^{t-1}$, whose cardinality is:

$$|\mathcal{H}_t| = |\mathcal{V}|^{t-1} = V^{t-1}. \tag{1.3}$$

Even for a modest vocabulary of $V = 10{,}000$ words, tracking histories of length $t = 4$ requires estimating probabilities over $10{,}000^3 = 10^{12}$ distinct conditioning states. In any finite training corpus, almost every sequence of length greater than three occurs exactly zero times.

To make estimation mathematically tractable, the principle of conditional independence formalizes that future tokens depend only on a bounded historical window. In the landmark 1948 treatise on information theory [4], it was demonstrated that modeling natural language as a discrete Markov process produces increasingly coherent synthetic English as the conditioning order increases. Modern pedagogical treatments [5] formalize this simplification as the foundation of classical statistical natural language processing.

Definition 1.3: *Markov assumption of order $n-1$*

An n-gram language model assumes that the conditional probability of the current

word w_t depends exclusively on the immediately preceding $n-1$ words, rendering it conditionally independent of all earlier discourse history:

$$\mathbb{P}(w_t \mid w_1, w_2, \ldots, w_{t-1}) \approx \mathbb{P}(w_t \mid w_{t-n+1}, \ldots, w_{t-1}). \tag{1.4}$$

The integer $n \geq 1$ dictates the order of the model:

- **Unigram Model ($n = 1$, Order 0)**: Completely memoryless. Each word is treated as statistically independent of all past and future tokens:

$$\mathbb{P}(w_1, \ldots, w_N) \approx \prod_{t=1}^{N} \mathbb{P}(w_t).$$

- **Bigram Model ($n = 2$, Order 1)**: First-order Markov chain. Conditioning depends exclusively on the single preceding token w_{t-1}:

$$\mathbb{P}(w_1, \ldots, w_N) \approx \prod_{t=1}^{N} \mathbb{P}(w_t \mid w_{t-1}).$$

- **Trigram Model ($n = 3$, Order 2)**: Second-order Markov chain. Conditioning depends on the preceding pair of tokens (w_{t-2}, w_{t-1}):

$$\mathbb{P}(w_1, \ldots, w_N) \approx \prod_{t=1}^{N} \mathbb{P}(w_t \mid w_{t-2}, w_{t-1}).$$

1.2.1 Combinatorial state-space reduction

The primary mathematical virtue of the Markov assumption is that it truncates an exponentially growing state space into a fixed, stationary cardinality. Under an n-gram model, the conditioning context is always restricted to $n-1$ words, regardless of the absolute sequence position t:

$$|\mathcal{H}^{(n)}| = |\mathcal{V}|^{n-1} = V^{n-1}. \tag{1.5}$$

Consequently, specifying the complete transition probability distribution across the entire vocabulary requires an $(n-1)$-dimensional conditioning tensor containing:

$$P_{\text{params}} = V^{n-1} \times (V - 1) \approx V^n \tag{1.6}$$

free parameters. Table 1.1 contrasts the unconstrained historical state space with fixed n-gram models for realistic vocabulary sizes.

While the unconstrained state space exceeds the total number of atoms in the observable universe ($\approx 10^{80}$) by step $t = 20$, a bigram model maintains a manageable state space of 50,000 rows. However, this drastic dimensional reduction comes at the cost of complete amnesia for any context preceding the immediate $n-1$ tokens.

1.2.2 Maximum likelihood estimation (MLE)

Under Maximum Likelihood Estimation, we estimate the conditional transition probabilities directly by counting occurrences of exact token sequences in a training corpus [5]. For a generic n-gram:

$$\mathbb{P}_{\text{MLE}}(w_t \mid w_{t-n+1}, \ldots, w_{t-1}) = \frac{C(w_{t-n+1}, \ldots, w_{t-1}, w_t)}{C(w_{t-n+1}, \ldots, w_{t-1})}, \tag{1.7}$$

Table 1.1: State-space cardinality $|\mathcal{H}|$ as a function of sequence position t versus fixed n-gram Markov orders for vocabularies $V = 10{,}000$ and $V = 50{,}000$.

Model / Order	**Effective Context**	**Formula**	**States ($V = 10^4$)**	**States ($V = 5 \times 10^4$)**
Unconstrained ($t = 2$)	1 previous token	V^1	1.0×10^4	5.0×10^4
Unconstrained ($t = 3$)	2 previous tokens	V^2	1.0×10^8	2.5×10^9
Unconstrained ($t = 5$)	4 previous tokens	V^4	1.0×10^{16}	6.25×10^{18}
Unconstrained ($t = 10$)	9 previous tokens	V^9	1.0×10^{36}	1.95×10^{42}
Unconstrained ($t = 20$)	19 previous tokens	V^{19}	1.0×10^{76}	1.90×10^{89}
Unigram ($n = 1$)	0 tokens (memoryless)	$V^0 = 1$	1	1
Bigram ($n = 2$)	1 previous token	V^1	1.0×10^4	5.0×10^4
Trigram ($n = 3$)	2 previous tokens	V^2	1.0×10^8	2.5×10^9
4-gram ($n = 4$)	3 previous tokens	V^3	1.0×10^{12}	1.25×10^{14}

where $C(\cdot)$ denotes the raw frequency of that exact sequence in the training text. For a bigram model ($n = 2$), Equation (1.7) specializes to:

$$\mathbb{P}_{\text{MLE}}(w_t \mid w_{t-1}) = \frac{C(w_{t-1}, w_t)}{C(w_{t-1})} = \frac{C(w_{t-1}, w_t)}{\sum_{w \in \mathcal{V}} C(w_{t-1}, w)}. \tag{1.8}$$

Normalizing each row by the unigram count of the conditioning prefix ensures that $\sum_{w \in \mathcal{V}} \mathbb{P}_{\text{MLE}}(w \mid w_{t-1}) = 1$.

1.3 Transition count matrix construction walkthrough

To understand exactly how statistical n-gram language models operate, we trace the step-by-step extraction of bigram frequencies and transition probabilities from the authoritative lecture corpus:

1. Sentence 1: *"The cat chased the mouse happily"*
2. Sentence 2: *"The mouse ate the cheese"*

Vocabulary extraction. Extracting all unique tokens (case-normalized with standard initial capitalization for clarity) yields a vocabulary of size $|V| = 7$:

$$\mathcal{V} = \{\text{Ate, Cat, Chased, Cheese, Happily, Mouse, The}\}.$$

Sequential bigram decomposition. We iterate through both training sentences token by token, recording each observed transition $(w_{t-1} \to w_t)$:

- From Sentence 1 (*"The cat chased the mouse happily"*):
  1. $(\text{The, cat}) \implies C(\text{The, Cat}) \leftarrow 1$
  2. $(\text{cat, chased}) \implies C(\text{Cat, Chased}) \leftarrow 1$
  3. $(\text{chased, the}) \implies C(\text{Chased, The}) \leftarrow 1$
  4. $(\text{the, mouse}) \implies C(\text{The, Mouse}) \leftarrow 1$
  5. $(\text{mouse, happily}) \implies C(\text{Mouse, Happily}) \leftarrow 1$
- From Sentence 2 (*"The mouse ate the cheese"*):

6. $(\text{The}, \text{mouse}) \implies C(\text{The}, \text{Mouse}) \leftarrow 2$
7. $(\text{mouse}, \text{ate}) \implies C(\text{Mouse}, \text{Ate}) \leftarrow 1$
8. $(\text{ate}, \text{the}) \implies C(\text{Ate}, \text{The}) \leftarrow 1$
9. $(\text{the}, \text{cheese}) \implies C(\text{The}, \text{Cheese}) \leftarrow 1$

Unigram normalization counts. Summing across rows provides the total unigram occurrences $C(w_{t-1})$:

$$\begin{aligned} C(\text{The}) &= 4 \quad (\text{cat} : 1, \text{mouse} : 2, \text{cheese} : 1) \\ C(\text{Cat}) &= 1 \quad (\text{chased} : 1) \\ C(\text{Chased}) &= 1 \quad (\text{the} : 1) \\ C(\text{Mouse}) &= 2 \quad (\text{happily} : 1, \text{ate} : 1) \\ C(\text{Ate}) &= 1 \quad (\text{the} : 1) \\ C(\text{Cheese}) &= 0 \quad (\text{terminal word, no outgoing transitions}) \\ C(\text{Happily}) &= 0 \quad (\text{terminal word, no outgoing transitions}) \end{aligned}$$

Full empirical transition probability matrix. Applying Equation (1.8) yields the complete 7×7 transition matrix $\mathbf{M} \in \mathbb{R}^{7\times7}$ where row i represents condition w_{t-1} and column j represents next token w_t. Table 1.2 presents both raw counts and normalized MLE probabilities.

Table 1.2: Empirical transition count matrix and normalized bigram conditional probabilities $\mathbb{P}(w_t \mid w_{t-1})$ for vocabulary $\mathcal{V} = \{\text{Ate, Cat, Chased, Cheese, Happily, Mouse, The}\}$.

Condition (w_{t-1})	Ate	Cat	Chased	Cheese	Happily	Mouse	The	Row Total
Ate	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	1 (1.00)	1
Cat	0 (0.0)	0 (0.0)	1 (1.00)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	1
Chased	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	1 (1.00)	1
Cheese	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0
Happily	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0
Mouse	1 (0.50)	0 (0.0)	0 (0.0)	0 (0.0)	1 (0.50)	0 (0.0)	0 (0.0)	2
The	0 (0.0)	1 (0.25)	0 (0.0)	1 (0.25)	0 (0.0)	2 (0.50)	0 (0.0)	4

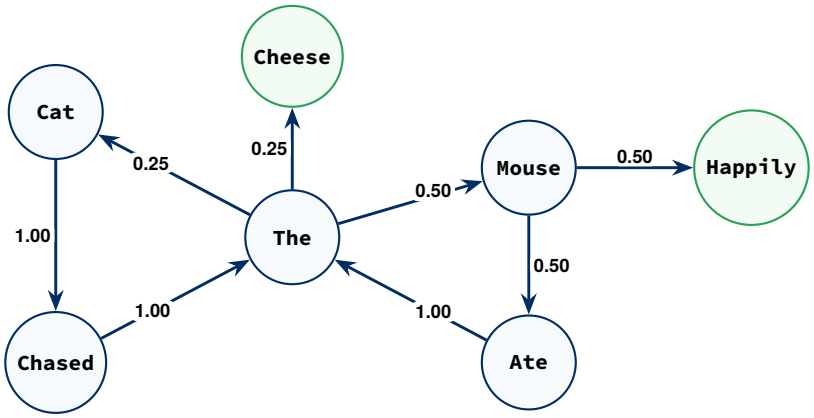


Figure 1.2: Markov state transition graph for the 7-token cat/mouse corpus. Directed edges indicate bigram transition probabilities $\mathbb{P}(w_t \mid w_{t-1})$. States `Cheese` and `Happily` (green) are terminal absorbing states with zero outgoing probability mass.

1.4 Autoregressive estimation and generation

With the transition matrix **M** constructed (Table 1.2) and its state graph mapped (Figure 1.2), the language model can execute two distinct operational tasks:

1. **Sequence Probability Estimation**: Assign a likelihood score to any proposed test sentence.
2. **Autoregressive Generation**: Incrementally sample tokens to produce novel synthetic text.

1.4.1 Worked sentence probability calculation

Consider evaluating the joint probability of the sentence:

"the cat chased the cheese"

Under the first-order Markov assumption, the joint probability factors as:

$$\mathbb{P}(\text{the, cat, chased, the, cheese}) = \mathbb{P}(\text{the})\cdot\mathbb{P}(\text{cat} \mid \text{the})\cdot\mathbb{P}(\text{chased} \mid \text{cat})\cdot\mathbb{P}(\text{the} \mid \text{chased})\cdot\mathbb{P}(\text{cheese} \mid \text{the}). \tag{1.9}$$

Assuming our generation begins deterministically with the initial token $\mathbb{P}(\text{the}) = 1.0$, we substitute the corresponding conditional probabilities from Table 1.2:

$$\begin{aligned}\mathbb{P}(\text{cat} \mid \text{the}) &= \frac{1}{4} = 0.25\\ \mathbb{P}(\text{chased} \mid \text{cat}) &= 1.0\\ \mathbb{P}(\text{the} \mid \text{chased}) &= 1.0\\ \mathbb{P}(\text{cheese} \mid \text{the}) &= \frac{1}{4} = 0.25\end{aligned}$$

Multiplying these values step by step:

$$\mathbb{P}(\text{the, cat, chased, the, cheese}) = 1.0 \times \frac{1}{4} \times 1 \times 1 \times \frac{1}{4} = \frac{1}{16} = 0.0625. \tag{1.10}$$

Despite the sequence *"the cat chased the cheese"* never occurring as an entire sentence in the training corpus, the bigram model assigns it a strictly positive probability of 6.25%. By decomposing text into local constituent pairs, n-gram models exhibit rudimentary combinatorial generalization.

1.4.2 5-step autoregressive generation trace

Now consider using the bigram model for open-ended text generation. Autoregressive sampling begins with a seed prefix, samples the next token from the resulting conditional probability distribution, appends the sampled token to the history, and repeats.

We trace the exact 5-step generation sequence recorded in the lecture slides, starting from the seed token *"the"*:

Step 1: Condition on prefix: history = ("the").

- Active row in transition matrix: Row "The".
- Candidate distribution: $\mathbb{P}(\text{cat} \mid \text{the}) = 0.25$, $\mathbb{P}(\text{cheese} \mid \text{the}) = 0.25$, $\mathbb{P}(\text{mouse} \mid \text{the}) = 0.50$.
- Random sample: **"mouse"**.
- Generated prefix: *"the mouse"*.

Step 2: Condition on prefix: history = (. . . , "mouse").

- Active row in transition matrix: Row "Mouse".
- Candidate distribution: $\mathbb{P}(\text{ate} \mid \text{mouse}) = 0.50$, $\mathbb{P}(\text{happily} \mid \text{mouse}) = 0.50$.
- Random sample: **"ate"**.
- Generated prefix: *"the mouse ate"*.

Step 3: Condition on prefix: history = (. . . , "ate").

- Active row in transition matrix: Row "Ate".
- Candidate distribution: $\mathbb{P}(\text{the} \mid \text{ate}) = 1.00$ (deterministic transition).
- Emitted token: **"the"**.
- Generated prefix: *"the mouse ate the"*.

Step 4: Condition on prefix: history = (. . . , "the").

- Active row in transition matrix: Row "The".
- Candidate distribution: $\mathbb{P}(\text{cat} \mid \text{the}) = 0.25$, $\mathbb{P}(\text{cheese} \mid \text{the}) = 0.25$, $\mathbb{P}(\text{mouse} \mid \text{the}) = 0.50$.
- Random sample: **"mouse"**.
- *Slide Note*: **:(** (A repetitive structural loop has emerged because the model cannot remember that it already emitted "mouse" two words ago).
- Generated prefix: *"the mouse ate the mouse"*.

Step 5: Condition on prefix: history = (. . . , "mouse").

- Active row in transition matrix: Row "Mouse".
- Candidate distribution: $\mathbb{P}(\text{ate} \mid \text{mouse}) = 0.50$, $\mathbb{P}(\text{happily} \mid \text{mouse}) = 0.50$.
- Random sample: **"happily"**.
- *Slide Note*: **(shocked face)** (A syntactically valid but bizarre, cannibalistic sentence is generated).
- Generated sequence: ***"The mouse ate the mouse happily"***.

Because the word *"happily"* has zero outgoing transitions in Table 1.2, the process reaches an absorbing terminal state and generation halts naturally.

Exam insights: Classroom Q&A: absorbing terminal states and infinite loops

During the lecture walkthrough, two crucial student questions were analyzed by the instructor:

1. **Question 1: What happens after words like "happily" or "cheese"?**
   In Table 1.2, terminal sentence-ending words have a row count of zero ($C(\text{Happily}) = 0$), making the conditional probability division $0/0$ mathematically undefined. In standard production language modeling, this is formalized by appending an explicit end-of-sequence token $\langle /s \rangle$ (or [EOS]) to every training sentence, guaranteeing that every word possesses a non-zero outgoing transition to either another word or $\langle /s \rangle$.

2. **Question 2: Why did greedy decoding enter an infinite cyclic loop?**
   Because bigram models condition exclusively on the single preceding token w_{t-1}, once the model transitions from *"ate"* to *"the"*, it retains zero memory that it already described the mouse eating cheese two steps earlier. If greedy decoding is used

(arg max), the transition path becomes fully deterministic and falls into a stationary limit cycle: *"the → mouse → ate → the → mouse → ..."*. Modern LLMs resolve this through long-context self-attention horizons, temperature-controlled stochastic sampling, and repetition penalties.

1.5 Fundamental limitations of statistical n-gram models: sparsity and zero-frequency collapse

While n-gram models provided the foundation for speech recognition, statistical machine translation, and spelling correction throughout the 1980s and 1990s, they suffer from fatal structural limitations that ultimately motivated the deep learning revolution.

1.5.1 Catastrophic data sparsity and parameter explosion

To condition on $n-1$ words, an n-gram model requires storing a discrete probability table of dimensionality $|\mathcal{V}|^n$. For realistic natural language and software applications:

- Vocabulary size: $|\mathcal{V}| \approx 50{,}000$ to $100{,}000$ unique words or code identifiers.
- Bigram table ($n = 2$): $|\mathcal{V}|^2 = 50{,}000^2 = 2.5 \times 10^9$ parameters ($\approx$ 10 GB at 32-bit floating point).
- Trigram table ($n = 3$): $|\mathcal{V}|^3 = 50{,}000^3 = 1.25 \times 10^{14}$ parameters (over 500 Terabytes).
- 4-gram table ($n = 4$): $|\mathcal{V}|^4 = 50{,}000^4 = 6.25 \times 10^{18}$ parameters (Exabyte scale).

Even if storage were free, the number of distinct 4-grams in all digitized human literature is dwarfed by 10^{18}. Consequently, the overwhelming majority of valid phrases never appear in training corpora.

1.5.2 The zero-frequency catastrophe: probability and perplexity collapse

The most lethal consequence of combinatorial sparsity is the *zero-frequency catastrophe*. Under Maximum Likelihood Estimation, any n-gram that never appeared verbatim in the training text receives count $C = 0$, yielding:

$$\mathbb{P}_{\text{MLE}}(w_t \mid w_{t-n+1}^{t-1}) = \frac{C(w_{t-n+1}^{t})}{C(w_{t-n+1}^{t-1})} = 0. \tag{1.11}$$

By the probabilistic chain rule (Equation (1.2)), evaluating a test sequence $W_{\text{test}} = (w_1, \ldots, w_N)$ requires taking the product across all constituent conditional transitions:

$$\mathbb{P}(W_{\text{test}}) = \prod_{t=1}^{N} \mathbb{P}(w_t \mid w_{<t}) = \mathbb{P}(w_1) \cdots 0 \cdots \mathbb{P}(w_N \mid w_{<N}) = 0. \tag{1.12}$$

A single unobserved token transition collapses the estimated joint probability of an entire document, article, or codebase to absolute zero.

Furthermore, when evaluating the model via Empirical Cross-Entropy $\mathcal{H}(W_{\text{test}})$ or Perplexity $\text{PPL}(W_{\text{test}})$:

$$\mathcal{H}(W_{\text{test}}) = -\frac{1}{N} \sum_{t=1}^{N} \log_2 \mathbb{P}(w_t \mid w_{<t}) \longrightarrow \infty, \quad \text{PPL}(W_{\text{test}}) = 2^{\mathcal{H}(W_{\text{test}})} \longrightarrow \infty. \tag{1.13}$$

An unobserved test n-gram results in infinite perplexity ($\text{PPL} = \infty$), rendering the entire language model statistically useless for evaluation or comparison.

1.5.3 Finite context horizon limitations

An n-gram model is strictly blind to any token that occurred more than $n-1$ steps in the past:

$$\mathbb{P}(w_t \mid w_1, w_2, \ldots, w_{t-1}) \approx \mathbb{P}(w_t \mid w_{t-n+1}, \ldots, w_{t-1}). \tag{1.14}$$

Because the conditioning window is rigidly restricted to the preceding $n-1$ tokens, any semantic constraint, topic coherence, or syntactic agreement established earlier in the discourse is discarded. This finite context horizon causes severe memory amnesia across two critical domains:

Linguistic agreement breakdown in natural language. Consider long-range syntactic agreements where an arbitrary number of modifiers, prepositional phrases, or subordinate relative clauses separate the grammatical subject from its governing predicate:

*"The **engineers** who designed the complex distributed compiler infrastructure [...] **were** vs **was** awarded the patent."*

If $n = 4$ (a 4-gram model), the model predicts the verb by conditioning exclusively on the three preceding tokens:

$$\mathbb{P}(\text{verb} \mid \text{compiler}, \text{infrastructure}, [\ldots]).$$

Because the singular noun *"infrastructure"* is immediately adjacent to the verb position, the local 4-gram transition statistics strongly favor the singular verb *"was"*. The model has completely forgotten the true plural antecedent subject *"engineers"*, producing an ungrammatical agreement violation.

Structural and scoping breakdown in software engineering code. In programming languages, code validity requires strict, long-range determinism governed by context-free grammars and scope resolution rules:

1. **Lexical Scoping and Variable Lifecycles**: A variable declared in a function header:

   ```
   void process_batch(std::vector<TransactionRequest>& transactions) { ...}
   ```

   must be invoked using valid `std::vector` member methods (e.g., `.push_back()`, `.size()`) thirty lines later. An n-gram model sees only `transactions.` without access to the declaration, leading to syntactically invalid completions.

2. **Bracket and Block Matching**: Program code requires exact hierarchical pairing of opening and closing delimiters (`{ ...}`, `[ ...]`, `( ...)`) and control flow constructs (`try ...catch ...finally`). Because n-gram models lack internal state stacks, they frequently emit mismatched braces or premature block terminations.

1.5.4 Discrete orthogonal symbols and lack of semantic generalization

In classical statistical NLP, words are treated as discrete, orthogonal atomic symbols. If the model observes:

*"The **cat** chased the mouse happily"*

it gains zero statistical confidence regarding the identical sentence containing synonyms:

*"The **feline** chased the mouse happily"*

Because *"cat"* and *"feline"* are completely separate row indices in Table 1.2, observations of one do not inform the other. There is no concept of semantic distance, synonymy, or conceptual overlap.

The causal imperative. While long-range memory amnesia and semantic blindness demand continuous neural architectures (introduced in Chapters 2 and 4), the immediate, fatal crisis in statistical language modeling is the **zero-frequency catastrophe** ($C = 0 \implies P = 0 \implies \text{PPL} = \infty$). Before we can even evaluate an n-gram model, we must establish rigorous mathematical algorithms that steal probability mass from frequent events and reallocate it to unobserved events: this is the domain of **statistical smoothing**.

1.6 Statistical smoothing and discounting algorithms

To prevent probability collapse ($P = 0$) and infinite perplexity ($\text{PPL} = \infty$) on unseen test sequences, statistical language modeling relies on *smoothing* (or discounting). The governing principle of smoothing is conservative probability reallocation: we shave off a fraction of probability mass from observed events and redistribute it across unseen events, ensuring that every valid sequence receives a strictly positive probability:

$$\forall w \in \mathcal{V}, \quad \mathbb{P}(w \mid \text{context}) > 0.$$

1.6.1 Laplace add-one and add-α smoothing

The simplest smoothing formulation, originated by Pierre-Simon Laplace, adds a uniform pseudo-count $\alpha > 0$ to every possible transition.

Mathematical formulation. For an unconditioned unigram model, Laplace add-α smoothing assigns:

$$\mathbb{P}_{\text{Laplace}}(w) = \frac{C(w) + \alpha}{N + \alpha|\mathcal{V}|}, \tag{1.15}$$

where $N = \sum_{w \in \mathcal{V}} C(w)$ is the total token count. For a bigram model conditioned on w_{t-1}, the estimator modifies Equation (1.8) as:

$$\mathbb{P}_{\text{Laplace}}(w_t \mid w_{t-1}) = \frac{C(w_{t-1}, w_t) + \alpha}{C(w_{t-1}) + \alpha|\mathcal{V}|}. \tag{1.16}$$

When $\alpha = 1$, the technique is known as *Laplace smoothing* (or Add-1); when $0 < \alpha < 1$, it is termed *Lidstone smoothing*.

Proof of proper probability distribution. We verify that Equation (1.16) satisfies the Kolmogorov axioms by summing over the entire vocabulary $\mathcal{V}$:

$$\begin{aligned}
\sum_{w_t \in \mathcal{V}} \mathbb{P}_{\text{Laplace}}(w_t \mid w_{t-1}) &= \sum_{w_t \in \mathcal{V}} \frac{C(w_{t-1}, w_t) + \alpha}{C(w_{t-1}) + \alpha|\mathcal{V}|} \\
&= \frac{\sum_{w_t \in \mathcal{V}} C(w_{t-1}, w_t) + \sum_{w_t \in \mathcal{V}} \alpha}{C(w_{t-1}) + \alpha|\mathcal{V}|} \\
&= \frac{C(w_{t-1}) + \alpha|\mathcal{V}|}{C(w_{t-1}) + \alpha|\mathcal{V}|} = 1.
\end{aligned} \tag{1.17}$$

Why Add-1 fails in practice. Although mathematically clean, Add-1 smoothing performs terribly in natural language processing. In realistic vocabularies where $|\mathcal{V}| = 50{,}000$, if word w_{t-1} occurred $C(w_{t-1}) = 100$ times, adding $\alpha|\mathcal{V}| = 50{,}000$ inflates the denominator from 100 to 50,100. A frequent observed transition with $C(w_{t-1}, w_t) = 50$ is diluted from an empirical probability of 0.50 to:

$$\frac{50 + 1}{100 + 50{,}000} = \frac{51}{50{,}100} \approx 0.00102.$$

Add-1 smoothing steals more than 99.8% of the probability mass from observed events and reallocates it uniformly to unobserved transitions. Even Lidstone smoothing ($\alpha = 0.05$) assigns the same probability to all unseen words, failing to recognize that some unseen words (e.g., common nouns) are inherently more probable than rare technical jargon.

1.6.2 Good-Turing discounting

Originally formulated in [6] during cryptographic research at Bletchley Park, *Good-Turing discounting* derives the probability of events seen r times directly from the count of events seen $r + 1$ times.

Frequency of frequencies. Let $r = C(w_{t-n+1}^{t}) \geq 0$ denote the empirical frequency of an n-gram. We define the *frequency of frequencies* N_r as the total number of distinct n-grams that appeared exactly r times in the training corpus:

$$N_r = |\{x : C(x) = r\}|. \tag{1.18}$$

The total number of tokens in the corpus is $N = \sum_{r=1}^{\infty} rN_r$, and N_0 represents the enormous number of valid n-grams that never appeared ($r = 0$).

Adjusted count derivation. Good-Turing replaces each raw count r with an effective discounted count r^*:

$$r^* = (r + 1)\frac{N_{r+1}}{N_r}. \tag{1.19}$$

For any observed n-gram with count $r > 0$, its Good-Turing smoothed probability is:

$$\mathbb{P}_{\text{GT}}(w_t \mid w_{t-n+1}^{t-1}) = \frac{r^*}{N} = \frac{(r + 1)N_{r+1}}{N \cdot N_r}. \tag{1.20}$$

The total probability mass reserved for all unobserved n-grams ($r = 0$) is determined by the proportion of *singletons* (N_1, words seen exactly once):

$$P_0 = \frac{N_1}{N}. \tag{1.21}$$

Because there are N_0 unobserved n-grams, each individual unseen n-gram receives probability:

$$\mathbb{P}_{\text{GT}}(\text{unseen}) = \frac{P_0}{N_0} = \frac{N_1}{N \cdot N_0}. \tag{1.22}$$

Smoothing N_r for large r. For large values of r, empirical counts become sparse (N_r fluctuates wildly or drops to zero, making N_{r+1}/N_r undefined). In production systems, *Simple Good-Turing* [7] replaces raw N_r values with smoothed estimates derived from a linear regression fit in log space ($\log N_r = a + b \log r$) for $r > k$.

1.6.3 Katz backoff with α normalization

The recursive framework introduced in [8] extended Good-Turing discounting through a hierarchical backoff mechanism: if an n-gram has been observed, use its discounted probability; if it has never been observed, back off to the $(n - 1)$-gram distribution, scaled by a normalizing factor.

Mathematical formulation. For a bigram model, the Katz backoff estimator is:

$$\mathbb{P}_{\text{Katz}}(w_t \mid w_{t-1}) = \begin{cases} \mathbb{P}^*(w_t \mid w_{t-1}) & \text{if } C(w_{t-1}, w_t) > 0, \\ \alpha(w_{t-1})\mathbb{P}_{\text{Katz}}(w_t) & \text{if } C(w_{t-1}, w_t) = 0, \end{cases} \tag{1.23}$$

where $\mathbb{P}^*(w_t \mid w_{t-1}) = \frac{C^*(w_{t-1}, w_t)}{C(w_{t-1})}$ represents the discounted probability calculated using Good-Turing adjusted counts for low frequencies ($r \leq k$, typically $k = 5$) and MLE for high counts ($r > k$):

$$C^*(w_{t-1}, w_t) = \frac{\frac{r^*}{r} - \frac{(k+1)N_{k+1}}{N_1}}{1 - \frac{(k+1)N_{k+1}}{N_1}} \cdot r \quad \text{for } 1 \leq r \leq k. \tag{1.24}$$

Derivation of the normalizing backoff weight $\alpha(w_{t-1})$. The leftover probability mass shaved off from observed bigrams is:

$$\beta(w_{t-1}) = 1 - \sum_{w_t : C(w_{t-1}, w_t) > 0} \mathbb{P}^*(w_t \mid w_{t-1}). \tag{1.25}$$

To ensure that $\sum_{w_t \in \mathcal{V}} \mathbb{P}_{\text{Katz}}(w_t \mid w_{t-1}) = 1$, the backoff weight $\alpha(w_{t-1})$ scales the lower-order unigram distribution over unobserved words:

$$\alpha(w_{t-1}) = \frac{\beta(w_{t-1})}{\sum_{w_t : C(w_{t-1}, w_t) = 0} \mathbb{P}_{\text{Katz}}(w_t)}. \tag{1.26}$$

Summing across the entire vocabulary confirms proper normalization:

$$\begin{aligned} \sum_{w_t \in \mathcal{V}} \mathbb{P}_{\text{Katz}}(w_t \mid w_{t-1}) &= \sum_{C>0} \mathbb{P}^*(w_t \mid w_{t-1}) + \sum_{C=0} \alpha(w_{t-1})\mathbb{P}_{\text{Katz}}(w_t) \\ &= \big(1 - \beta(w_{t-1})\big) + \alpha(w_{t-1}) \sum_{C=0} \mathbb{P}_{\text{Katz}}(w_t) \\ &= \big(1 - \beta(w_{t-1})\big) + \beta(w_{t-1}) = 1. \end{aligned} \tag{1.27}$$

1.6.4 Kneser-Ney smoothing and continuation probabilities

The pinnacle of statistical language modeling is *Kneser-Ney smoothing* [9]. Kneser-Ney introduces two transformative innovations: absolute discounting and the continuation probability.

Absolute discounting. Rather than scaling counts proportionally, empirical analysis of corpus frequencies demonstrates that subtracting a fixed constant $d \in (0, 1)$ (typically $d \approx 0.75$) from all positive counts accurately models the true discounted frequency:

$$C_{\text{discounted}} = \max\left(C(w_{t-n+1}^t) - d,\, 0\right).$$

The continuation probability failure mode of standard backoff. Consider predicting the missing word in the sentence:

"I can't see without my reading _____"

The word *"glasses"* should receive a high conditional probability. Now consider the word *"Francisco"*. In any standard text corpus, *"Francisco"* has a high unigram frequency because the proper noun *"San Francisco"* appears frequently. Under Katz backoff, because the bigram *"reading Francisco"* has zero count, the model backs off to the unigram frequency of *"Francisco"*, assigning it a dangerously high probability of completing the sentence!

The breakthrough insight in [9] recognized that the unigram backoff distribution should not reflect *how often* a word appears overall, but *how versatile* the word is across diverse preceding contexts.

Definition 1.4: *Continuation probability*

The *continuation probability* $\mathbb{P}_{\text{cont}}(w_t)$ is the ratio of the number of unique preceding word types that precede w_t to the total number of unique bigram types in the vocabulary:

$$\mathbb{P}_{\text{cont}}(w_t) = \frac{|\{w_{t-1} : C(w_{t-1}, w_t) > 0\}|}{\sum_{w'} |\{w_{t-1} : C(w_{t-1}, w') > 0\}|} = \frac{|\{w_{t-1} : C(w_{t-1}, w_t) > 0\}|}{|\{(w_{t-1}, w_t) : C(w_{t-1}, w_t) > 0\}|}. \tag{1.28}$$

Because *"Francisco"* almost exclusively appears preceded by the single word *"San"*, $|\{w_{t-1} : C(w_{t-1}, \text{Francisco}) > 0\}| \approx 1$, resulting in a tiny continuation probability. In contrast, words like *"glasses"* or *"book"* appear in dozens of distinct contexts, yielding high continuation probabilities.

Interpolated Kneser-Ney formulation. Combining absolute discounting with continuation probabilities yields the *Interpolated Kneser-Ney* estimator:

$$\mathbb{P}_{\text{KN}}(w_t \mid w_{t-n+1}^{t-1}) = \frac{\max\left(C(w_{t-n+1}^{t}) - d,\, 0\right)}{C(w_{t-n+1}^{t-1})} + \lambda(w_{t-n+1}^{t-1})\mathbb{P}_{\text{KN}}(w_t \mid w_{t-n+2}^{t-1}), \tag{1.29}$$

where $\lambda(w_{t-n+1}^{t-1})$ is an explicit normalizing interpolation weight:

$$\lambda(w_{t-n+1}^{t-1}) = \frac{d}{C(w_{t-n+1}^{t-1})} \cdot \left|\{w : C(w_{t-n+1}^{t-1}, w) > 0\}\right|. \tag{1.30}$$

For a bigram model ($n = 2$), the base recursion terminates at the unigram continuation probability:

$$\mathbb{P}_{\text{KN}}(w_t \mid w_{t-1}) = \frac{\max\left(C(w_{t-1}, w_t) - d,\, 0\right)}{C(w_{t-1})} + \frac{d}{C(w_{t-1})} \cdot |\{w : C(w_{t-1}, w) > 0\}| \cdot \mathbb{P}_{\text{cont}}(w_t). \tag{1.31}$$

Proof of proper normalization. Summing Equation (1.31) over all $w_t \in \mathcal{V}$:

$$\begin{aligned}
\sum_{w_t \in \mathcal{V}} \mathbb{P}_{\text{KN}}(w_t \mid w_{t-1}) &= \sum_{w_t : C>0} \frac{C(w_{t-1}, w_t) - d}{C(w_{t-1})} + \lambda(w_{t-1}) \sum_{w_t \in \mathcal{V}} \mathbb{P}_{\text{cont}}(w_t) \\
&= \frac{\sum_{C>0} C(w_{t-1}, w_t) - d \cdot |\{w : C(w_{t-1}, w) > 0\}|}{C(w_{t-1})} + \lambda(w_{t-1}) \cdot 1 \\
&= \frac{C(w_{t-1}) - d|\{w : C(w_{t-1}, w) > 0\}|}{C(w_{t-1})} + \frac{d|\{w : C(w_{t-1}, w) > 0\}|}{C(w_{t-1})} = 1.
\end{aligned} \tag{1.32}$$

Interpolated Kneser-Ney consistently outperforms all other statistical smoothing techniques, serving as the industry standard until the advent of deep learning.

The causal imperative. Now that we have established competing smoothing algorithms (Laplace, Good-Turing, Katz, Kneser-Ney), each reallocating probability mass under different mathematical assumptions, we face a crucial practical question: *How do we quantitatively benchmark and compare these models on held-out evaluation text without running costly downstream applications?* This brings us directly to formal evaluation metrics.

1.7 Evaluation metrics: cross-entropy and perplexity

To benchmark and compare different language models quantitatively without executing costly downstream applications, natural language processing relies on intrinsic, information-theoretic metrics: *Cross-Entropy* and *Perplexity* [4, 5].

1.7.1 Information-theoretic foundations: entropy and KL divergence

Let $\mathcal{X}$ denote a discrete sample space (the vocabulary $\mathcal{V}$), and let $P(x)$ represent the true, underlying, but inaccessible probability distribution of human language.

Definition 1.5: *Shannon entropy*

The *Shannon entropy* $H(P)$ measures the fundamental, irreducible uncertainty or average information content inherent in distribution P [4]:

$$H(P) = -\sum_{x \in \mathcal{X}} P(x) \log_2 P(x) = \mathop{\mathbb{E}}_{x \sim P} \left[\log_2 \frac{1}{P(x)} \right]. \tag{1.33}$$

By Shannon's source coding theorem, $H(P)$ represents the absolute theoretical lower bound on the expected number of bits required to encode a symbol emitted by P without loss.

When constructing a language model, we approximate the true distribution P with an estimated parameterized distribution Q_θ. To quantify the statistical discrepancy between the true distribution P and our approximating model Q_θ, we employ the *Kullback-Leibler (KL) divergence*.

Definition 1.6: *Kullback-Leibler (KL) divergence*

The *KL divergence* (or relative entropy) from model distribution Q to target distribution P is:

$$D_{\mathrm{KL}}(P \parallel Q) = \sum_{x \in \mathcal{X}} P(x) \log_2 \left(\frac{P(x)}{Q(x)} \right) = \mathop{\mathbb{E}}_{x \sim P} \left[\log_2 \frac{P(x)}{Q(x)} \right]. \tag{1.34}$$

Theorem 1.1: *Gibbs' inequality*

For any two discrete probability distributions P and Q, the KL divergence is non-negative:

$$D_{\mathrm{KL}}(P \parallel Q) \geq 0, \tag{1.35}$$

with equality $D_{\mathrm{KL}}(P \parallel Q) = 0$ if and only if $P(x) = Q(x)$ for all $x \in \mathcal{X}$.

Proof. Applying the natural logarithm and utilizing the standard inequality $\ln(u) \leq u - 1$ for all $u > 0$ (with equality if and only if $u = 1$):

$$\begin{aligned} -D_{\mathrm{KL}}(P \parallel Q) &= \sum_{x \in \mathcal{X}} P(x) \log_2 \left(\frac{Q(x)}{P(x)} \right) = \frac{1}{\ln 2} \sum_{x \in \mathcal{X}} P(x) \ln \left(\frac{Q(x)}{P(x)} \right) \\ &\leq \frac{1}{\ln 2} \sum_{x \in \mathcal{X}} P(x) \left(\frac{Q(x)}{P(x)} - 1 \right) \\ &= \frac{1}{\ln 2} \left(\sum_{x \in \mathcal{X}} Q(x) - \sum_{x \in \mathcal{X}} P(x) \right) = \frac{1}{\ln 2} (1 - 1) = 0. \end{aligned}$$

Multiplying both sides by -1 reverses the inequality, yielding $D_{\mathrm{KL}}(P \parallel Q) \geq 0$. ■

1.7.2 Cross-entropy decomposition

Expanding the logarithm in the definition of KL divergence demonstrates the fundamental relationship connecting Entropy, KL Divergence, and Cross-Entropy:

$$\begin{aligned} D_{\mathrm{KL}}(P \parallel Q) &= \sum_{x \in \mathcal{X}} P(x) \log_2 P(x) - \sum_{x \in \mathcal{X}} P(x) \log_2 Q(x) \\ &= -H(P) + \left(- \sum_{x \in \mathcal{X}} P(x) \log_2 Q(x) \right). \end{aligned} \tag{1.36}$$

Rearranging terms defines the *Cross-Entropy* $H(P, Q)$:

$$H(P, Q) \triangleq - \sum_{x \in \mathcal{X}} P(x) \log_2 Q(x) = H(P) + D_{\mathrm{KL}}(P \parallel Q). \tag{1.37}$$

Why optimizing cross-entropy optimizes KL divergence. Equation (1.37) reveals the theoretical justification for cross-entropy training:

- The true language distribution P is determined by human linguistic behavior; therefore, the true entropy $H(P)$ is a fixed, immutable physical constant.
- When we minimize cross-entropy with respect to model parameters θ:

$$\arg\min_{\theta} H(P, Q_\theta) = \arg\min_{\theta} \left(H(P) + D_{\mathrm{KL}}(P \parallel Q_\theta) \right) = \arg\min_{\theta} D_{\mathrm{KL}}(P \parallel Q_\theta). \tag{1.38}$$

Minimizing cross-entropy is mathematically identical to driving the KL divergence between model Q_θ and the true language distribution P to zero.

1.7.3 Empirical sequence cross-entropy

Because we do not have closed-form access to the true distribution $P(x)$, we cannot compute the expectation $\mathbb{E}_{x \sim P}[\cdot]$ directly. Instead, by the Shannon-McMillan-Breiman theorem (the ergodic theorem for stationary information sources), the expected value is estimated empirically by sampling a long test sequence $W = (w_1, w_2, \ldots, w_T) \sim P$:

$$\mathcal{H}(W) = -\frac{1}{T} \log_2 Q(w_1, w_2, \ldots, w_T) = -\frac{1}{T} \sum_{t=1}^{T} \log_2 Q(w_t \mid w_{<t}). \tag{1.39}$$

Empirical cross-entropy measures the average number of bits required to compress each token of test text using our model Q.

1.7.4 Perplexity (PPL)

While cross-entropy evaluates performance on a logarithmic scale, researchers and practitioners overwhelmingly report *Perplexity* (PPL), defined as the exponentiated cross-entropy loss:

$$\mathrm{PPL}(W) = 2^{\mathcal{H}(W)} = 2^{-\frac{1}{T}\sum_{t=1}^{T}\log_2 Q(w_t|w_{<t})} \tag{1.40}$$

$$= \left(\prod_{t=1}^{T}\frac{1}{Q(w_t \mid w_{<t})}\right)^{\frac{1}{T}}. \tag{1.41}$$

Perplexity represents the reciprocal geometric mean of the token probabilities assigned by the model across the test corpus.

1.7.5 Rigorous proof: perplexity as the effective branching factor

The fundamental intuition taught across NLP literature is that perplexity quantifies the *effective branching factor* of a language model. We now formalize this property through a rigorous proof.

Theorem 1.2: *Perplexity of a uniform K-ary distribution*

Let a language model Q predict next tokens such that at each timestep t, the probability mass is distributed uniformly across exactly K equally likely candidate tokens ($Q(w_t \mid w_{<t}) = \frac{1}{K}$ for each candidate). Then the perplexity on any generated sequence W of length T is identically K:

$$\mathrm{PPL}(W) = K. \tag{1.42}$$

Proof. Substituting the uniform prediction probability $Q(w_t \mid w_{<t}) = \frac{1}{K}$ into the empirical cross-entropy formula:

$$\mathcal{H}(W) = -\frac{1}{T}\sum_{t=1}^{T}\log_2\left(\frac{1}{K}\right) = -\frac{1}{T}\sum_{t=1}^{T}\left(-\log_2 K\right)$$
$$= \frac{1}{T}\left(T\log_2 K\right) = \log_2 K.$$

Exponentiating the cross-entropy:

$$\mathrm{PPL}(W) = 2^{\mathcal{H}(W)} = 2^{\log_2 K} = K. \quad \blacksquare$$

Physical interpretation. When a model evaluates a text corpus and achieves PPL = 42, it indicates that, on average, the model's predictive uncertainty at each token is mathematically equivalent to choosing uniformly at random from a shortlist of 42 equally plausible words:

- A uniform random guesser across vocabulary $\mathcal{V}$ achieves PPL = $|\mathcal{V}|$ (e.g., PPL = 50,000).
- An optimal oracle that predicts every token with probability 1.0 achieves the theoretical lower bound PPL = 1.0.
- State-of-the-art LLMs on natural language benchmarks typically achieve perplexities between 8 and 15.

Exam insights: Base-2 vs base-e logarithmic conversions and worked exam example

A recurrent point of confusion in oral and written examinations at Politecnico di Torino is mixing base-2 information theory with base-e deep learning frameworks:

Property	Information Theory (Bits)	Deep Learning / PyTorch (Nats)
Loss Formula	$\mathcal{H}_2 = -\frac{1}{T}\sum \log_2 Q(w_t \mid w_{<t})$	$\mathcal{L}_{\text{CE}} = -\frac{1}{T}\sum \ln Q(w_t \mid w_{<t})$
Unit	Bits per token	Nats per token
Perplexity Formula	$\text{PPL} = 2^{\mathcal{H}_2}$	$\text{PPL} = \exp(\mathcal{L}_{\text{CE}}) = e^{\mathcal{L}_{\text{CE}}}$
Conversion Identity	$\mathcal{H}_2 = \frac{\mathcal{L}_{\text{CE}}}{\ln 2} \approx \frac{\mathcal{L}_{\text{CE}}}{0.693147}$,	$\mathcal{L}_{\text{CE}} = \mathcal{H}_2 \cdot \ln 2 \approx 0.693147 \cdot \mathcal{H}_2$

Worked exam calculation. Suppose a PyTorch model trained on code completion reports an evaluation loss of $\mathcal{L}_{\text{CE}} = 2.7726$ nats.

1. **Direct Base-e Perplexity**:

$$\text{PPL} = e^{2.7726} = \exp(2.7726) \approx 16.00.$$

2. **Conversion to Information Bits**:

$$\mathcal{H}_2 = \frac{2.7726}{\ln 2} = \frac{2.7726}{0.693147} = 4.00 \text{ bits/token}.$$

3. **Base-2 Perplexity Verification**:

$$\text{PPL} = 2^{\mathcal{H}_2} = 2^{4.00} = 16.00.$$

Common Exam Error: If a student incorrectly evaluates $2^{\mathcal{L}_{\text{CE}}} = 2^{2.7726} \approx 6.83$, they underestimate the model's true branching factor by more than 57%. Always verify whether the logarithm used is base-2 or base-e before computing perplexity.

1.7.6 The glass ceiling of smoothed n-gram models

While Kneser-Ney smoothing dramatically reduces test perplexity by eliminating zero-frequency collapses, smoothed n-gram models remain fundamentally handicapped:

1. **Horizon Truncation**: Conditioning remains locked to $n-1$ tokens; increasing $n \geq 5$ yields negligible gains because higher-order frequencies are almost entirely unobserved, causing the model to constantly back off to lower orders.

2. **Zero Parameter Sharing**: Words remain orthogonal indices. The model cannot generalize that *"The programmer wrote a test"* informs *"The developer authored a test"*.

3. **Combinatorial Storage**: Counting tables for large corpora require gigabytes to terabytes of RAM.

These structural limits motivated researchers to explore new paradigms that replace discrete counting tables with continuous, parameterized functions.

1.8 Historical evolution of language models

Understanding modern large language models requires appreciating the 75-year evolutionary trajectory that progressively dismantled the mathematical and computational barriers of discrete symbol counting. Figure 1.3 synthesizes the macro-historical progression of language modeling from Claude Shannon's foundational 1948 information theory to modern autonomous software engineering systems.

1.8.1 1948–1980s: Shannon's information theory and Markov n-grams

The conceptual origin of language modeling stems from the landmark 1948 paper *"A Mathematical Theory of Communication"* [4]. In that work, written human language was treated as a discrete stochastic process generated by an underlying stationary Markov source over a finite alphabet.

Shannon introduced the *Shannon game*—a pedagogical thought experiment where an observer attempts to predict the next letter in an unknown sequence of text given the preceding letters. Shannon demonstrated that as the conditioning context length increases from zero-order (independent uniform character selection) to unigram (individual character frequencies), bigram (pairs), and trigram (triplets) models, the generated text rapidly converges toward natural English orthography and syntax.

In this framework, the conditional probability of token w_t given context $w_1, \ldots, w_{t-1}$ is simplified via an order-$(n-1)$ Markov assumption, as illustrated in Figure 1.4 for code syntax.

1.8.2 1990s: statistical decision trees and context partitioning

Throughout the 1990s, as speech recognition and machine translation systems scaled, researchers recognized that flat n-gram tables could not condition on long histories without exponential parameter explosion.

To overcome this limitation in the course progression, *decision-tree language models* were introduced in [10] (as examined on Slide 19 of the course lecture deck). Rather than conditioning probability on a rigid, exact sequence of $n-1$ tokens, a binary decision tree asks a sequence of syntactic and lexical questions about the historical context $\mathcal{H} = (w_{t-1}, w_{t-2}, \ldots)$.

As illustrated in Figure 1.5, context histories that share identical predictive characteristics are clustered into the same leaf node $\mathcal{L}_k$, yielding conditional distributions $\mathbb{P}(w_t \mid \mathcal{L}_k)$. While this reduced the number of free parameters compared to full tables, the model remained fundamentally bound to discrete categorical splitting without true semantic interpolation.

1.8.3 2000–2003: The neural probabilistic language model

In the landmark 2003 paper *"A Neural Probabilistic Language Model"* [11], natural language processing was revolutionized by introducing continuous distributed representations to defeat the curse of dimensionality (as highlighted on Slide 20 of the course lectures).

The neural probabilistic formulation in [11] replaced discrete count tables with a continuous parameterized architecture (Figure 1.6):

1. **Continuous Embedding Projection**: Each word $w \in \mathcal{V}$ is associated with a real-valued distributed feature vector $\mathbf{c}_w \in \mathbb{R}^d$ ($d \ll |\mathcal{V}|$, e.g., $d = 100$). The vocabulary lookup matrix $\mathbf{C} \in \mathbb{R}^{|\mathcal{V}| \times d}$ is learned jointly via backpropagation.

2. **Semantic Smoothing via Continuous Space**: If *"cat"* and *"feline"* have similar embedding vectors ($\mathbf{c}_{\text{cat}} \approx \mathbf{c}_{\text{feline}}$), observing *"the cat chased the mouse"* automatically increases the model's confidence in *"the feline chased the mouse"*. Continuous geometry solves the orthogonal symbol limitation.

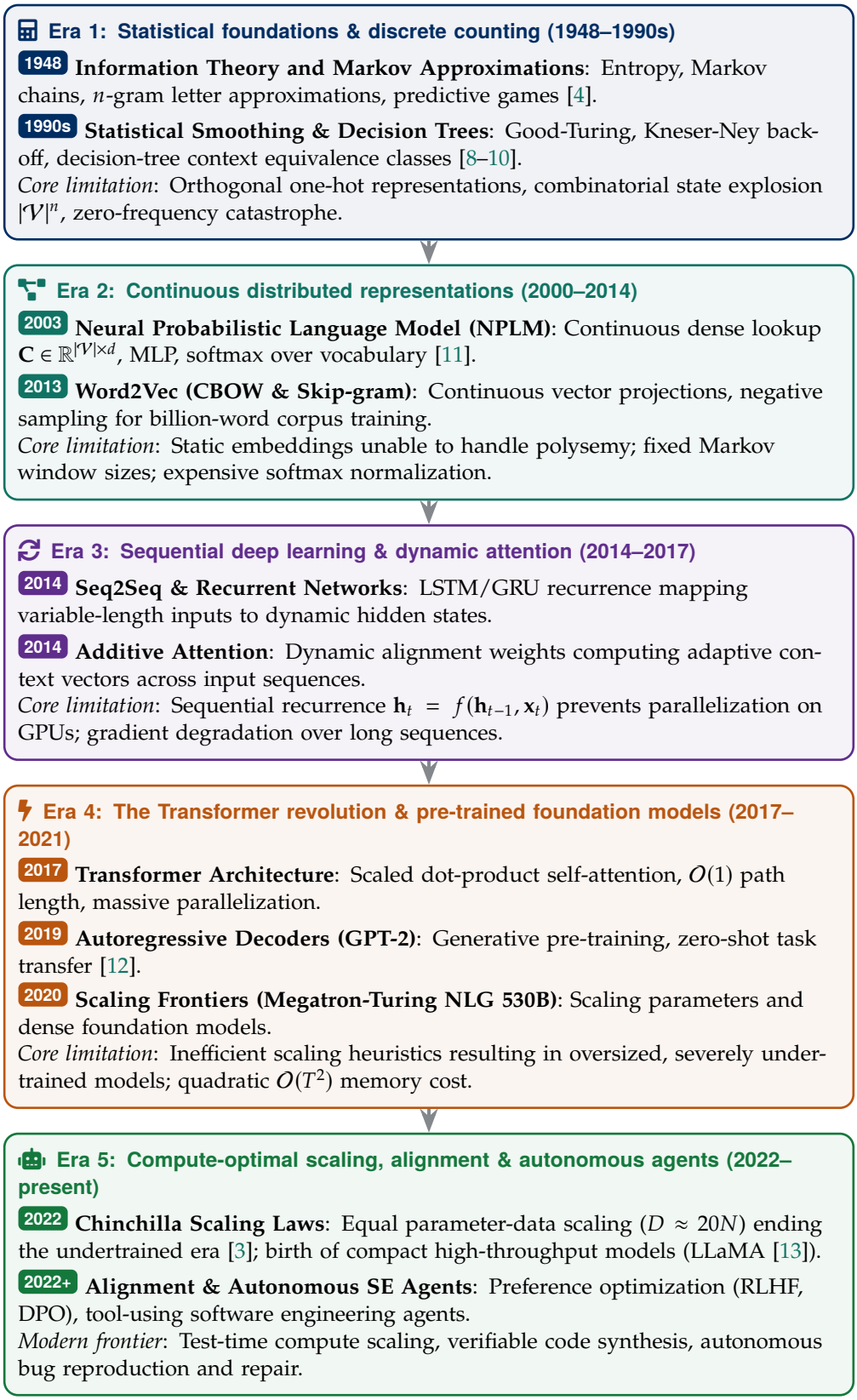


Figure 1.3: Chronological timeline of language model evolution from 1948 to the present.

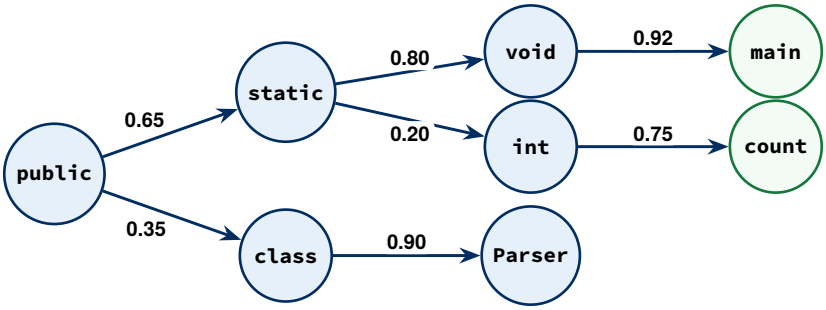


Figure 1.4: Markov chain state transition graph for software tokens.

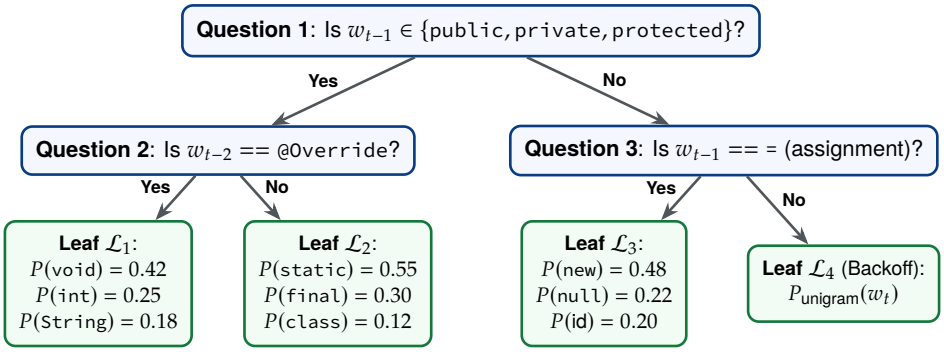


Figure 1.5: Decision tree context partitioning for conditional language modeling [10].

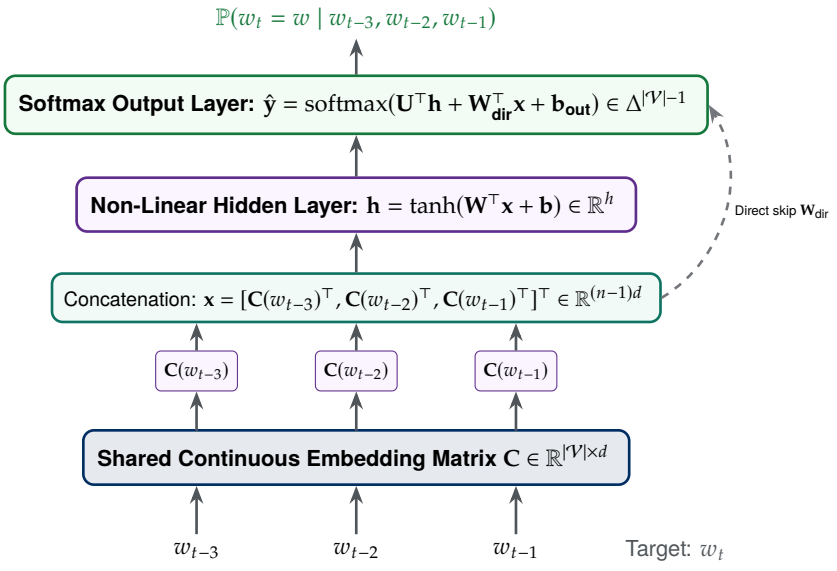


Figure 1.6: The Neural Probabilistic Language Model (NPLM) architecture [11]. Discrete context words are mapped into continuous dense feature vectors via shared matrix $\mathbf{C} \in \mathbb{R}^{|\mathcal{V}| \times d}$, concatenated, and processed through a non-linear hidden layer to predict next-token probabilities via softmax.

3. **Softmax Output Normalization**: The network projects hidden activations to $|\mathcal{V}|$ unnormalized logits and applies the softmax function to guarantee a valid probability distribution:

$$\mathbb{P}(w_t = i \mid w_{<t}) = \frac{\exp(z_i)}{\sum_{j=1}^{|\mathcal{V}|} \exp(z_j)}. \tag{1.43}$$

1.8.4 2018–2019: Generative pre-training and GPT-2

While continuous representations were established in [11], models throughout the 2000s and 2010s remained restricted in scale and context length. In 2018–2019, the GPT-2 architecture was introduced in [12] (*"Language Models are Unsupervised Multitask Learners"*, examined on Slide 21 of the course slides).

GPT-2 demonstrated that massive, autoregressive decoder-only models trained on hundreds of gigabytes of uncurated internet text (WebText) could perform diverse natural language and software tasks without task-specific supervised training. By predicting next tokens over context windows of 1,024 tokens, the model exhibited emergent zero-shot transfer across reading comprehension, machine translation, and Python code generation.

1.8.5 2020–2022: Megatron-Turing NLG 530B and compute-optimal scaling

Following GPT-2 and GPT-3, the deep learning industry initiated an aggressive scaling arms race, culminating in monolithic models such as the Megatron-Turing NLG 530B (Slide 22). However, early scaling heuristics [14] recommended scaling model parameters faster than training data. Consequently, models like Megatron-Turing NLG 530B and GPT-3 (175B parameters trained on 300B tokens) were severely **undertrained** relative to their parameter capacity.

In 2022, the landmark study *"Training Compute-Optimal Large Language Models"* [3] established the **Chinchilla** scaling laws:

$$D \approx 20 \times N, \tag{1.44}$$

where N is parameter count and D is training tokens. For compute optimality, parameter count and training tokens must scale in equal proportion. This insight prompted the modern era of compact, heavily trained open-weight models (e.g., LLaMA [13] trained on trillions of tokens), dramatically reducing deployment and inference overhead in software engineering environments.

1.8.6 From discrete counting tables to continuous parametric learners: bridge to deep learning

Our exploration of statistical language models in this chapter has revealed a fundamental evolutionary trajectory:

- **The Discrete Counting Limit**: Exact probability factorization via the chain rule (Equation (1.2)) is theoretically exact, but empirical n-gram counting tables suffer from exponential state-space explosion (V^n), context horizon amnesia ($n-1$), and orthogonal semantic blindness.
- **The Smoothing Ceiling**: While smoothing algorithms (Good-Turing, Katz, Kneser-Ney) rescue models from zero-frequency collapse ($P = 0 \implies \text{PPL} = \infty$), they merely redistribute probability mass across discrete table bins. They cannot learn high-dimensional feature interactions or share statistical strength between synonymous phrases.

To transcend the discrete counting paradigm, natural language processing abandoned frequency tables in favor of **continuous, parameterized neural networks**:

$$f(\mathbf{x}; \boldsymbol{\theta}) = \mathbf{W}^\top \mathbf{x} + \mathbf{b}.$$

Instead of counting co-occurrences in text, we parameterize language predictors as continuous functions whose weights $\boldsymbol{\theta}$ are optimized iteratively through gradient descent and backpropagation.

This transition requires establishing the mathematical and computational foundations of deep learning: How does an artificial neuron compute? Why is stacking linear layers without non-linear activations mathematically futile? What allows multi-layer perceptrons to approximate arbitrary complex functions? And how does backpropagation automatically compute gradients across computational graphs?

We address each of these foundational questions in Chapter 2: *Deep Learning Foundations for NLP*.

CHAPTER 2

Deep learning foundations for NLP

2.1 From biological neurons to the artificial perceptron

Having established the probabilistic formulation of language modeling and the combinatorial limitations of discrete symbol counting in Chapter 1, we now develop the neural computational foundations that allow language models to scale. In contemporary Natural Language Processing and AI-assisted Software Engineering, next-token prediction is reformulated as a high-dimensional, non-linear classification problem over continuous vector spaces. Rather than storing astronomical tables of discrete counts, we parameterize language predictors as continuous functions whose weights are optimized via gradient descent.

The foundational building block of modern deep learning is the *perceptron*, originally inspired by biological neurons in the human nervous system. A biological neuron receives electrochemical signals from upstream cells through branching *dendrites*, integrates these incoming potentials in its central cell body (*soma*), and, when the accumulated electrical voltage exceeds a physiological threshold (the action potential), fires a spike through its *axon* to downstream synapses.

In computational neural networks, this biological mechanism was first formalized in 1943 [15] as a threshold logic unit and subsequently refined in 1958 [16] into the trainable *Perceptron* (examined on Slide 7 of the course deep learning foundations).

Definition 2.1: *The artificial perceptron*

Let $\mathbf{x} = [x_1, x_2, \ldots, x_n]^\top \in \mathbb{R}^n$ represent an input feature vector, and let $\mathbf{w} = [w_1, w_2, \ldots, w_n]^\top \in \mathbb{R}^n$ represent the corresponding parameter weights, with scalar bias $w_0 = b \in \mathbb{R}$. The artificial perceptron computes:

$$y = f\left(w_0 + \sum_{i=1}^{n} w_i x_i\right) = f\left(w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_n x_n\right), \tag{2.1}$$

where $f : \mathbb{R} \to \mathbb{R}$ is the activation function. By adopting the standard convention of augmenting the input vector with a constant dummy feature $x_0 = 1$ and defining the parameter vector as $\mathbf{w} = [w_0, w_1, \ldots, w_n]^\top \in \mathbb{R}^{n+1}$, Equation (2.1) is expressed in compact vector inner-product notation:

$$y = f\left(\sum_{i=0}^{n} w_i x_i\right) = f\left(\mathbf{w}^\top \mathbf{x}\right) \quad \text{with } x_0 = 1,\ w_0 = \text{bias}. \tag{2.2}$$

Figure 2.1 illustrates the structural mechanics of the artificial perceptron: input signals are weighted by synaptic connection strengths, summed at a central junction, and passed through an activation function to generate the output signal.

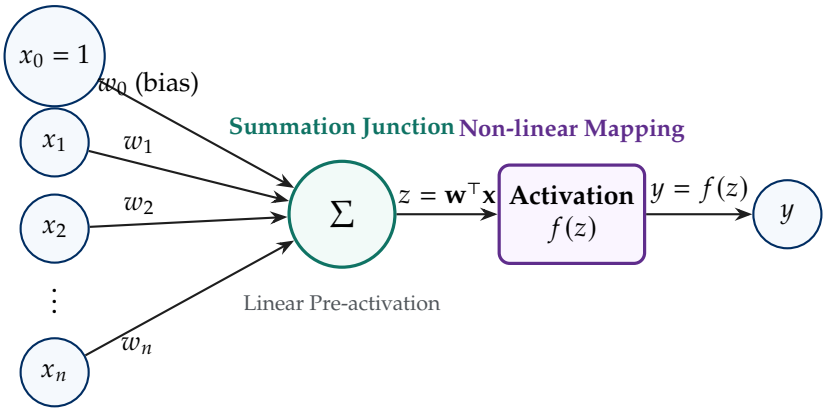


Figure 2.1: Architectural schematic of the artificial perceptron. Inputs x_i are modulated by synaptic weights w_i, accumulated with bias w_0 at the summation junction Σ to yield pre-activation $z = \mathbf{w}^\top \mathbf{x}$, and mapped through activation function $f(z)$ to produce scalar output y.

2.1.1 The perceptron in two dimensions: linear decision hyperplane

To establish geometric intuition, consider a two-dimensional feature space $\mathbf{x} = [x_1, x_2]^\top \in \mathbb{R}^2$ with weights $\mathbf{w} = [w_1, w_2]^\top$ and bias w_0. The pre-activation value is:

$$z(\mathbf{x}) = w_1 x_1 + w_2 x_2 + w_0. \tag{2.3}$$

When the perceptron acts as a binary classifier using a step or sign activation ($y = \text{sgn}(z)$), the decision boundary separating the two output classes is precisely the set of points where the pre-activation vanishes:

$$\mathcal{H} = \left\{ \mathbf{x} \in \mathbb{R}^2 \mid w_1 x_1 + w_2 x_2 + w_0 = 0 \right\}. \tag{2.4}$$

Geometric interpretation of weights and bias. In two dimensions, Equation (2.4) defines a straight line in the (x_1, x_2) Cartesian plane:

- **Normal Vector**: The weight vector $\mathbf{w} = [w_1, w_2]^\top$ is strictly perpendicular (orthogonal) to the separating line $\mathcal{H}$. For any two points $\mathbf{x}_a, \mathbf{x}_b \in \mathcal{H}$:

  $$\mathbf{w}^\top \mathbf{x}_a + w_0 = 0 \quad \text{and} \quad \mathbf{w}^\top \mathbf{x}_b + w_0 = 0 \implies \mathbf{w}^\top (\mathbf{x}_a - \mathbf{x}_b) = 0.$$

  The vector $\mathbf{w}$ points directly into the positive decision region ($z > 0$).

- **Orthogonal Distance to Origin**: The shortest Euclidean distance from the origin $\mathbf{0} = [0, 0]^\top$ to the separating hyperplane $\mathcal{H}$ is:

  $$d = \frac{|w_0|}{\|\mathbf{w}\|_2} = \frac{|w_0|}{\sqrt{w_1^2 + w_2^2}}. \tag{2.5}$$

  The bias w_0 shifts the hyperplane parallel to itself: if $w_0 = 0$, the decision boundary passes exactly through the origin; adjusting w_0 translates the boundary without altering its orientation.

Figure 2.2 diagrams this geometric relationship in $\mathbb{R}^2$.

2.1.2 Connection to classical linear and logistic regression

The artificial perceptron unifies two foundational statistical models studied in classical data analysis:

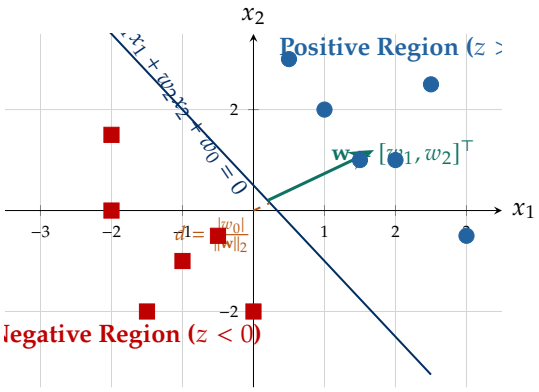


Figure 2.2: Geometric representation of the perceptron decision boundary in $\mathbb{R}^2$. The line $w_1 x_1 + w_2 x_2 + w_0 = 0$ partitions feature space into positive ($\mathcal{H}^+$) and negative ($\mathcal{H}^-$) half-spaces. The weight vector $\mathbf{w}$ defines the normal vector to the separating hyperplane, and the scalar bias w_0 determines its distance from the origin.

- **Linear Regression**: If the activation function is the linear identity $f(z) = z$, Equation (2.2) computes:
$$y = \mathbf{w}^\top \mathbf{x},$$
predicting an unbounded real-valued continuous response variable.
- **Logistic Regression**: If the activation function is the logistic sigmoid $f(z) = \sigma(z) = \frac{1}{1+e^{-z}}$, Equation (2.2) computes:
$$y = \sigma(\mathbf{w}^\top \mathbf{x}) = \mathbb{P}(Y = 1 \mid \mathbf{x}),$$
predicting the posterior Bernoulli probability of belonging to the positive class.

2.1.3 The fatal barrier of single-layer perceptrons: XOR linear inseparability

Despite its mathematical elegance, the single-layer perceptron suffers from a fatal representational barrier that brought artificial intelligence research to a virtual standstill for over a decade. In their seminal 1969 monograph *"Perceptrons"*, Marvin Minsky and Seymour Papert demonstrated that a single-layer perceptron cannot compute the basic *exclusive-or* (XOR) logical function.

The XOR truth table and linear system. Consider the XOR logic function over binary inputs $x_1, x_2 \in \{0, 1\}$:

x_1	x_2	Target $y = x_1 \oplus x_2$
0	0	0
0	1	1
1	0	1
1	1	0

For a single-layer perceptron with weights w_1, w_2 and bias w_0 to compute this function, there must exist parameters such that the linear boundary $w_1 x_1 + w_2 x_2 + w_0$ outputs positive values ($z \geq 0$) for target 1 and negative values ($z < 0$) for target 0. Substituting the four input pairs yields the system of linear inequalities:

$$(x_1 = 0, x_2 = 0) \implies w_0 < 0, \tag{2.6}$$
$$(x_1 = 1, x_2 = 0) \implies w_1 + w_0 \geq 0, \tag{2.7}$$
$$(x_1 = 0, x_2 = 1) \implies w_2 + w_0 \geq 0, \tag{2.8}$$
$$(x_1 = 1, x_2 = 1) \implies w_1 + w_2 + w_0 < 0. \tag{2.9}$$

Proof by mathematical contradiction. Summing inequalities (2.7) and (2.8):

$$(w_1 + w_0) + (w_2 + w_0) \geq 0 \implies w_1 + w_2 + 2w_0 \geq 0 \implies w_1 + w_2 \geq -2w_0. \tag{2.10}$$

Now examine inequality (2.9):

$$w_1 + w_2 + w_0 < 0 \implies w_1 + w_2 < -w_0. \tag{2.11}$$

Combining inequalities (2.10) and (2.11) yields:

$$-2w_0 \leq w_1 + w_2 < -w_0 \implies -2w_0 < -w_0.$$

Adding $2w_0$ to both sides yields:

$$0 < w_0 \implies w_0 > 0.$$

However, inequality (2.6) strictly demands that $w_0 < 0$. The assumption that a single linear hyperplane can compute XOR produces a direct mathematical contradiction ($w_0 > 0$ and $w_0 < 0$). ■

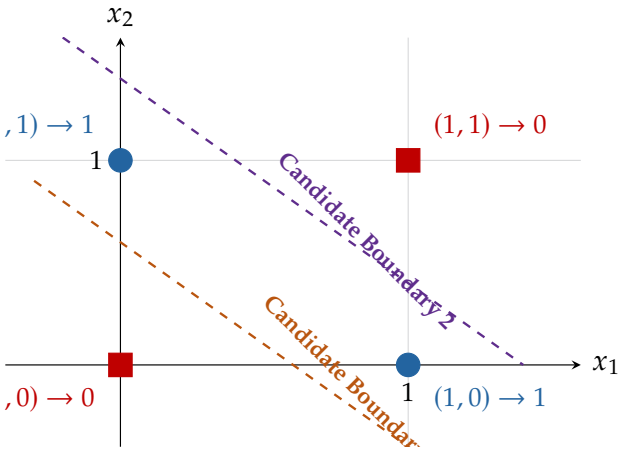


Figure 2.3: The XOR linear inseparability barrier (Minsky & Papert 1969). Class 1 points $(0, 1)$ and $(1, 0)$ (blue circles) cannot be separated from Class 0 points $(0, 0)$ and $(1, 1)$ (red squares) by any single linear hyperplane $w_1x_1 + w_2x_2 + w_0 = 0$. Any single line either isolates $(0, 0)$ or $(1, 1)$, but cannot simultaneously group both off-diagonal points.

As visualized in Figure 2.3, no straight line can separate the diagonal pairs. Because natural language and software engineering phenomena are riddled with non-linear relationships (e.g., mutually dependent programming idioms, nested conditional scopes), single-layer linear perceptrons are fundamentally inadequate.

The causal question. How can we overcome this linear inseparability barrier? An immediate intuitive engineering hypothesis presents itself: *If one linear perceptron can draw one line, can we solve non-linear problems simply by cascading multiple linear perceptrons into a multi-layer network?* We test this hypothesis next.

2.2 Stacking linear layers and the proof of linear stacking collapse

2.2.1 Fully connected linear layers (dense layers)

In multi-layer architectures, individual perceptrons do not operate in isolation; instead, multiple neurons run concurrently across a shared input vector $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$, forming a *dense layer* (or fully connected layer).

Let d_{in} denote the input feature dimensionality and d_{out} denote the number of hidden or output units. Stacking the individual weight vectors $\mathbf{w}_j \in \mathbb{R}^{d_{\text{in}}}$ as columns of a weight matrix $\mathbf{W} \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$ and collecting the scalar biases b_j into a bias vector $\mathbf{b} \in \mathbb{R}^{d_{\text{out}}}$, the layer computation expresses compactly in matrix notation:

$$\mathbf{y} = f\left(\mathbf{W}^\top \mathbf{x} + \mathbf{b}\right), \tag{2.12}$$

where $f(\cdot)$ is applied elementwise. Figure 2.4 diagrams this fully connected bipartite structure with explicit tensor dimensions and connection weights.

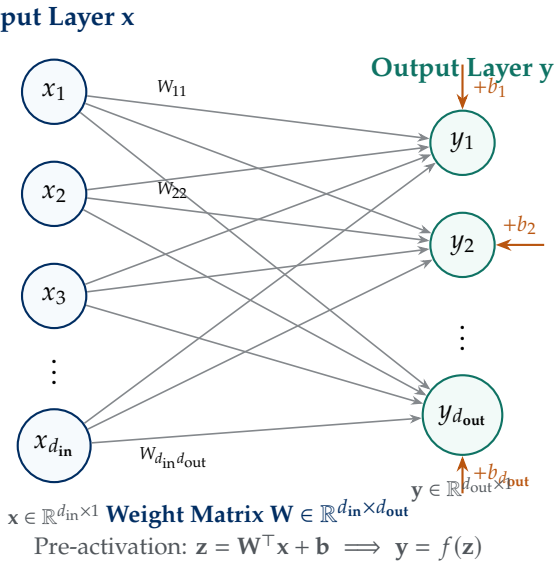


Figure 2.4: Architecture of a fully connected dense layer mapping an input vector $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$ to output $\mathbf{y} \in \mathbb{R}^{d_{\text{out}}}$. Each directed edge carries an individual scalar weight W_{ij}, and each output neuron incorporates an additive bias b_j.

Exam insights: Parameter counting across dense layers

A standard oral and written examination question tests exact parameter counting across stacked dense layers. For an affine transformation mapping $\mathbb{R}^{d_{\text{in}}} \to \mathbb{R}^{d_{\text{out}}}$:

$$N_{\text{params}} = \underbrace{d_{\text{in}} \times d_{\text{out}}}_{\text{weight parameters}} + \underbrace{d_{\text{out}}}_{\text{bias parameters}} = d_{\text{out}}(d_{\text{in}} + 1). \tag{2.13}$$

Example Calculation: Consider a two-layer Multi-Layer Perceptron (MLP) mapping an embedding of dimension $d_{\text{in}} = 768$ through a hidden layer $d_{\text{hidden}} = 3072$ to an output projection $d_{\text{out}} = 768$:

1. Layer 1: $3072 \times (768 + 1) = 2{,}359{,}296 + 3072 = 2{,}362{,}368$ parameters.
2. Layer 2: $768 \times (3072 + 1) = 2{,}359{,}296 + 768 = 2{,}360{,}064$ parameters.
3. Total Parameters: $2{,}362{,}368 + 2{,}360{,}064 = 4{,}722{,}432$ parameters.

Neglecting bias terms during parameter inventory counts is a frequent source of point deductions.

2.2.2 The mathematical proof of linear stacking collapse

Now we return to our hypothesis: *Can we solve XOR and non-linear problems simply by cascading multiple linear layers without activation functions?*

The following theorem demonstrates that without non-linear activations, stacking arbitrary numbers of linear layers provides zero additional representational power over a single-layer linear model.

Theorem 2.1: *Linear stacking collapse*

Let $f_1 : \mathbb{R}^n \to \mathbb{R}^m$ and $f_2 : \mathbb{R}^m \to \mathbb{R}^k$ denote two consecutive fully connected layers operating without non-linear activations ($f(x) = x$). Let $\mathbf{W}_1 \in \mathbb{R}^{n\times m}$ and $\mathbf{b}_1 \in \mathbb{R}^m$ be the parameters of the first layer, and let $\mathbf{W}_2 \in \mathbb{R}^{m\times k}$ and $\mathbf{b}_2 \in \mathbb{R}^k$ be the parameters of the second layer:

$$\mathbf{h} = f_1(\mathbf{x}) = \mathbf{W}_1^\top \mathbf{x} + \mathbf{b}_1, \tag{2.14}$$

$$\mathbf{z} = f_2(\mathbf{h}) = \mathbf{W}_2^\top \mathbf{h} + \mathbf{b}_2. \tag{2.15}$$

The composite function $\mathbf{z} = f_2(f_1(\mathbf{x}))$ collapses identically to a single affine transformation:

$$\begin{aligned} \mathbf{z} &= \mathbf{W}_2^\top \left(\mathbf{W}_1^\top \mathbf{x} + \mathbf{b}_1\right) + \mathbf{b}_2 \\ &= \mathbf{W}_2^\top \mathbf{W}_1^\top \mathbf{x} + \mathbf{W}_2^\top \mathbf{b}_1 + \mathbf{b}_2 \\ &= (\mathbf{W}_1 \mathbf{W}_2)^\top \mathbf{x} + \left(\mathbf{W}_2^\top \mathbf{b}_1 + \mathbf{b}_2\right) \\ &= \mathbf{W}_{\text{net}}^\top \mathbf{x} + \mathbf{b}_{\text{net}}, \end{aligned} \tag{2.16}$$

where $\mathbf{W}_{\text{net}} = \mathbf{W}_1 \mathbf{W}_2 \in \mathbb{R}^{n\times k}$ and $\mathbf{b}_{\text{net}} = \mathbf{W}_2^\top \mathbf{b}_1 + \mathbf{b}_2 \in \mathbb{R}^k$.

Proof by mathematical induction. Matrix multiplication is associative and closed under composition. The product of two matrices $\mathbf{W}_1 \in \mathbb{R}^{n\times m}$ and $\mathbf{W}_2 \in \mathbb{R}^{m\times k}$ is simply a single matrix $\mathbf{W}_{\text{net}} \in \mathbb{R}^{n\times k}$. Likewise, the affine transformation of bias vector $\mathbf{b}_1$ by $\mathbf{W}_2^\top$ added to $\mathbf{b}_2$ is simply a single constant vector $\mathbf{b}_{\text{net}} \in \mathbb{R}^k$.

By mathematical induction, for any cascade of L linear layers:

$$\mathbf{z} = \mathbf{W}_L^\top \left(\dots \left(\mathbf{W}_2^\top \left(\mathbf{W}_1^\top \mathbf{x} + \mathbf{b}_1\right) + \mathbf{b}_2\right) \dots\right) + \mathbf{b}_L = \mathbf{W}_{\text{net}}^\top \mathbf{x} + \mathbf{b}_{\text{net}}, \tag{2.17}$$

where:

$$\mathbf{W}_{\text{net}} = \prod_{l=1}^{L} \mathbf{W}_l \in \mathbb{R}^{d_{\text{in}}\times d_{\text{out}}}, \quad \mathbf{b}_{\text{net}} = \mathbf{b}_L + \sum_{l=1}^{L-1} \left(\prod_{j=l+1}^{L} \mathbf{W}_j\right)^\top \mathbf{b}_l. \tag{2.18}$$

The composite mapping is strictly linear. Stacking 100 purely linear layers introduces millions of redundant matrix multiplications while learning nothing beyond standard linear regression. It strictly cannot compute XOR or any non-linear function. ■

The fundamental takeaway. Non-linear activation functions are strictly and mathematically mandatory. Without non-linearity between layers, deep networks collapse into trivial shallow affine mappings. Non-linear activations warp and fold coordinate space, transforming linearly inseparable data manifolds into representations where classes can be cleanly separated.

2.2.3 Universal approximation theorem and bump-function synthesis

Given that non-linear activation functions prevent linear collapse, what family of functions can a non-linear neural network represent? The theoretical foundation validating neural networks as general-purpose function approximators is the *Universal Approximation Theorem*.

Theorem 2.2: *Universal approximation theorem*

Let $\sigma : \mathbb{R} \to \mathbb{R}$ be any continuous, non-constant, bounded, and monotonically increasing activation function (e.g., sigmoid) [17]. Let $I_n \subset \mathbb{R}^n$ denote a compact (closed and bounded) subset of n-dimensional Euclidean space, and let $C(I_n)$ denote the space of continuous real-valued functions on I_n. Then, for any target continuous function $g \in C(I_n)$ and any arbitrary error tolerance $\epsilon > 0$, there exist a finite integer $M \in \mathbb{N}$, parameter weights $\mathbf{w}_j \in \mathbb{R}^n$, outer weights $\alpha_j \in \mathbb{R}$, and biases $b_j \in \mathbb{R}$ such that the single-hidden-layer feedforward network:

$$F(\mathbf{x}) = \sum_{j=1}^{M} \alpha_j \sigma \left(\mathbf{w}_j^\top \mathbf{x} + b_j \right) \tag{2.19}$$

satisfies uniform approximation error:

$$\sup_{\mathbf{x} \in I_n} \left| F(\mathbf{x}) - g(\mathbf{x}) \right| < \epsilon. \tag{2.20}$$

Originally proved in [17] for sigmoidal activations and generalized in [18] and [19] to arbitrary continuous non-polynomial activations, this theorem guarantees that a feedforward network with a single hidden layer has the mathematical capacity to approximate any continuous function (as discussed in the course theoretical foundations).

Intuitive geometric derivation: bump-function synthesis. To understand intuitively why a single hidden layer can approximate any continuous curve, consider a 1D function $g(x) : [a, b] \to \mathbb{R}$.

Step 1: Constructing a localized rectangular bump. Consider two sigmoid neurons with very high slope $w \gg 0$ located at spatial offsets x_1 and x_2 (with $x_1 < x_2$). As $w \to \infty$, the sigmoid $\sigma(w(x - x_1))$ converges to a Heaviside step function $H(x - x_1)$.

Subtracting the second sigmoid from the first produces a localized *bump function* (or rectangular wavelet):

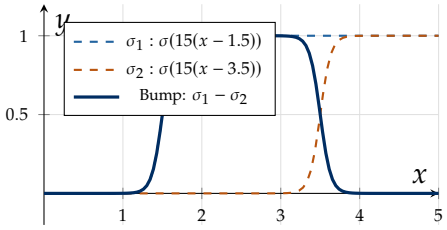
$$B(x) = \sigma(w(x - x_1)) - \sigma(w(x - x_2)) \approx \begin{cases} 1 & \text{if } x_1 \le x \le x_2 \\ 0 & \text{otherwise} \end{cases}. \tag{2.21}$$

Figure 2.5(a) illustrates how two opposing sigmoids subtract to create a localized rectangular pulse.

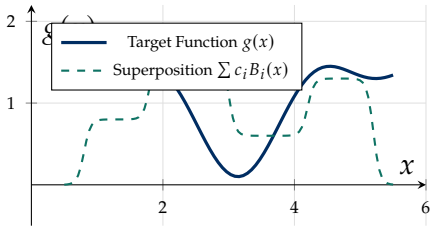
Step 2: Riemann sum function reconstruction. By the definition of continuous functions on compact intervals, $g(x)$ can be approximated to arbitrary accuracy ϵ by a piecewise-constant step function consisting of M uniform sub-intervals $[x_k, x_{k+1}]$ with heights $c_k = g(x_k)$, exactly as in standard Riemann integration:

$$g(x) \approx \sum_{k=1}^{M} c_k B_k(x) = \sum_{k=1}^{M} c_k \left[\sigma(w(x - x_k)) - \sigma(w(x - x_{k+1})) \right]. \tag{2.22}$$

Each bump $B_k(x)$ requires exactly two hidden sigmoid neurons. Consequently, a single hidden layer containing $2M$ neurons can synthesize any continuous function to arbitrary precision, as visualized in Figure 2.5(b).



(a) Bump Synthesis via Opposing Sigmoids



(b) Riemann-style Function Reconstruction

Figure 2.5: Visual derivation of the Universal Approximation Theorem. (a) Subtracting two shifted steep sigmoids produces a localized rectangular bump function $B(x)$; (b) Superimposing multiple scaled bump functions reconstructs any continuous target function to arbitrary accuracy ϵ.

2.2.4 Why deep networks if shallow networks are universal approximators?

If a single hidden layer can approximate any function, why does modern deep learning exclusively use deep architectures with dozens or hundreds of stacked layers (e.g., 96 layers in GPT-3)?

1. **Non-Constructive Existence**: The theorem guarantees that parameters exist, but provides no gradient-based algorithm to discover them and establishes no finite bound on the width M.

2. **Exponential Parameter Inefficiency**: Approximating highly compositional functions in n dimensions with a single hidden layer requires a width M that scales exponentially with input dimensionality ($M \sim O(2^n)$).

3. **Hierarchical Compositionality**: Deep networks factorize functions into hierarchical abstractions: early layers learn low-level primitives (subwords, syntax), intermediate layers learn grammatical structure, and deep layers capture long-range semantic discourse. Deep networks achieve equal approximation power with exponentially fewer parameters than shallow networks.

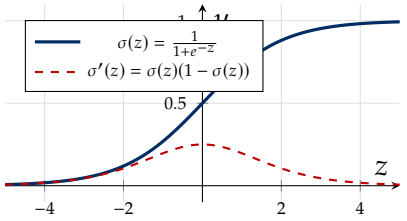
2.3 Taxonomy of neural activation functions

Modern neural NLP architectures deploy diverse activation functions tailored to hidden representations and output heads. Figure 2.6 presents exact mathematical plots and derivatives for the primary activation functions.

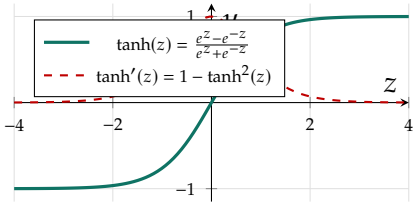
1. The logistic sigmoid (σ).

$$\sigma(z) = \frac{1}{1+e^{-z}}, \quad \text{Range: } (0,1), \quad \sigma'(z) = \sigma(z)(1-\sigma(z)). \tag{2.23}$$

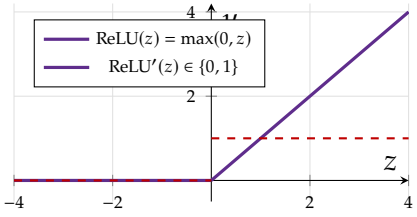
The derivative attains its global maximum of $\sigma'(0) = 0.25$ at the origin. For large positive or negative inputs ($|z| \geq 4$), $\sigma'(z) \to 0$, causing severe *vanishing gradients* in deep networks.



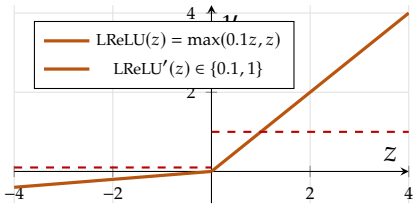
(a) Sigmoid (Logistic)



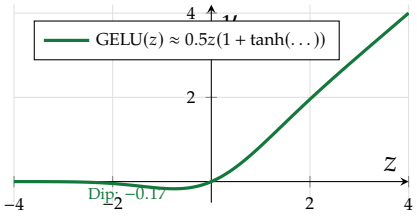
(b) Hyperbolic Tangent (tanh)



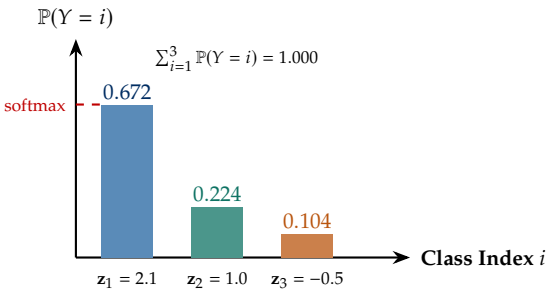
(c) Rectified Linear Unit (ReLU)



(d) Leaky ReLU ($\alpha = 0.1$)



(e) Gaussian Error Linear Unit (GELU)



(f) Softmax Simplex Mapping ($\mathbb{R}^3 \to \Delta^2$)

Figure 2.6: Taxonomy of primary neural activation functions plotted across their operating ranges: (a) Sigmoid $\sigma(z)$ with peak derivative 0.25; (b) Zero-centered $\tanh(z)$ with peak derivative 1.0; (c) Piecewise linear $\mathrm{ReLU}(z)$; (d) Leaky ReLU preventing dead units; (e) Non-monotonic $\mathrm{GELU}(z)$ used in modern Transformers; (f) Multi-class Softmax mapping logits to the probability simplex.

2. Hyperbolic tangent (tanh**).**

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} = 2\,\sigma(2z) - 1, \quad \text{Range: } (-1, 1), \quad \tanh'(z) = 1 - \tanh^2(z). \tag{2.24}$$

The hyperbolic tangent is strictly zero-centered, possessing average activation values near zero, which accelerates gradient descent compared to non-negative sigmoids. However, it still saturates for $|z| \geq 3$, yielding vanishing gradients.

3. Rectified linear unit (ReLU**).**

$$\text{ReLU}(z) = \max(0, z), \quad \text{Range: } [0, \infty), \quad \text{ReLU}'(z) = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{if } z < 0 \end{cases}. \tag{2.25}$$

ReLU completely eliminates gradient vanishing for positive pre-activations ($z > 0$). However, if a large gradient update shifts a neuron's weights such that $z \leq 0$ across all training samples, its activation and derivative permanently collapse to zero ($\text{ReLU}'(z) = 0$), a pathology known as the *dying ReLU problem*.

4. Leaky ReLU (LReLU).

$$\text{LReLU}(z) = \max(\alpha z, z) \quad \text{with } 0 < \alpha \ll 1, \quad \text{Range: } (-\infty, \infty), \quad \text{LReLU}'(z) = \begin{cases} 1 & \text{if } z > 0 \\ \alpha & \text{if } z \leq 0 \end{cases}. \tag{2.26}$$

By maintaining a small constant slope α (typically $\alpha = 0.01$) in the negative half-plane, Leaky ReLU guarantees that a non-zero gradient always backpropagates, preventing dead neurons.

5. Gaussian error linear unit (GELU**).** Introduced by Dan Hendrycks and Kevin Gimpel, GELU scales the input by its cumulative standard normal probability $\Phi(z)$:

$$\text{GELU}(z) = z \cdot \Phi(z) = z \cdot \frac{1}{2}\left[1 + \text{erf}\left(\frac{z}{\sqrt{2}}\right)\right] \approx 0.5z\left(1 + \tanh\left(\sqrt{\frac{2}{\pi}}\left(z + 0.044715z^3\right)\right)\right). \tag{2.27}$$

GELU is smooth, differentiable everywhere, and non-monotonic, dipping to a global minimum of ≈ -0.17 at $z \approx -0.75$. This smoothness provides curvature information during optimization, making GELU the default activation in Transformer language models (GPT-2, GPT-3, BERT, RoBERTa).

6. Softmax. For categorical multi-class predictions over C mutually exclusive classes, Softmax normalizes a real-valued logit vector $\mathbf{z} = [z_1, \ldots, z_C]^\top \in \mathbb{R}^C$ onto the unit probability simplex Δ^{C-1}:

$$\hat{y}_i = \text{softmax}(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{C} e^{z_j}} \quad \text{such that } \hat{y}_i \in (0, 1) \quad \text{and} \quad \sum_{i=1}^{C} \hat{y}_i = 1. \tag{2.28}$$

Softmax forms the standard vocabulary output head of modern autoregressive language models, computing next-token probability distributions across vocabulary $\mathcal{V}$.

Table 2.1 provides a comparative reference summarizing the mathematical properties of each activation function.

Table 2.1: Comprehensive taxonomy of neural activation functions, mathematical formulations, operating bounds, and architectural domains.

Activation	Domain	Range	Peak Gradient	Primary Use Case in Language Modeling
Sigmoid (σ)	$\mathbb{R}$	$(0, 1)$	0.25 ($z = 0$)	Binary classification output heads; gating mechanisms in LSTM/GRU highways.
Tanh	$\mathbb{R}$	$(-1, 1)$	1.00 ($z = 0$)	Recurrent hidden state candidate generation; zero-centered coordinate representations.
ReLU	$\mathbb{R}$	$[0, \infty)$	1.00 ($z > 0$)	Classical deep feedforward networks; compute-efficient linear regions.
Leaky ReLU	$\mathbb{R}$	$\mathbb{R}$	1.00 ($z > 0$), α ($z \leq 0$)	Prevents dying neuron syndrome; robust discriminator networks.
GELU	$\mathbb{R}$	$[-0.17, \infty)$	≈ 1.08	Canonical Transformer activation (GPT, BERT); smooth, probabilistic gating.
Softmax	$\mathbb{R}^C$	Δ^{C-1}	Matrix Jacobian	Next-token prediction vocabulary head; maps logits to discrete probability simplex.
Identity	$\mathbb{R}$	$\mathbb{R}$	1.00 ($\forall z$)	Regression output head; unbounded linear prediction.

2.4 Architectural task formulations and software engineering case studies

In machine learning applications for software engineering, task formulations map specific software artifacts (commits, ASTs, bug reports, source code tokens) into continuous predictions. Three primary architectural heads dominate:

2.4.1 1. Binary classification: automated vulnerability detection

Consider predicting whether a proposed code commit contains a security vulnerability (e.g., buffer overflow, SQL injection).

- **Input Vector**: $\mathbf{x} \in \mathbb{R}^d$ (extracted code embeddings or static analysis metrics).
- **Output Layer**: A single linear neuron producing logit $z = \mathbf{w}^\top \mathbf{h} + b \in \mathbb{R}$.
- **Activation**: Logistic Sigmoid $\hat{y} = \sigma(z) = \mathbb{P}(\text{Vulnerable} = 1 \mid \mathbf{x}) \in [0, 1]$.
- **Decision Rule**: Flag for manual audit if $\hat{y} \geq \tau$ (where $\tau = 0.5$ by default, tuned to optimize recall).

Figure 2.7(a) illustrates the resulting separating hyperplane in feature space.

2.4.2 2. Multi-class classification: automated AST node type prediction

In program repair and neural compiler optimization, models classify Abstract Syntax Tree (AST) nodes into C discrete syntactic types (e.g., `MethodDeclaration`, `VariableDeclaration`, `IfStatement`, `WhileStatement`).

- **Output Layer**: C parallel linear neurons producing logit vector $\mathbf{z} = \mathbf{W}^\top \mathbf{h} + \mathbf{b} \in \mathbb{R}^C$.
- **Activation**: Softmax operator $\hat{\mathbf{y}} = \text{softmax}(\mathbf{z}) \in \Delta^{C-1}$.

- **Decision Rule**: Predicted AST type $\hat{c} = \arg\max_i \hat{y}_i$.

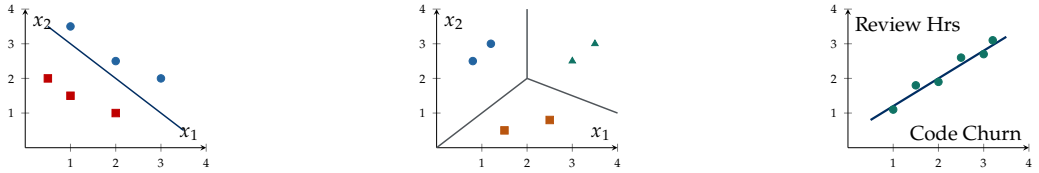
Figure 2.7(b) diagrams the resulting Voronoi-like partitioned multi-class decision space.

2.4.3 3. Regression: pull request review effort estimation

Engineering managers must predict continuous quantities, such as the estimated review time (in hours) required for an incoming pull request based on code churn, changed files, and author experience.

- **Output Layer**: A single linear neuron producing scalar $z = \mathbf{w}^\top \mathbf{h} + b$.
- **Activation**: Linear Identity function $f(z) = z \in \mathbb{R}$.
- **Prediction**: $\hat{y} = z \in \mathbb{R}$ (continuous unbounded prediction).

Figure 2.7(c) illustrates continuous line fitting through software engineering observation points.



(a) Binary: Defect Detection (b) Multi-class: AST Type (c) Regression: PR Review Effort

Figure 2.7: Task formulations across Software Engineering workflows: (a) Binary vulnerability detection with separating hyperplane; (b) Multi-class defect categorization with partitioned decision regions; (c) Continuous PR review effort regression with residual errors.

2.5 Loss function formulations and optimization landscapes

2.5.1 Loss function formulations

Training a neural network requires an objective function $\mathcal{L}(\theta)$ that measures the discrepancy between model predictions $\hat{y}$ and ground-truth targets y.

1. Mean squared error (MSE). For continuous regression targets:

$$\mathcal{L}_{\text{MSE}}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \left(y_i - \hat{y}_i\right)^2. \tag{2.29}$$

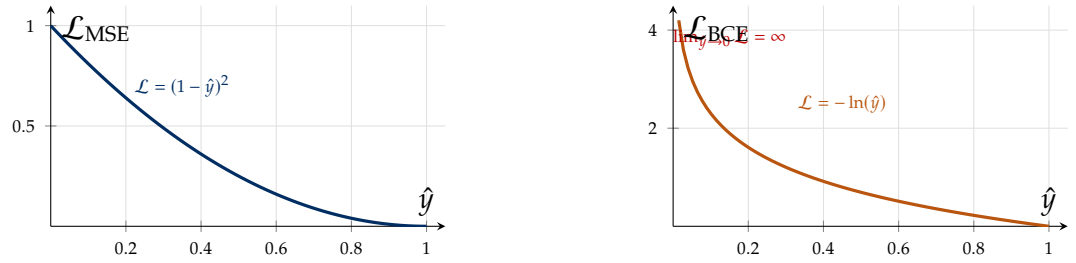
2. Binary cross-entropy (BCE). For binary classification with target $y \in \{0, 1\}$ and predicted probability $\hat{y} = \sigma(z) \in [0, 1]$:

$$\mathcal{L}_{\text{BCE}}(\theta) = -y \ln(\hat{y}) - (1 - y) \ln(1 - \hat{y}). \tag{2.30}$$

3. Categorical cross-entropy (CCE). For multi-class classification over C mutually exclusive classes with one-hot target vector $\mathbf{y} \in \{0, 1\}^C$:

$$\mathcal{L}_{\text{CE}}(\theta) = -\sum_{i=1}^{C} y_i \ln(\hat{y}_i) = -\ln\left(\hat{y}_{\text{target}}\right) = -\ln\left(\frac{e^{z_{\text{target}}}}{\sum_{j=1}^{C} e^{z_j}}\right). \tag{2.31}$$

Figure 2.8 compares the mathematical profiles of MSE and BCE for a positive target $y = 1$.



(a) MSE Loss for $y = 1$ (Bounded: $\mathcal{L} \in [0, 1]$)

(b) BCE Loss for $y = 1$ (Asymptotic Penalty)

Figure 2.8: Comparison of loss function behaviors for target $y = 1$. While MSE is bounded quadratically at 1.0, BCE imposes an asymptotic infinite penalty ($\mathcal{L} \to \infty$) as predicted probability $\hat{y} \to 0$, providing steep, non-vanishing corrective gradients.

2.5.2 Rigorous proof: why MSE fails for classification

A fundamental theoretical question is: *Why do we never use Mean Squared Error with sigmoid activation for classification?* The answer lies in gradient saturation and non-convexity.

Theorem 2.3: *Gradient saturation of MSE under sigmoid activation*

Let a binary classification model predict $\hat{y} = \sigma(z) = \frac{1}{1+e^{-z}}$ with ground-truth target $y \in \{0, 1\}$. Under the quadratic loss $\mathcal{L}_{\text{MSE}} = \frac{1}{2}(y - \hat{y})^2$, the gradient with respect to the pre-activation logit z is:

$$\frac{\partial \mathcal{L}_{\text{MSE}}}{\partial z} = -(y - \hat{y}) \cdot \hat{y}(1 - \hat{y}). \tag{2.32}$$

Conversely, under Binary Cross-Entropy $\mathcal{L}_{\text{BCE}} = -y \ln(\hat{y}) - (1 - y) \ln(1 - \hat{y})$, the gradient is:

$$\frac{\partial \mathcal{L}_{\text{BCE}}}{\partial z} = \hat{y} - y. \tag{2.33}$$

Proof. 1. For MSE, applying the calculus chain rule:

$$\frac{\partial \mathcal{L}_{\text{MSE}}}{\partial z} = \frac{\partial \mathcal{L}_{\text{MSE}}}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z}. \tag{2.34}$$

The derivative of the loss with respect to $\hat{y}$ is:

$$\frac{\partial \mathcal{L}_{\text{MSE}}}{\partial \hat{y}} = \frac{\partial}{\partial \hat{y}} \left[\frac{1}{2}(y - \hat{y})^2 \right] = -(y - \hat{y}).$$

The derivative of the sigmoid is $\sigma'(z) = \sigma(z)(1 - \sigma(z)) = \hat{y}(1 - \hat{y})$. Substituting:

$$\frac{\partial \mathcal{L}_{\text{MSE}}}{\partial z} = -(y - \hat{y})\hat{y}(1 - \hat{y}). \tag{2.35}$$

2. For BCE, expanding the loss:

$$\mathcal{L}_{\text{BCE}} = -y \ln(\sigma(z)) - (1 - y) \ln(1 - \sigma(z)). \tag{2.36}$$

Differentiating directly with respect to logit z:

$$\begin{aligned}\frac{\partial \mathcal{L}_{\text{BCE}}}{\partial z} &= -y\frac{\sigma'(z)}{\sigma(z)} - (1-y)\frac{-\sigma'(z)}{1-\sigma(z)} \\ &= -y\frac{\sigma(z)(1-\sigma(z))}{\sigma(z)} + (1-y)\frac{\sigma(z)(1-\sigma(z))}{1-\sigma(z)} \\ &= -y(1-\sigma(z)) + (1-y)\sigma(z) \\ &= -y + y\sigma(z) + \sigma(z) - y\sigma(z) \\ &= \sigma(z) - y = \hat{y} - y. \quad \blacksquare\end{aligned}$$

Analytical comparison of optimization behavior. The fundamental architectural flaws of MSE for classification are immediately apparent from Equations (2.35) and (2.33):

- **The Worst-Case MSE Pathology (Confident and Wrong)**: Suppose the true label is $y = 1$, but the network is completely mistaken, predicting $z = -10 \implies \hat{y} \approx 0.000045$. The prediction error is maximal: $(y - \hat{y}) \approx 1.0$. However, evaluating the MSE gradient:

  $$\frac{\partial \mathcal{L}_{\text{MSE}}}{\partial z} = -(1.0) \times (0.000045) \times (0.999955) \approx -0.000045 \approx 0!$$

  The gradient completely vanishes! Even though the model is entirely wrong, parameter updates halt because the sigmoid derivative $\hat{y}(1-\hat{y})$ saturates to zero. Optimization gets trapped on a flat plateau.

- **The BCE Remedy**: Under BCE, for the identical scenario ($y = 1, \hat{y} \approx 0$):

  $$\frac{\partial \mathcal{L}_{\text{BCE}}}{\partial z} = \hat{y} - y = 0 - 1 = -1.0.$$

  The sigmoid derivative in the numerator cancels perfectly with the denominator in the derivative of the logarithm! The gradient is strictly linear with respect to the error $(y - \hat{y})$ and provides maximum learning signal precisely when the model is most wrong.

- **Non-Convexity**: While MSE with linear regression is strictly convex, combining MSE with non-linear sigmoid activations creates a non-convex loss surface with numerous local plateaus. In contrast, BCE is strictly convex with respect to logit z.

2.5.3 Maximum likelihood estimation derivation of cross-entropy

Cross-entropy is not an empirical heuristic; it is the mathematically canonical loss derived from Maximum Likelihood Estimation (MLE).

Bernoulli likelihood derivation (BCE). For binary classification, target $y_i \in \{0, 1\}$ follows a Bernoulli distribution parameterized by probability $p_i = \hat{y}_i = \sigma(z_i)$:

$$\mathbb{P}(Y = y_i \mid \mathbf{x}_i) = \hat{y}_i^{y_i}(1-\hat{y}_i)^{1-y_i}. \tag{2.37}$$

For an independent dataset of N samples, the total likelihood is $\mathcal{L}_{\text{data}}(\theta) = \prod_{i=1}^{N} \hat{y}_i^{y_i}(1-\hat{y}_i)^{1-y_i}$. Taking the negative log-likelihood (NLL):

$$\mathcal{L}(\theta) = -\ln \mathcal{L}_{\text{data}}(\theta) = -\sum_{i=1}^{N} \left[y_i \ln \hat{y}_i + (1-y_i)\ln(1-\hat{y}_i) \right] = \sum_{i=1}^{N} \mathcal{L}_{\text{BCE},i}. \tag{2.38}$$

Minimizing BCE is mathematically identical to maximizing the data likelihood under a Bernoulli distribution.

Categorical likelihood derivation (CCE). For multi-class classification over C classes with one-hot targets $\mathbf{y}_i$, the likelihood follows a Categorical distribution:

$$\mathbb{P}(\mathbf{y}_i \mid \mathbf{x}_i) = \prod_{c=1}^{C} \hat{y}_{ic}^{\,y_{ic}} \implies -\ln \mathbb{P}(\mathbf{y}_i \mid \mathbf{x}_i) = -\sum_{c=1}^{C} y_{ic} \ln \hat{y}_{ic} = \mathcal{L}_{\text{CE}}. \tag{2.39}$$

2.5.4 Gradient descent and adaptive optimizers

Parameters θ are updated iteratively along the negative gradient:

$$\theta^{(t+1)} := \theta^{(t)} - \alpha \nabla_\theta \mathcal{L}(\theta^{(t)}), \tag{2.40}$$

where $\alpha > 0$ is the learning rate.

Limitations of vanilla gradient descent. In deep neural architectures, vanilla Gradient Descent struggles with:

1. **Ravines and Ill-Conditioned Curvature**: Valleys where gradient magnitude is large across the ravine walls but tiny along the base, causing erratic cross-valley oscillation.
2. **Saddle Points**: Points where $\nabla\mathcal{L} = 0$ while the Hessian possesses both positive and negative eigenvalues.

Adaptive remedies: Momentum and Adam.

- **Momentum**: Incorporates an exponentially decaying velocity vector $\mathbf{v}_t = \beta \mathbf{v}_{t-1} + (1 - \beta)\nabla\mathcal{L}(\theta_t)$, accelerating descent through saddle regions.
- **Adam (Adaptive Moment Estimation)** [20]: Tracks running averages of both gradients (first moment $\mathbf{m}_t$) and squared gradients (second uncentered moment $\mathbf{v}_t$), with bias corrections:

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1)\mathbf{g}_t, \quad \hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t}, \tag{2.41}$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2)\mathbf{g}_t^2, \quad \hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_2^t}, \tag{2.42}$$

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon} \hat{\mathbf{m}}_t. \tag{2.43}$$

  Adam scales learning rates adaptively per parameter, making it the standard optimizer across LLM pre-training and fine-tuning pipelines.

2.6 Computational graphs and exact adjoint backpropagation

To compute gradients efficiently across billions of parameters, deep learning frameworks (PyTorch, JAX) represent mathematical workflows as a *Directed Acyclic Graph (DAG)*, known as a *computational graph*.

Definition 2.2: *Computational graph*

A computational graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ is a directed acyclic graph where:

- Each node $v \in \mathcal{V}$ represents an input variable, parameter, or primitive mathematical

operation $(+, \times, -, \exp)$.

- Each directed edge $e = (u, v) \in \mathcal{E}$ represents intermediate data flow from variable u into operation v.

2.6.1 Exact backpropagation walkthrough from slide 24

We reproduce and derive the exact pedagogical computational graph and adjoint trace from Slide 24 of the course materials.

Problem setup. Consider a model with two scalar parameters $\theta_1, \theta_2 \in \mathbb{R}$ evaluated on training pair $(x, y) \in \mathbb{R}^2$:

$$\text{Model Prediction:} \quad \hat{y} = \theta_1 \theta_2 x, \tag{2.44}$$

$$\text{Objective Loss:} \quad \mathcal{L} = (\theta_1 \theta_2 x - y)^2. \tag{2.45}$$

Forward decomposition. The graph decomposes Equation (2.45) into four primitive operations:

$$a = \theta_1 \theta_2 \quad \text{(parameter interaction product)}, \tag{2.46}$$

$$b = ax = \theta_1 \theta_2 x \quad \text{(prediction } \hat{y}\text{)}, \tag{2.47}$$

$$c = b - y = \theta_1 \theta_2 x - y \quad \text{(residual error)}, \tag{2.48}$$

$$\mathcal{L} = c^2 = (\theta_1 \theta_2 x - y)^2 \quad \text{(scalar squared loss)}. \tag{2.49}$$

Backward adjoint chain rule derivation. Backpropagation computes the derivative of scalar loss $\mathcal{L}$ with respect to every intermediate variable and parameter by traversing the graph in reverse topological order:

$$\text{Step 1:} \quad \frac{\partial \mathcal{L}}{\partial c} = \frac{\partial (c^2)}{\partial c} = 2c. \tag{2.50}$$

$$\text{Step 2:} \quad \frac{\partial \mathcal{L}}{\partial b} = \frac{\partial \mathcal{L}}{\partial c} \frac{\partial c}{\partial b} = 2c \cdot \frac{\partial (b - y)}{\partial b} = 2c \cdot 1 = 2c. \tag{2.51}$$

$$\text{Step 3:} \quad \frac{\partial \mathcal{L}}{\partial a} = \frac{\partial \mathcal{L}}{\partial b} \frac{\partial b}{\partial a} = 2c \cdot \frac{\partial (ax)}{\partial a} = 2c \cdot x = 2cx. \tag{2.52}$$

$$\text{Step 4:} \quad \frac{\partial \mathcal{L}}{\partial \theta_1} = \frac{\partial \mathcal{L}}{\partial a} \frac{\partial a}{\partial \theta_1} = (2cx) \cdot \frac{\partial (\theta_1 \theta_2)}{\partial \theta_1} = 2cx \cdot \theta_2 = 2(\theta_1 \theta_2 x - y) x \theta_2. \tag{2.53}$$

$$\text{Step 5:} \quad \frac{\partial \mathcal{L}}{\partial \theta_2} = \frac{\partial \mathcal{L}}{\partial a} \frac{\partial a}{\partial \theta_2} = (2cx) \cdot \frac{\partial (\theta_1 \theta_2)}{\partial \theta_2} = 2cx \cdot \theta_1 = 2(\theta_1 \theta_2 x - y) x \theta_1. \tag{2.54}$$

Figure 2.9 illustrates this complete forward and backward adjoint execution.

Intuition & analogy: 3Blue1Brown's geometric view of backpropagation

Grant Sanderson (3Blue1Brown) articulates backpropagation not as abstract calculus algebra, but as a systematic tracking of sensitivity *nudges*. Imagine wiggling parameter θ_1 by an infinitesimal perturbation ϵ. That nudge ripples through the product node to scale a by $\epsilon \theta_2$. The change in a wiggles b by $\epsilon \theta_2 x$, which directly shifts error c and finally scales loss $\mathcal{L}$ by $2c\epsilon\theta_2 x$. Backpropagation simply reads these connected multiplicative sensitivity tracks in reverse, gathering the composite derivative at each fork in the graph.

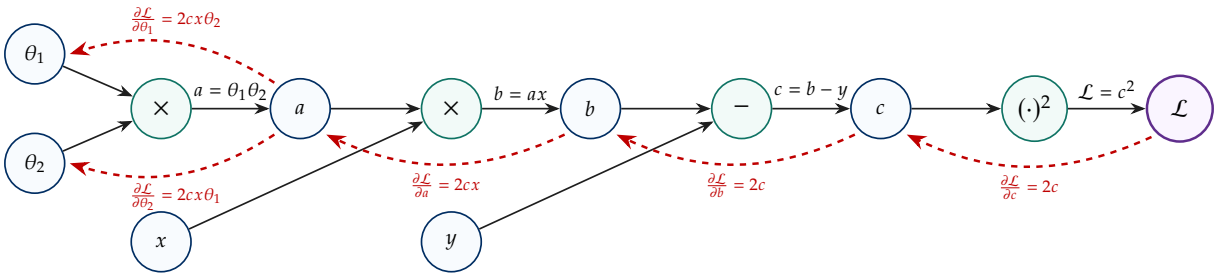


Figure 2.9: Directed Acyclic Computational Graph (DAG) for model $\hat{y} = \theta_1\theta_2 x$ with quadratic loss $\mathcal{L} = (\theta_1\theta_2 x - y)^2$ (reproduced from Slide 24). Solid black arrows trace forward execution; dashed red arrows trace reverse-mode adjoint backpropagation via the chain rule [21].

External deep dive: Building autograd from scratch: Andrej Karpathy's Micrograd

Andrej Karpathy's open-source library *Micrograd* implements reverse-mode automatic differentiation in fewer than 100 lines of pure Python, demonstrating the exact computational graph mechanics shown in Figure 2.9.
Each scalar variable is wrapped inside a `Value` class containing:

- `.data`: the floating-point scalar computed during the forward pass.
- `.grad`: the accumulated adjoint derivative $\partial\mathcal{L}/\partial\cdot$, initialized to 0.0.
- `._prev`: a set storing pointers to operand children nodes.
- `._backward`: a closure storing the local multivariate chain rule.

For example, multiplication is implemented as:

```
def __mul__(self, other):
    other = other if isinstance(other, Value) else Value(other)
    out = Value(self.data * other.data, (self, other), '*')
    def _backward():
        self.grad += other.data * out.grad
        other.grad += self.data * out.grad
    out._backward = _backward
    return out
```

When calling `loss.backward()`, the engine performs a **topological sort** on the DAG to establish reverse dependency ordering, sets `loss.grad = 1.0`, and executes `node._backward()` across all parent nodes in reverse topological order.

2.6.2 Vectorized matrix adjoint backpropagation for dense layers

While scalar computational graphs illustrate the chain rule clearly, production deep learning libraries execute backpropagation via *vectorized matrix calculus* across high-dimensional tensors. We now derive the exact vectorized matrix adjoint equations for fully connected layers with explicit tensor dimensions.

Layer formulation and forward variables. Consider a fully connected dense layer mapping an input vector $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$ through weight matrix $\mathbf{W} \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$ and bias $\mathbf{b} \in \mathbb{R}^{d_{\text{out}}}$ to pre-activation

$\mathbf{z} \in \mathbb{R}^{d_{\text{out}}}$ and activation $\mathbf{y} \in \mathbb{R}^{d_{\text{out}}}$:

$$\mathbf{z} = \mathbf{W}^\top \mathbf{x} + \mathbf{b} \in \mathbb{R}^{d_{\text{out}}}, \tag{2.55}$$

$$\mathbf{y} = f(\mathbf{z}) \in \mathbb{R}^{d_{\text{out}}}, \tag{2.56}$$

where scalar component z_j expands as:

$$z_j = \sum_{i=1}^{d_{\text{in}}} W_{ij} x_i + b_j. \tag{2.57}$$

The layer output $\mathbf{y}$ propagates downstream to contribute to a final scalar objective loss $\mathcal{L} \in \mathbb{R}$.

Adjoint definition. Assume downstream backpropagation has computed the loss gradient with respect to layer pre-activation $\mathbf{z}$. We define the adjoint error vector $\boldsymbol{\delta} \in \mathbb{R}^{d_{\text{out}}}$:

$$\boldsymbol{\delta} \triangleq \frac{\partial \mathcal{L}}{\partial \mathbf{z}} = \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \odot f'(\mathbf{z}) \in \mathbb{R}^{d_{\text{out}}}, \tag{2.58}$$

where $\odot$ denotes the elementwise Hadamard product.

1. Weight matrix adjoint derivation ($\frac{\partial \mathcal{L}}{\partial \mathbf{W}}$). To compute the derivative of scalar loss $\mathcal{L}$ with respect to individual matrix element W_{ij}, we apply the multivariate chain rule across all output pre-activations z_k:

$$\frac{\partial \mathcal{L}}{\partial W_{ij}} = \sum_{k=1}^{d_{\text{out}}} \frac{\partial \mathcal{L}}{\partial z_k} \frac{\partial z_k}{\partial W_{ij}}. \tag{2.59}$$

Because $z_k = \sum_{l=1}^{d_{\text{in}}} W_{lk} x_l + b_k$, the partial derivative is non-zero only when $k = j$:

$$\frac{\partial z_k}{\partial W_{ij}} = \begin{cases} x_i & \text{if } k = j \\ 0 & \text{if } k \neq j \end{cases}. \tag{2.60}$$

Substituting this back:

$$\frac{\partial \mathcal{L}}{\partial W_{ij}} = \frac{\partial \mathcal{L}}{\partial z_j} x_i = \delta_j x_i = x_i \delta_j. \tag{2.61}$$

Collecting all elements W_{ij} into matrix form reveals that the weight gradient is identically the **outer product** of the upstream input vector $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$ and the adjoint error vector $\boldsymbol{\delta} \in \mathbb{R}^{d_{\text{out}}}$:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \mathbf{x}\boldsymbol{\delta}^\top \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}. \tag{2.62}$$

2. Bias vector adjoint derivation ($\frac{\partial \mathcal{L}}{\partial \mathbf{b}}$). For each scalar bias component b_j:

$$\frac{\partial \mathcal{L}}{\partial b_j} = \sum_{k=1}^{d_{\text{out}}} \frac{\partial \mathcal{L}}{\partial z_k} \frac{\partial z_k}{\partial b_j} = \delta_j \cdot 1 = \delta_j. \tag{2.63}$$

In vector notation:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}} = \boldsymbol{\delta} \in \mathbb{R}^{d_{\text{out}}}. \tag{2.64}$$

3. Upstream input adjoint derivation ($\frac{\partial \mathcal{L}}{\partial \mathbf{x}}$). To propagate error signals back to previous layers in a deep neural network, we must evaluate the gradient of loss $\mathcal{L}$ with respect to the input feature vector $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$. Applying the chain rule to input component x_i:

$$\frac{\partial \mathcal{L}}{\partial x_i} = \sum_{k=1}^{d_{\text{out}}} \frac{\partial \mathcal{L}}{\partial z_k} \frac{\partial z_k}{\partial x_i} = \sum_{k=1}^{d_{\text{out}}} \delta_k W_{ik} = \sum_{k=1}^{d_{\text{out}}} W_{ik} \delta_k. \tag{2.65}$$

Recognizing this sum as the i-th element of the matrix-vector multiplication between weight matrix $\mathbf{W} \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$ and adjoint vector $\boldsymbol{\delta} \in \mathbb{R}^{d_{\text{out}}}$:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \mathbf{W}\boldsymbol{\delta} \in \mathbb{R}^{d_{\text{in}}}. \tag{2.66}$$

Figure 2.10 diagrams this vectorized adjoint error propagation across the dense layer.

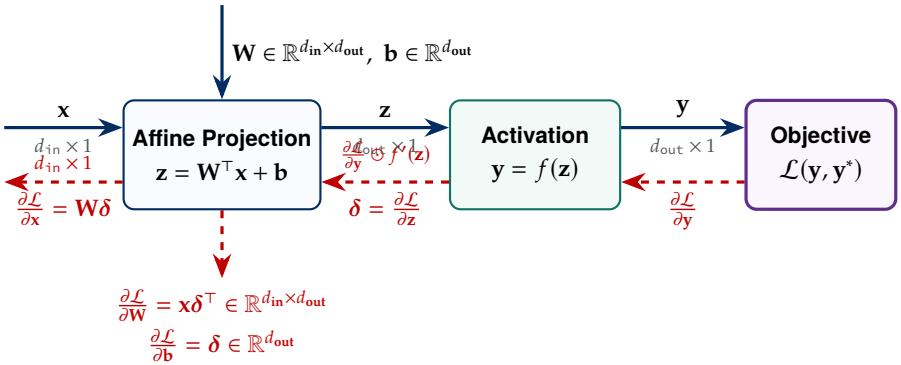


Figure 2.10: Vectorized matrix adjoint backpropagation across a fully connected dense layer. Solid navy arrows trace forward execution mapping input $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$ to output $\mathbf{y} \in \mathbb{R}^{d_{\text{out}}}$; dashed red arrows trace reverse-mode adjoint error propagation ($\boldsymbol{\delta} \in \mathbb{R}^{d_{\text{out}}}$), yielding parameter updates $\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \mathbf{x}\boldsymbol{\delta}^\top$, $\frac{\partial \mathcal{L}}{\partial \mathbf{b}} = \boldsymbol{\delta}$, and upstream gradient $\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \mathbf{W}\boldsymbol{\delta}$.

Mini-batch matrix generalization. In GPU implementations, B samples are grouped into a batch data matrix $\mathbf{X} \in \mathbb{R}^{B \times d_{\text{in}}}$. The layer computes:

$$\mathbf{Z} = \mathbf{X}\mathbf{W} + \mathbf{1}_B \mathbf{b}^\top \in \mathbb{R}^{B \times d_{\text{out}}}, \quad \mathbf{Y} = f(\mathbf{Z}) \in \mathbb{R}^{B \times d_{\text{out}}}. \tag{2.67}$$

Defining batch adjoint matrix $\boldsymbol{\Delta} = \frac{\partial \mathcal{L}}{\partial \mathbf{Z}} \in \mathbb{R}^{B \times d_{\text{out}}}$, the gradient equations generalize to highly efficient Basic Linear Algebra Subprograms (BLAS) matrix multiplications:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \mathbf{X}^\top \boldsymbol{\Delta} \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}, \tag{2.68}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}} = \boldsymbol{\Delta}^\top \mathbf{1}_B \in \mathbb{R}^{d_{\text{out}}}, \tag{2.69}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{X}} = \boldsymbol{\Delta}\mathbf{W}^\top \in \mathbb{R}^{B \times d_{\text{in}}}. \tag{2.70}$$

These matrix equations form the computational workhorse executed across billions of parameters in modern GPU training clusters.

2.7 The sequential computation impasse: bridge to recurrent neural networks

In this chapter, we have formulated the mathematical and architectural foundations of deep feedforward neural networks (FCNNs / Multi-Layer Perceptrons):

1. **Continuous Units**: The artificial perceptron computes weighted inner products over real-valued vectors.
2. **Non-Linearity**: Non-linear activations (σ, tanh, ReLU, GELU) prevent multi-layer linear stacking collapse, folding space to solve linearly inseparable problems like XOR and enabling universal function approximation.
3. **Principled Optimization**: Cross-entropy objective functions provide linear error signals without gradient saturation, and backpropagation calculates matrix adjoints across computational DAGs via vectorized matrix calculus.

The feedforward assumption and its sequential failure. Throughout these formulations, feedforward networks operate under a fundamental structural assumption: the input is a static, fixed-dimensional Euclidean vector $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$, and the network maps this vector through a cascade of static linear projections $\mathbf{W}_l \in \mathbb{R}^{d_l \times d_{l-1}}$ to produce a fixed-dimensional output $\mathbf{y} \in \mathbb{R}^{d_{\text{out}}}$:

$$\mathbf{y} = \mathcal{F}(\mathbf{x}; \Theta) = f_L(\mathbf{W}_L \cdots f_1(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) \cdots + \mathbf{b}_L). \tag{2.71}$$

While this architecture is highly effective for tabular data and fixed-attribute classification, it immediately encounters a computational impasse when confronted with sequential data—such as human natural language, software source code tokens, execution traces, and temporal signals. Sequential data violates the core assumptions of feedforward processing in three fundamental ways:

1. **Rigid Input Dimensionality**: Natural language sentences and source code blocks have elastic, variable lengths T. An FCNN strictly expects an input of dimension d_{in}, forcing artificial truncation or massive zero-padding that wastes computational throughput and memory.
2. **Zero Internal State and Temporal Memory**: Feedforward networks are memoryless. Each forward pass $\mathcal{F}(\mathbf{x})$ is independent and decoupled from prior inputs. They cannot maintain temporal context or resolve long-range grammatical and syntactic dependencies across multiple time steps.
3. **Absence of Parameter Sharing Across Positions**: In an FCNN, each position in the input is governed by separate weight parameters. If a syntactic construct (such as a variable declaration `"let x = 5;"` or a noun phrase *"a large language model"*) appears at token position 1 versus token position 20, the FCNN must learn separate weight matrices for each position independently, destroying sample efficiency.

The path forward. To process variable-length sequences, maintain evolving context over time, and achieve translational invariance across sequence positions, deep learning requires a computational paradigm with internal recurrence and temporal weight sharing.

This architectural necessity leads directly to Chapter 3: *Recurrent Neural Networks and Sequence Modeling*.

CHAPTER 3

Recurrent neural networks and sequence modeling

3.1 Sequential data and constraints of feed-forward neural networks

In Chapter 2, we formulated the mathematical and architectural foundations of deep feedforward neural networks (FCNNs) and backpropagation across computational DAGs. As established at the conclusion of that chapter, however, feedforward architectures operate under a rigid, memoryless paradigm: they assume fixed-dimensional Euclidean inputs $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$, possess no internal recurrence or persistent temporal state across sequential time steps, and cannot enforce grammatical agreement across temporal horizons.

In standard deep learning, a Fully-Connected Neural Network (FCNN) or Multi-Layer Perceptron is represented as a cascade of weight matrices interleaved with non-linear activation functions:

$$f(\mathbf{x}) = \sigma\left(\mathbf{W}_{\text{out}}\,\text{ReLU}\left(\mathbf{W}_2\,\text{ReLU}\left(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1\right) + \mathbf{b}_2\right) + \mathbf{b}_{\text{out}}\right). \tag{3.1}$$

While FCNNs excel on tabular datasets with fixed, predefined attributes, they exhibit three critical architectural constraints when applied to natural language and temporal sequences (Slides 2–5):

1. The input is expected to be fixed in size. The input dimensionality is rigidly fixed by the number of columns in the first weight matrix $\mathbf{W}_1 \in \mathbb{R}^{d_1 \times d_{\text{in}}}$. For structured tabular data, this constraint is natural because every sample contains the same predefined attributes. However, for unstructured sequential data, fixed input sizing is deeply problematic:

- Natural language sentences come in all lengths: from 2 words (*"Run now"*) to 80 words in complex legal contracts.
- Audio recordings exhibit variable sampling frequencies and durations.
- Images come in varying resolutions and aspect ratios.

Flattening a variable-length text sequence into a rigid vector $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$ requires setting an arbitrary maximum length $T_{\max}$ and applying zero-padding to shorter sentences while truncating longer ones. This wastes vast memory on empty pad tokens and destroys the elastic temporal nature of human language.

2. Lack of parameter sharing and translation invariance. In an FCNN, each position in the flattened input vector is associated with its own distinct set of weight parameters. Consequently, if the same linguistic pattern occurs in different portions of the input, the network must learn that pattern multiple times from scratch:

- In computer vision, a dog is a dog regardless of whether it appears on the left or the right side of an image. An FCNN must learn a separate "dog-detector" for the left pixels and the right pixels.
- In NLP, the noun phrase *"a dog"* possesses the exact same grammatical and semantic structure whether it appears at token position 1 (*"A dog barked loudly"*) or token position 10 (*"After a long walk in the park, a dog barked"*).

Because an FCNN does not share weights across sequence positions, it is wildly inefficient and requires massive datasets to observe every linguistic pattern at every conceivable spatial index.

3. The output is expected to be fixed in size. The output dimensionality is rigidly determined by the number of rows in the final projection matrix $\mathbf{W}_{\text{out}}$. In many sequential tasks (such as machine translation, text summarization, and dialogue generation), the output sequence length is unknown a priori and varies dynamically based on the input:

English (5 words): *"It is indeed sunny today"* $\implies$ Italian (4 words): *"Effettivamente oggi è soleggiato"*.

An FCNN cannot dynamically adjust its output layer to emit variable-length token sequences.

3.2 The recurrent neural network cell and weight sharing

To overcome the constraints of FCNNs, **Recurrent Neural Networks (RNNs)**—first formalized as the Simple Recurrent Network in [22] (as presented on Slides 6–7 of the course lecture deck)—process sequential data element by element, maintaining an internal continuous *hidden state* $\mathbf{h}_t \in \mathbb{R}^{d_h}$ that acts as a recurrent memory summarizing the sequence history observed up to time step t.

3.2.1 Recurrent cell architecture

At each discrete time step $t \in \{1, 2, \ldots, T\}$, the recurrent cell is characterized by two inputs and two outputs:

- **Input 1**: The current sequence element $\mathbf{x}_t \in \mathbb{R}^{d_{\text{in}}}$ (e.g., a one-hot vector or continuous embedding).
- **Input 2**: The hidden state produced in the previous time step $\mathbf{h}_{t-1} \in \mathbb{R}^{d_h}$.
- **Output 1**: The model's prediction for the current time step $\hat{\mathbf{y}}_t \in \mathbb{R}^{d_{\text{out}}}$.
- **Output 2**: The updated hidden state $\mathbf{h}_t \in \mathbb{R}^{d_h}$, which is recursively forwarded to time step $t + 1$.

3.2.2 Mathematical formulation with explicit tensor dimensions

Internally, the vanilla RNN cell comprises an input projection layer, a recurrent state transition layer, and an output projection layer (Slide 12):

$$\underbrace{\mathbf{h}_t}_{[d_h\times 1]} = \tanh\left(\underbrace{\mathbf{W}_{hh}}_{[d_h\times d_h]}\underbrace{\mathbf{h}_{t-1}}_{[d_h\times 1]} + \underbrace{\mathbf{W}_{xh}}_{[d_h\times d_{\text{in}}]}\underbrace{\mathbf{x}_t}_{[d_{\text{in}}\times 1]} + \underbrace{\mathbf{b}_h}_{[d_h\times 1]}\right), \tag{3.2}$$

$$\underbrace{\hat{\mathbf{y}}_t}_{[V\times 1]} = \text{softmax}\left(\underbrace{\mathbf{W}_{hy}}_{[V\times d_h]}\underbrace{\mathbf{h}_t}_{[d_h\times 1]} + \underbrace{\mathbf{b}_y}_{[V\times 1]}\right). \tag{3.3}$$

3.2.3 Weight sharing across time

The defining feature of RNNs is **temporal weight sharing** (Slide 13). The parameter matrices $\mathbf{W}_{xh}$, $\mathbf{W}_{hh}$, and $\mathbf{W}_{hy}$ are *strictly identical* across all time steps $t \in \{1, \ldots, T\}$. Because the same weights are applied regardless of time step, the model develops **translation invariance**: if the same context occurs at step $t = 2$ or step $t = 50$, the recurrent cell behaves identically. The network learns grammatical rules (such as capitalizing the first letter after a quotation mark) once, and applies that rule everywhere in the text.

Intuition & analogy: Recurrent dynamics as a trajectory through high-dimensional phase space

As visualized in geometric visual analyses [23], the hidden state $\mathbf{h}_t \in \mathbb{R}^{d_h}$ can be understood geometrically as the position of a particle traveling through a continuous d_h-dimensional *phase space*:

- At rest ($t = 0$), the system sits at initial coordinate $\mathbf{h}_{-1} = \mathbf{0}$.
- When each discrete input token $\mathbf{x}_t$ arrives, the linear projection $\mathbf{W}_{xh}\mathbf{x}_t$ applies an external directional impulse or "vector nudge" to the particle.
- Simultaneously, the recurrent transformation $\mathbf{W}_{hh}\mathbf{h}_{t-1}$ rotates, scales, and folds the previous position, while the non-linear squashing activation tanh confines the entire state space within the hypercube $(-1, 1)^{d_h}$.
- The unfolding sequence of text traces a smooth continuous trajectory through state space. Regions of this phase space function as **attractor basins**: for instance, reading an opening quote mark (") knocks the state into a specific sub-manifold where the emission projection $\mathbf{W}_{hy}$ naturally maps coordinates into uppercase character distributions. The state remains trapped in this basin until an exit trigger (such as a whitespace or subsequent character) forces the trajectory out into the lowercase manifold.

3.2.4 Unrolling the recurrent cell across time

To trace how an RNN processes a sequence, we *unroll* the recursive feedback loop across discrete time steps $t = 0, 1, \ldots, T$ (Slides 8–11).

Consider the character casing task:

- Input sequence: `"you are very kind, mr.  holmes, but i cannot do that"`
- Target sequence: `"You are very kind, Mr.  Holmes, but I cannot do that"`

Each ASCII character is represented as a 256-dimensional one-hot vector $\mathbf{x}_t \in \{0,1\}^{256}$:

- **Step** $t = 0$: Receives input $s_0 =$ `"` with initial state $\mathbf{h}_{-1} = \mathbf{0}$. The cell produces state $\mathbf{h}_0$. The network realizes: *"We just encountered an opening quote (`"`), which signals the beginning of dialogue; the following character should be capitalized."*
- **Step** $t = 1$: Receives input $s_1 =$ `y` and prior state $\mathbf{h}_0$. The network emits output $\hat{y}_1 =$ `Y` and produces updated state $\mathbf{h}_1$. The state records: *"We just emitted a capitalized letter at the start of dialogue; following letters in this word should probably be lowercase."*
- **Step** $t = 2$: Receives input $s_2 =$ `o` and prior state $\mathbf{h}_1$. The network emits output $\hat{y}_2 =$ `o` and produces state $\mathbf{h}_2$, continuing lowercase word progression.

Figure 3.1 illustrates this unrolled computational structure and the backward flow of gradients.

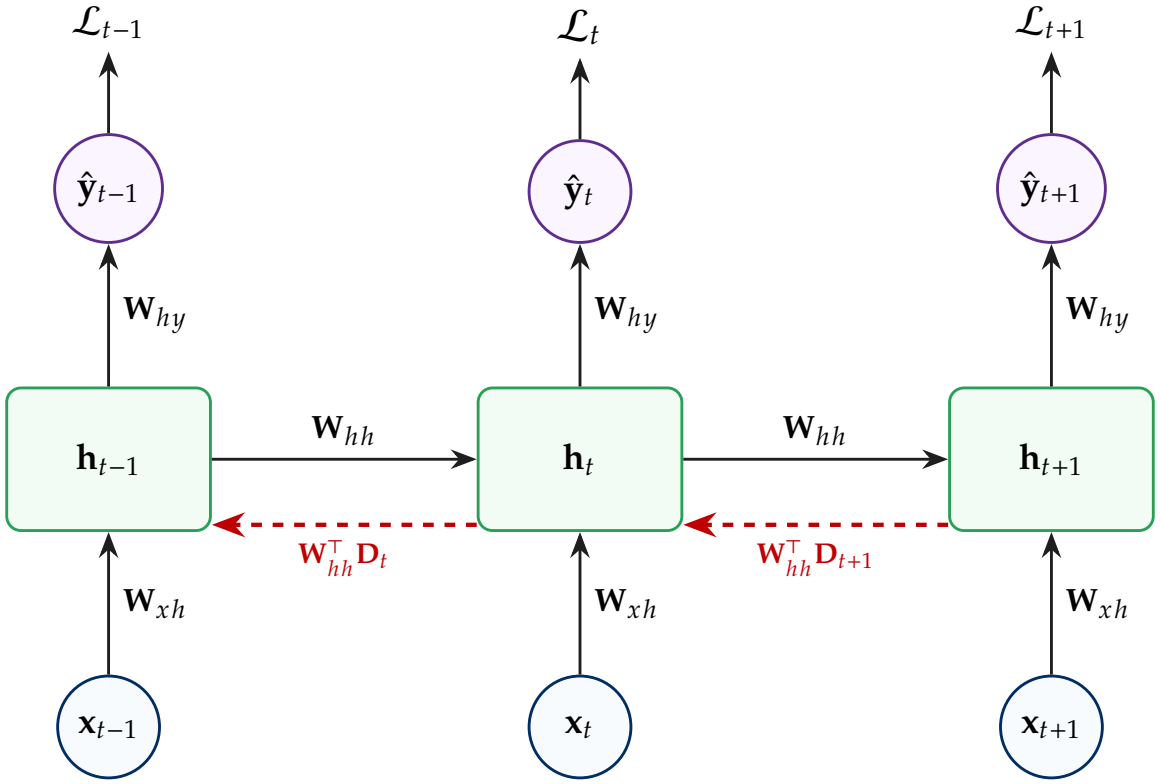


Figure 3.1: Unrolling an RNN cell across time steps $t-1, t, t+1$. Solid black arrows show the forward propagation of inputs $\mathbf{x}_t$ and recurrent states $\mathbf{h}_t$. Dashed red arrows trace Backpropagation Through Time (BPTT), where gradients accumulate across time via repeated multiplications by the Jacobian $\mathbf{W}_{hh}^\top \mathbf{D}_t$.

3.3 Backpropagation through time and the proof of vanishing gradients

When an RNN processes a sequence of length T, the computational graph is unrolled T times. The total sequence loss is the sum of cross-entropy losses incurred at each time step (Slides

14–17):

$$\mathcal{L} = \sum_{t=1}^{T} \mathcal{L}_t. \tag{3.4}$$

To optimize the shared recurrent weight matrix $\mathbf{W}_{hh}$ via Gradient Descent, we apply **Backpropagation Through Time (BPTT)**, first formalized for recurrent structures in [24]. The fundamental difficulty of learning long-term dependencies through gradient descent was rigorously analyzed in [25] and [26] (and emphasized on Slides 14–17 of the course lecture deck). The gradient of the total loss with respect to $\mathbf{W}_{hh}$ accumulates the contributions across all time steps:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{hh}} = \sum_{t=1}^{T} \sum_{k=1}^{t} \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} \frac{\partial^+ \mathbf{h}_k}{\partial \mathbf{W}_{hh}}, \tag{3.5}$$

where $\frac{\partial^+ \mathbf{h}_k}{\partial \mathbf{W}_{hh}}$ denotes the direct derivative of $\mathbf{h}_k$ with respect to $\mathbf{W}_{hh}$ treating $\mathbf{h}_{k-1}$ as constant.

The temporal transmission term $\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k}$ represents the sensitivity of state $\mathbf{h}_t$ to an earlier state $\mathbf{h}_k$ ($t > k$). By the multivariate chain rule, this sensitivity decomposes into a product of single-step Jacobian matrices:

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} = \prod_{j=k+1}^{t} \frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}}. \tag{3.6}$$

3.3.1 Rigorous mathematical proof of gradient pathology

We now derive the explicit single-step Jacobian and prove why vanilla RNNs suffer from vanishing and exploding gradients.

Theorem 3.1: *Jacobian product and spectral radius gradient decay*

Let the recurrent state transition be $\mathbf{h}_j = \tanh(\mathbf{z}_j)$ where $\mathbf{z}_j = \mathbf{W}_{hh}\mathbf{h}_{j-1} + \mathbf{W}_{xh}\mathbf{x}_j + \mathbf{b}_h$ with $\mathbf{h}_j \in \mathbb{R}^{d_h}$ and $\mathbf{W}_{hh} \in \mathbb{R}^{d_h \times d_h}$. Under standard column-vector convention, the single-step forward temporal Jacobian $\mathbf{J}_j = \frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}} \in \mathbb{R}^{d_h \times d_h}$ is:

$$\mathbf{J}_j = \frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}} = \operatorname{diag}\left(1 - \mathbf{h}_j^2\right) \mathbf{W}_{hh} = \mathbf{D}_j \mathbf{W}_{hh}, \tag{3.7}$$

where $\mathbf{D}_j = \operatorname{diag}(1 - \tanh^2(\mathbf{z}_j)) \in \mathbb{R}^{d_h \times d_h}$ is a diagonal matrix containing the local derivatives of the hyperbolic tangent function. Consequently, the multi-step temporal transmission evaluates to:

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} = \prod_{j=k+1}^{t} \mathbf{D}_j \mathbf{W}_{hh}. \tag{3.8}$$

When transmitting the gradient of a scalar loss $\mathcal{L}_t$ backward through time, the column-vector gradient $\nabla_{\mathbf{h}_{j-1}} \mathcal{L}_t \in \mathbb{R}^{d_h}$ satisfies the adjoint recurrence $\nabla_{\mathbf{h}_{j-1}} \mathcal{L}_t = \mathbf{W}_{hh}^\top \mathbf{D}_j \nabla_{\mathbf{h}_j} \mathcal{L}_t$. Let $\lambda_{\max} = \|\mathbf{W}_{hh}\|_2 = \sigma_{\max}(\mathbf{W}_{hh})$ denote the spectral norm (maximum singular value) of $\mathbf{W}_{hh}$. The Euclidean norm of the transmitted gradient is bounded by:

$$\left\| \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_k} \right\| \leq \left\| \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} \right\| (\lambda_{\max})^{t-k}. \tag{3.9}$$

Proof. Differentiating the i-th component of $\mathbf{h}_j$ with respect to the m-th component of $\mathbf{h}_{j-1}$:

$$\frac{\partial h_{j,i}}{\partial h_{j-1,m}} = \frac{\partial \tanh(z_{j,i})}{\partial z_{j,i}} \frac{\partial z_{j,i}}{\partial h_{j-1,m}} = (1 - h_{j,i}^2) W_{hh,i,m}.$$

In matrix notation, the entry in row i and column m of $\frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}}$ is $(1 - h_{j,i}^2) W_{hh,i,m} = (\mathbf{D}_j)_{ii} W_{hh,i,m}$, which corresponds exactly to the matrix product $\mathbf{D}_j \mathbf{W}_{hh}$. Taking the transpose for backward column-vector gradient propagation yields $(\mathbf{D}_j \mathbf{W}_{hh})^\top = \mathbf{W}_{hh}^\top \mathbf{D}_j$, matching the backward adjoint arrows in Figure 3.1. Because $|\tanh'(z)| = 1 - \tanh^2(z) \in [0, 1]$, the operator 2-norm of each diagonal matrix satisfies $\|\mathbf{D}_j\|_2 \leq 1$. Applying the sub-multiplicative property of matrix norms to Equation (3.8):

$$\begin{aligned} \|\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k}\|_2 &= \|\prod_{j=k+1}^{t} \mathbf{D}_j \mathbf{W}_{hh}\|_2 \leq \prod_{j=k+1}^{t} \|\mathbf{D}_j\|_2 \|\mathbf{W}_{hh}\|_2 \\ &\leq \prod_{j=k+1}^{t} 1 \cdot \lambda_{\max} = (\lambda_{\max})^{t-k}. \end{aligned}$$

Multiplying by $\|\frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t}\|_2$ yields Equation (3.9). □

Two pathological regimes (slides 21–23):

1. **Vanishing Gradient ($\lambda_{\max} < 1$)**: If the singular values of $\mathbf{W}_{hh}$ are slightly less than 1 (e.g., $\lambda_{\max} = 0.9$), over a temporal distance of 50 steps:

   $$0.9^{50} \approx 0.00515 \quad \text{(over 99.5\% of the gradient signal is extinguished).}$$

   Over 100 steps, $0.9^{100} \approx 0.000026$. The gradient decays exponentially to zero. Parameter updates from distal tokens vanish, meaning the network cannot learn long-range temporal dependencies. Saturated activations (such as $\sigma'(z) \leq 0.25$) accelerate this decay exponentially.

2. **Exploding Gradient ($\lambda_{\max} > 1$)**: If the singular values exceed 1 (e.g., $\lambda_{\max} = 1.1$), over 50 steps:

   $$1.1^{50} \approx 117.39.$$

   Over 100 steps, $1.1^{100} \approx 13{,}780$. Gradients grow exponentially, destabilizing training, inducing severe numerical overflow, and producing `NaN` parameter corruptions. (Practitioners counter exploding gradients using heuristic *gradient clipping*: $\mathbf{g} \leftarrow \mathbf{g} \cdot \min(1, \frac{\text{threshold}}{\|\mathbf{g}\|})$).

The long-term dependency degradation (slides 24–25). Because hidden states are rewritten at every time step without persistent memory protection, vanilla RNNs suffer from catastrophic recency bias: they remember only the most recent tokens. When tested on long-range word prediction:

*"The **mouse** was getting chased by the cat through the big house. After a long chase, the cat finally managed to catch the _____"*

the target word is unambiguously *"mouse"*. However, vanilla RNNs fail completely because the hidden state has overwritten the subject *"mouse"* across the intervening 20 tokens.

3.4 Case correction demonstration: Sherlock Holmes training dynamics

To demonstrate how an RNN learns sequential syntax in practice, Slides 18–20 document an empirical case study trained on Arthur Conan Doyle's *The Adventures of Sherlock Holmes*:

Experimental configuration.

- **Corpus**: Raw text from Arthur Conan Doyle's *The Adventures of Sherlock Holmes*.
- **Sequence Length**: Subsequences sliced between 10 and 100 ASCII characters.
- **Token Representation**: 256-dimensional one-hot vectors ($d_{\text{in}} = 256$).
- **Architecture**: Single-layer vanilla RNN with hidden dimension $d_h = 32$.
- **Optimization**: Adam optimizer (lr = 0.001), batch size 32, trained for 10 epochs.
- **Padding Mechanism**: Null bytes (`0x00`, rendered visually as `X`) used to pad variable-length sequences to 100 characters.

Fixed evaluation test sentence. The model is evaluated after each epoch on a fixed unseen test string:

- **Input Test Sequence**: `upon his pale face.  "it may be so, or it may not, mr. holmes," said he, "but if you are so very sharp`
- **Target Test Sequence**: `upon his pale face.  "It may be so, or it may not, Mr. Holmes," said he, "but if you are so very sharp`

Qualitative phase transitions across 10 epochs. Table 3.1 records the loss trajectory and qualitative phase transitions across training:

3.4.1 Character-level modeling, temperature sampling, and hidden state interpretability

The Sherlock Holmes casing experiment exemplifies a broader paradigm analyzed in the landmark study *"The Unreasonable Effectiveness of Recurrent Neural Networks"* [27]: **character-level sequence modeling**.

1. The elegance of character-level modeling. Unlike word-level models that require complex tokenizers, morphological analyzers, and massive vocabularies ($V \geq 50{,}000$), character-level RNNs operate over an extremely compact alphabet (e.g., standard 256-dimensional ASCII).

- There are **zero Out-of-Vocabulary (OOV) tokens**: any arbitrary string, code snippet, mathematical formula, or neologism can be represented as an sequence of bytes.
- The model is forced to discover the hierarchical structure of language entirely from scratch: it must learn how characters form words, that words are separated by spaces, how punctuation delimits clauses, and how grammatical agreement spans across clauses.

Table 3.1: Epoch-by-epoch loss progression and qualitative phase transitions on the Sherlock Holmes casing task.

Epoch	Loss	Observed Generation & Qualitative Linguistic Transition
01	3.1681	`XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX` *Phase 1 (Null-byte frequency)*: The model discovers that null padding bytes (`0x00` / `X`) appear with high statistical frequency due to padding imbalance, predicting all `X`'s.
02	2.3334	`eXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX` *Phase 2 (Vowel emergence)*: The model predicts `e` (the most frequent character in English orthography) alongside padding characters.
03	1.7436	`tst X s te e eee t e e e te se ee se e e ee eee XXXee` *Phase 3 (Whitespace and unigrams)*: The model learns that whitespace characters occur regularly, predicting spaces, `e`, `t`, and `s`.
04	0.9519	`upon his pale face.  cit may be sos or it may nots mr.  holresst` *Phase 4 (The Identity Breakthrough)*: A massive drop in loss occurs when the network learns to forward lowercase letters directly from input to output (`W.grad` captures the identity diagonal).
05–06	0.3579	`upon his pale face.  "it may be so, or it may not, mr.  holmes,"` *Phase 5 (Punctuation and Syntax)*: The model captures quotation marks (`"`), commas, and spaces accurately.
07–08	0.1009	`upon his pale face.  "It may be so, or it may not, mr.  holmes,"` *Phase 6 (Quote Capitalization)*: The model learns that characters immediately following opening quotes (`"`) must be capitalized (`"It`).
09–10	0.0515	`upon his pale face.  "It may be so, or it may not, mr.  Holmes,"` *Phase 7 (Period Capitalization and Causal Blindness)*: The model capitalizes `Holmes` because it follows a period (`.`). Crucially, the model **fails to capitalize** `mr.` because it cannot see into the future! It does not know that `m` begins the title `Mr.` until the `r.` arrives.

2. Autoregressive sampling and temperature dynamics. During text generation, the RNN emits an unnormalized logit vector $\mathbf{z}_t = \mathbf{W}_{hy}\mathbf{h}_t + \mathbf{b}_y \in \mathbb{R}^V$. Rather than taking the deterministic arg max, we convert logits into a sampling distribution modulated by a scalar hyperparameter: the **temperature** $T > 0$:

$$\mathbb{P}(y_t = c \mid \mathbf{h}_t) = \frac{\exp\left(\frac{z_{t,c}}{T}\right)}{\sum_{j=1}^{V} \exp\left(\frac{z_{t,j}}{T}\right)}. \tag{3.10}$$

The temperature controls the entropy and randomness of the emitted sequence:

- **Low Temperature ($T \to 0$ / Greedy Decoding)**: Exaggerates differences between logits. The largest logit absorbs almost all probability mass ($p_{\max} \to 1$). The model makes confident, conservative predictions, but frequently falls into pathological mode collapse—getting trapped in infinite repetitive loops (e.g., `"the the the the"` or repeating identical sentence fragments).
- **Balanced Temperature ($T \in [0.6, 0.8]$)**: Preserves the syntactic and morphological structure discovered by the network while introducing sufficient stochastic diversity to produce coherent, rich generations. In empirical experiments demonstrated in [27], character-level LSTMs trained at $T = 0.7$ generated structurally sound Shakespearean plays (including character names, stage directions, and poetic meter), syntactically compilable C code from the Linux kernel, and valid algebraic geometry LaTeX documents.
- **High Temperature ($T > 1.2$)**: Flattens the probability distribution toward a uniform distribution ($p_c \to 1/V$). The network makes chaotic, exploratory predictions, generating misspelled words, hallucinated punctuation, and unpronounceable character noise.

3. Interpretable hidden state neurons. By inspecting the internal cell activations $\mathbf{h}_t$ of a trained LSTM as it reads character sequences, the empirical investigation in [27] revealed that individual hidden units act as interpretable linguistic finite-state machines:

1. **The Quote Detection Neuron**: A single scalar unit in $\mathbf{h}_t$ that fires strongly positive (+1) the moment an opening quotation mark (`"`) is parsed, remains persistently active throughout the enclosed dialogue, and resets immediately to -1 when the matching closing quote mark is encountered.
2. **The Code Indentation / Scope Depth Neuron**: In code generation models, a specific neuron's activation ramps up linearly with every tab or whitespace indentation inside nested loops or conditionals, tracking variable scope depth and resetting when closing brackets (`}`) appear.
3. **The Line Length Tracker**: A unit whose activation monotonically increases across 80 characters, anticipating when to emit the carriage return (`\n`) character to conform to standard formatting conventions.

4. Classroom insights: causal lookahead blindness and overfitting dynamics. In the Politecnico di Torino course lecture, two essential practical takeaways are highlighted from the Sherlock Holmes casing demonstration:

- **Causal Lookahead Blindness**: Why did the one-to-one RNN fail to capitalize `"mr."` into `"Mr."` even after 10 epochs? The model operates strictly causally from left to right. When processing the character `'m'`, the model only sees past history. It cannot see into the future to know whether `'m'` begins the proper title `"Mr."` or common words like `"me"`, `"my"`, or `"much"`. Without lookahead, capitalizing `'m'` would be an unsupported guess. Conversely, the model successfully capitalizes `"Holmes"` because it has already observed the preceding period and space (`". h"`), triggering capitalization.

- **Overfitting vs. Generalization Dynamics**: When addressing why training was stopped at 10 epochs during the course lecture demo, it was emphasized that extending training for hundreds of epochs on a finite corpus (such as Doyle's novel) causes the model to memorize specific character n-grams verbatim. While training loss continues to drop toward zero, the network loses its ability to generalize to novel sentence structures, reinforcing the necessity of early stopping and validation monitoring emphasized throughout the course.

3.5 Gated architectures: LSTM and GRU

To solve the vanishing gradient problem and allow neural networks to retain dependencies over hundreds of time steps, researchers introduced **Gated RNNs** (Slide 26–29):

3.5.1 Long short-term memory (LSTM)

Introduced in [28] and covered on Slides 18–25 of the course lecture deck, the **Long Short-Term Memory (LSTM)** network partitions memory into two separate states:

1. **Cell State $\mathbf{c}_t \in \mathbb{R}^{d_h}$**: The *long-term memory* highway, which runs straight down the entire chain with minimal linear interactions.
2. **Hidden State $\mathbf{h}_t \in \mathbb{R}^{d_h}$**: The *short-term memory*, which acts as the visible output emitted at each step.

Definition 3.1: *LSTM gating equations*

At time step t, given input $\mathbf{x}_t \in \mathbb{R}^{d_{\text{in}}}$ and previous hidden state $\mathbf{h}_{t-1} \in \mathbb{R}^{d_h}$, the LSTM evaluates three sigmoid gates and a candidate state:

$$\text{Forget Gate:} \quad \mathbf{f}_t = \sigma\left(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f\right) \in (0,1)^{d_h}, \tag{3.11}$$

$$\text{Input Gate:} \quad \mathbf{i}_t = \sigma\left(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i\right) \in (0,1)^{d_h}, \tag{3.12}$$

$$\text{Candidate Cell:} \quad \tilde{\mathbf{c}}_t = \tanh\left(\mathbf{W}_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c\right) \in (-1,1)^{d_h}, \tag{3.13}$$

$$\text{Cell State Update:} \quad \mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t \in \mathbb{R}^{d_h}, \tag{3.14}$$

$$\text{Output Gate:} \quad \mathbf{o}_t = \sigma\left(\mathbf{W}_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o\right) \in (0,1)^{d_h}, \tag{3.15}$$

$$\text{Hidden State Update:} \quad \mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \in (-1,1)^{d_h}, \tag{3.16}$$

where $\odot$ denotes the element-wise Hadamard product, and $[\mathbf{h}_{t-1}, \mathbf{x}_t] \in \mathbb{R}^{d_h + d_{\text{in}}}$ denotes vector concatenation.

Semantic roles of the gates (slide 29): Each gate squashes pre-activations through a sigmoid function $\sigma \in [0,1]$, functioning as a continuous valve:

- Multiplying by $0 \implies$ drop/forget everything.
- Multiplying by $1 \implies$ preserve/pass everything.
- **Forget Gate ($\mathbf{f}_t$)**: Regulates what proportion of the past cell state $\mathbf{c}_{t-1}$ to discard. (e.g., when a closing quote or period arrives, the forget gate purges past sentence subjects).
- **Input Gate ($\mathbf{i}_t$)**: Regulates what proportion of the new candidate information $\tilde{\mathbf{c}}_t$ to write into the long-term cell state.

- **Output Gate ($\mathbf{o}_t$)**: Regulates which portions of the internal cell state $\mathbf{c}_t$ should be revealed as visible hidden state $\mathbf{h}_t$ for current output predictions.

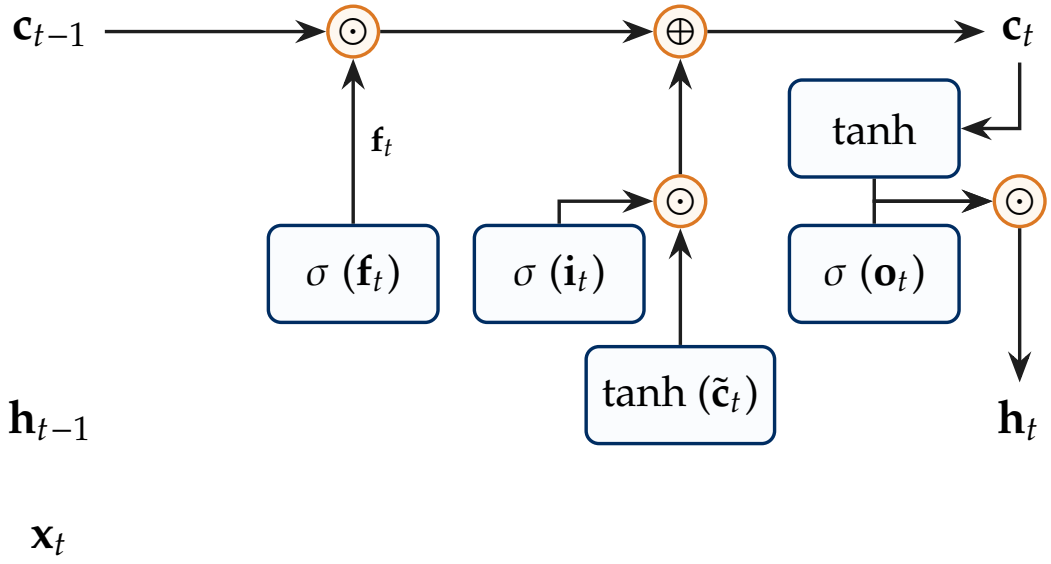


Figure 3.2: The LSTM memory cell architecture. The top horizontal path forms the uninterrupted additive cell state highway $\mathbf{c}_t$ (the Constant Error Carousel). Forget gate $\mathbf{f}_t$, input gate $\mathbf{i}_t$, and output gate $\mathbf{o}_t$ modulate information flow using multiplicative valves.

Theorem 3.2: *The Constant Error Carousel proof*

In an LSTM, the gradient of the loss with respect to past cell state $\mathbf{c}_{t-1}$ contains an additive direct path:

$$\frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_{t-1}} = \mathbf{f}_t + \frac{\partial \mathbf{f}_t}{\partial \mathbf{c}_{t-1}}\mathbf{c}_{t-1} + \frac{\partial \mathbf{i}_t}{\partial \mathbf{c}_{t-1}}\tilde{\mathbf{c}}_t + \frac{\partial \tilde{\mathbf{c}}_t}{\partial \mathbf{c}_{t-1}}\mathbf{i}_t. \tag{3.17}$$

When the network sets forget gate $\mathbf{f}_t \approx \mathbf{1}$, the leading derivative collapses to $\frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_{t-1}} \approx \mathbf{I}$. Consequently, over any arbitrary temporal horizon $T - k$:

$$\frac{\partial \mathbf{c}_T}{\partial \mathbf{c}_k} = \prod_{j=k+1}^{T} \frac{\partial \mathbf{c}_j}{\partial \mathbf{c}_{j-1}} \approx \prod_{j=k+1}^{T} \mathbf{I} = \mathbf{I}. \tag{3.18}$$

Gradients flow backward across hundreds of time steps without exponential decay. This linear additive gradient highway is the **Constant Error Carousel (CEC)**.

Intuition & analogy: The conveyor belt of memory

Think of the cell state $\mathbf{c}_t$ as a continuous factory conveyor belt running from the start of the sequence to the end. The standard recurrent transition in a vanilla RNN continually picks up the items, squashes them with weights and tanh, and puts a degraded version back on the belt. Over 50 time steps, the original items are unrecognizable.
In an LSTM, items on the conveyor belt are untouched by default. As the belt passes each station:

1. The forget gate $\mathbf{f}_t$ acts as a discard mechanism: if $\mathbf{f}_t \approx 1$, the past item rides past undisturbed; if $\mathbf{f}_t \approx 0$, it is swept off the belt.
2. The input gate $\mathbf{i}_t$ adds new relevant cargo $\tilde{\mathbf{c}}_t$ onto the belt via simple vector addition ($\oplus$).
3. The output gate $\mathbf{o}_t$ inspects the cargo on the belt, reads off a copy, and displays it to the outside world as hidden state $\mathbf{h}_t$, leaving the underlying cell state intact on the highway.

Because writing is additive rather than multiplicative, information can ride the conveyor belt across hundreds of words without experiencing exponential decay.

3.5.2 Gated recurrent unit (GRU)

Introduced in [29] and analyzed on Slides 28–32 of the course lecture deck, the **Gated Recurrent Unit (GRU)** simplifies the LSTM architecture by merging the cell state and hidden state into a single recurrent state $\mathbf{h}_t$ and coupling the forget and input gates into a single *update gate* $\mathbf{z}_t$:

Definition 3.2: *GRU equations*

At time step t, given input $\mathbf{x}_t$ and prior state $\mathbf{h}_{t-1}$:

$$\text{Update Gate:} \quad \mathbf{z}_t = \sigma\left(\mathbf{W}_z[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_z\right), \tag{3.19}$$

$$\text{Reset Gate:} \quad \mathbf{r}_t = \sigma\left(\mathbf{W}_r[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_r\right), \tag{3.20}$$

$$\text{Candidate Hidden State:} \quad \tilde{\mathbf{h}}_t = \tanh\left(\mathbf{W}_h[\mathbf{r}_t \odot \mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_h\right), \tag{3.21}$$

$$\text{Hidden State Interpolation:} \quad \mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t. \tag{3.22}$$

Physical gate routing and dynamic memory modulation. The GRU cell operates through two coupled mechanisms illustrated in Figure 3.3:

1. **Reset Gate Modulation ($\mathbf{r}_t$)**: The reset gate vector $\mathbf{r}_t \in (0,1)^{d_h}$ acts as a dynamic filter applied to the previous hidden state $\mathbf{h}_{t-1}$ via Hadamard multiplication ($\mathbf{r}_t \odot \mathbf{h}_{t-1}$). If $\mathbf{r}_t \approx \mathbf{0}$, the candidate state $\tilde{\mathbf{h}}_t$ completely ignores past memory, resetting the representation to depend exclusively on the current token $\mathbf{x}_t$ (e.g., at sentence boundaries or clause resets). If $\mathbf{r}_t \approx \mathbf{1}$, the full history flows into the candidate calculation.
2. **Update Gate Convex Interpolation ($\mathbf{z}_t$)**: Rather than using distinct input and forget gates like the LSTM, the GRU couples memory retention and candidate addition into a single convex combination:
$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t.$$
When $\mathbf{z}_t \approx \mathbf{0}$, the unit passes the previous state $\mathbf{h}_{t-1}$ forward unchanged ($1 \odot \mathbf{h}_{t-1} + 0 \odot \tilde{\mathbf{h}}_t = \mathbf{h}_{t-1}$), establishing an unobstructed linear identity shortcut similar to the LSTM Constant Error Carousel. When $\mathbf{z}_t \approx \mathbf{1}$, the unit overwrites the state entirely with the newly proposed candidate $\tilde{\mathbf{h}}_t$.

🎓 **Exam insights: Parameter counting: vanilla RNN vs. LSTM vs. GRU**

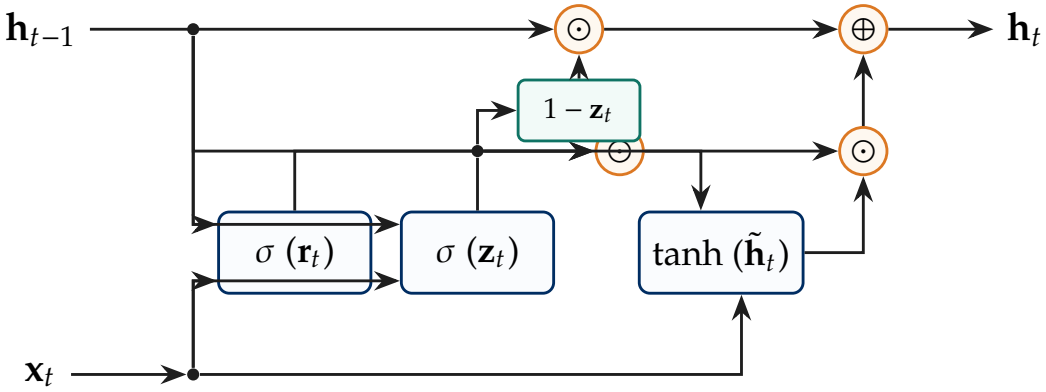


Figure 3.3: The Gated Recurrent Unit (GRU) cell architecture. Reset gate $\mathbf{r}_t$ dynamically filters previous hidden state $\mathbf{h}_{t-1}$ before constructing candidate state $\tilde{\mathbf{h}}_t$. Update gate $\mathbf{z}_t$ controls the convex interpolation $(1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t$, unifying long- and short-term memory into a single state vector $\mathbf{h}_t$.

A classic written examination question asks students to compare the exact parameter counts of recurrent cells for hidden dimension d and input dimension m:

- **Vanilla RNN**: Contains 1 transition set $(\mathbf{W}_{hh}, \mathbf{W}_{xh}, \mathbf{b}_h) \implies d^2+dm+d = d(d+m+1)$.
- **GRU**: Contains 3 gate sets $(\mathbf{z}_t, \mathbf{r}_t, \tilde{\mathbf{h}}_t) \implies 3 \times d(d + m + 1)$.
- **LSTM**: Contains 4 gate sets $(\mathbf{f}_t, \mathbf{i}_t, \tilde{\mathbf{c}}_t, \mathbf{o}_t) \implies 4 \times d(d + m + 1)$.

The GRU reduces parameters by 25% compared to LSTM while exhibiting comparable empirical performance on moderate sequence lengths.

3.6 Sequence-to-sequence (Seq2Seq) and encoder-decoder pipelines

3.6.1 Limitations of one-to-one sequence mapping

Thus far, we have considered models where each input step produces exactly one output step ($|X| = |Y|$), such as character-level case correction. However, the majority of core NLP tasks (machine translation, question answering, code generation) violate this assumption (Slides 30–32):

1. **Unequal Sequence Lengths**: In machine translation, source and target languages rarely match in word count:

   English (5 words): *[it] [is] [indeed] [sunny] [today]* $\implies$ Italian (4 words): *[effettivamente] [oggi] [è] [soleggiato]*.

   A one-to-one recurrent model would process 5 tokens and be forced to produce 5 output tokens.

2. **Non-Monotonic Alignment and Causal Blindness**: Natural languages exhibit different word orders. In the example above, the Italian word *"effettivamente"* must be emitted at step 1, but its English counterpart *"indeed"* does not appear until step 3! In a standard one-to-one model, the cell cannot see the future, rendering early word generation impossible.

3.6.2 The encoder-decoder architecture

To overcome these limitations, the **Encoder-Decoder (Seq2Seq)** architecture was introduced in [30] and [29] (as explored on Slides 33–34 of the course lecture deck):

1. **Encoder**: An LSTM processes the entire variable-length source sequence $(x_1, \ldots, x_N)$ step by step, updating its internal state. The final hidden state $\mathbf{h}_N$ acts as a dense, compressed summary vector encoding the semantics of the whole input sentence.

2. **Decoder**: A separate LSTM initialized with state $\mathbf{h}_N$. The decoder autoregressively emits target tokens step by step. The initial input token is the special Beginning of Sequence symbol ⟨BOS⟩. At each subsequent step t, the input is the token emitted at step $t-1$. Generation continues until the model emits the special End of Sequence symbol ⟨EOS⟩.

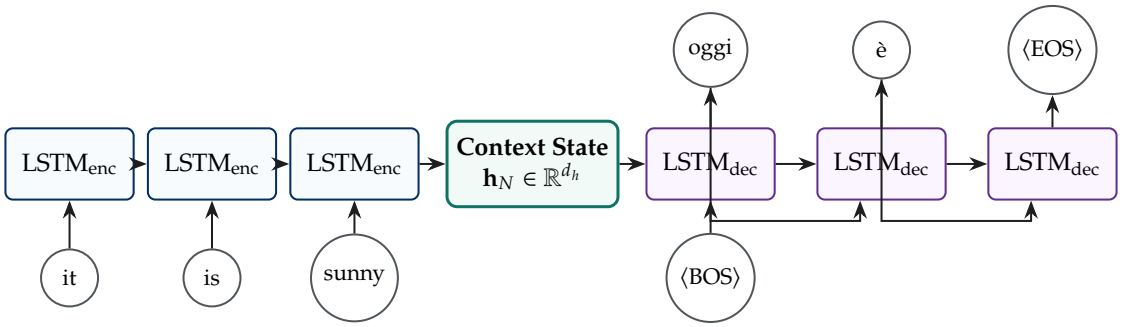


Figure 3.4: The Sequence-to-Sequence (Seq2Seq) Encoder-Decoder architecture. The Encoder LSTM compresses the variable-length source sentence into a single context state $\mathbf{h}_N$. The Decoder LSTM is initialized with $\mathbf{h}_N$ and autoregressively emits target tokens from ⟨BOS⟩ until emitting ⟨EOS⟩.

3.6.3 Teacher forcing vs. inference exposure bias

During training, the decoder's predicted token $\hat{y}_{t-1}$ is often incorrect in early epochs. If the model feeds its own faulty prediction into step t, errors compound catastrophically down the sequence.

To stabilize and accelerate training, practitioners use **Teacher Forcing** (Slide 34):

$$\mathbf{x}_{\text{dec},t} = y^*_{t-1} \quad \text{(pass true ground-truth target token instead of predicted } \hat{y}_{t-1}\text{)}. \tag{3.23}$$

Exposure Bias Warning: While Teacher Forcing provides rapid convergence during training, it creates a discrepancy at inference time when ground-truth tokens are unavailable, causing models to be fragile when recovering from early generation mistakes.

3.6.4 Sherlock Holmes case correction revised: one-to-one vs. encoder-decoder

To empirically contrast the representational power of one-to-one recurrent mapping against the Sequence-to-Sequence Encoder-Decoder architecture, Slides 35 and 36 evaluate both models on an identical, challenging character casing sentence:

Input Test Sequence: `"indeed i am sherlock holmes,"`
Target Test Sequence: `"Indeed I am Sherlock Holmes,"`

Theoretical expectations from architectural constraints (Slide 35). This evaluation sentence contains four distinct capitalization targets, each testing a specific causal and temporal capacity:

1. **`"I"` of `"Indeed"` (beginning of quote)**: Should be capitalized because it is immediately preceded by an opening quotation mark (`"`). A one-to-one model can easily learn this local transition, as the trigger character is already present in its past recurrent state $\mathbf{h}_{t-1}$.

2. **`"I"` of `"I"` (standalone subject pronoun)**: Must be capitalized because it functions as the first-person singular pronoun. A one-to-one model is structurally blind here: when reading `'i'`, the network only knows that a word is beginning with `'i'`. It cannot determine whether this character is an isolated pronoun or the initial letter of common words like `"in"`, `"it"`, or `"into"` until the subsequent whitespace arrives at step $t+1$. Without future lookahead, capitalizing `'i'` at step t is mathematically unsupported.

3. **`"S"` of `"Sherlock"` (proper noun first name)**: Must be capitalized because it begins the proper noun *Sherlock*. When the one-to-one cell receives `'s'`, it only observes an arbitrary lowercase word beginning with `'s'`. It cannot anticipate that the subsequent characters will form the name `"herlock"`.

4. **`"H"` of `"Holmes"` (proper noun last name)**: After the model parses the preceding token sequence `"sherlock "`, its recurrent state $\mathbf{h}_{t-1}$ retains the memory of the detective's first name. When an `'h'` arrives, the transition matrix $\mathbf{W}_{hh}$ leverages the learned bigram association $\mathbb{P}(\texttt{Holmes} \mid \texttt{Sherlock})$ to correctly capitalize the last name.

Empirical results across 1070 training epochs (Slide 36). Table 3.2 documents the verbatim loss trajectories and generated outputs across training checkpoints from Epoch 1 to Epoch 1070.

Table 3.2: Empirical comparison of character casing dynamics on the Sherlock Holmes corpus (Slide 36): One-to-One Recurrent Mapping vs. Sequence-to-Sequence Encoder-Decoder across training epochs 1 to 1070.

	One-to-One Mapping		Encoder-Decoder Architecture	
Epoch	**Loss**	**Generated Output**	**Loss**	**Generated Output**
0001	5.3212	`xxxxxxxxxxxxxxxxxxxxxxxxxxxxxx`	5.4591	`wwXXXXXXXXXXXXXXXXXXXXXXXXXX`
0002	3.7157	`xxxxxxxxxxxxxxxxxxxxxxxxxxxxxx`	4.0161	`aXXXXXXXXXXXXXXXXXXXXXXXXXXX`
0003	2.7964	`xxxxxxxxxxxxxxxxxxxxxxxxxxxxxx`	2.8268	`xxxxxxxxxxxxxxxxxxxxxxxxxxxxx`
0004	2.7981	`xxxxxxxxxxxxxxxxxxxxxxxxxxxxxx`	2.7671	`xxxxxxxxxxxxxxxxxx`
0030	0.2264	`"indeed i am sherlock holmes,"`	0.2315	`"indeed i am sherlock holmes,"`
0050	0.0802	`"indeed i am sherlock holmes,"`	0.0705	`"indeed i am sherlock holmes,"`
0160	0.0301	`"Indeed i am sherlock Holmes,"`	0.0259	`"Indeed i am sherlock Holmes,"`
0530	0.0224	`"Indeed i am sherlock Holmes,"`	0.0197	`"Indeed I am sherlock Holmes,"`
1070	0.0232	`"Indeed i am sherlock Holmes,"`	0.0156	`"Indeed I am Sherlock Holmes,"`

Analysis of qualitative phase transitions and convergence trade-offs. The comparative trajectories in Table 3.2 reveal three fundamental pedagogical principles:

1. **The Causal Barrier of One-to-One Recurrence**: The one-to-one model plateaus at Epoch 160 with loss 0.0301 and cannot improve further through Epoch 1070 (loss 0.0232). As predicted, it successfully capitalizes `"Indeed"` (from the quote mark) and `"Holmes"` (from the preceding `"sherlock"`), but remains permanently trapped producing lowercase `"i"` and `"sherlock"`. Because the lockstep recurrent cell must emit output $\hat{y}_t$ simultaneously with reading x_t, it suffers from irreversible causal lookahead blindness.

2. **Sequence-Level Information Integration in Seq2Seq**: The Encoder-Decoder model overcomes causal blindness by decoupling sequence ingestion from sequence emission. The encoder reads the entire string before generation begins. Consequently, the context vector $\mathbf{h}_N$ encodes global contextual features: it "remembers" that `"i"` is bounded by whitespace on both sides (making it the isolated pronoun `"I"`) and that `"s"` begins the full proper name `"sherlock"`. By Epoch 530, the decoder resolves `"Indeed I am sherlock Holmes,"` (loss 0.0197), and by Epoch 1070 it fully resolves all four capitalization targets: `"Indeed I am Sherlock Holmes,"` with superior loss 0.0156.

3. **The Sample Complexity Trade-off (100× Training Epochs)**: Why does the Encoder-Decoder require 1070 epochs when the simple character model converged in 10 epochs? In one-to-one mapping, temporal alignment is strictly hardcoded: character t at the input maps directly to character t at the output. In Seq2Seq, the model is given zero alignment hints; it must simultaneously discover: (i) how to compress the entire sentence into vector $\mathbf{h}_N$, (ii) how to decode variable-length autoregressive tokens, and (iii) how to align output positions with latent source semantics. Learning this unconstrained non-monotonic mapping from scratch drastically increases optimization difficulty and sample complexity.

3.7 Persistent bottlenecks of gated RNNs and the road to transformers

While LSTMs and Encoder-Decoder architectures dominated sequence modeling from 2014 to 2017, Slide 37 identifies three irreducible physical and computational bottlenecks that ultimately necessitated a revolutionary paradigm shift:

1. The Shannon channel capacity and vector bottleneck [30]. In the canonical Seq2Seq framework, an input sequence $(x_1, \ldots, x_{T_x})$ of arbitrary length and linguistic entropy is forced to compress into a single continuous vector of fixed dimensionality:

$$\mathbf{h}_{T_x} = \text{Encoder}(x_1, \ldots, x_{T_x}) \in \mathbb{R}^{d_h}. \tag{3.24}$$

By Shannon's source coding and rate-distortion theory, a finite-dimensional vector in floating-point precision has bounded information capacity. As sequence length increases beyond $T_x \geq 30$ tokens, the recurrent state suffers from catastrophic information dilution: earlier tokens (often containing essential syntactic subjects and conditioning context) are progressively overwritten by later inputs.

2. The sequential computation wall ($O(T)$ training latency). Modern deep learning hardware accelerators (e.g., NVIDIA Hopper/Blackwell GPUs, Google TPUs) derive their performance from massive SIMD (Single Instruction, Multiple Data) parallelism across thousands of tensor cores. Recurrent architectures impose an unbreakable temporal recurrence dependency:

$$\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t). \tag{3.25}$$

Computing hidden state $\mathbf{h}_t$ strictly requires the completion of state $\mathbf{h}_{t-1}$. Consequently, an RNN cannot parallelize training across the temporal dimension T. While convolutional and attention layers process the entire sequence in $O(1)$ parallel sequential steps via batched matrix multiplications, RNNs require $O(T)$ sequential operations, rendering pre-training on multi-trillion-token web-scale corpora computationally intractable.

3. Temporal path length ($O(T)$ vs. $O(1)$ direct routing). In an unrolled recurrent graph, the shortest path between token x_1 and prediction y_T scales linearly with sequence length: $O(T)$.

Every additional step introduces an intermediate Jacobian multiplication $\frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}}$. Even with the additive Constant Error Carousel of LSTMs, error signals become noisy and diffuse over horizons exceeding 100 to 200 tokens.

The two-stage historical leap to foundation models: These structural ceilings precipitated a two-stage intellectual revolution:

1. **Stage 1: Soft Alignment [31]**: Additive Attention shattered the information vector bottleneck by maintaining direct memory access to all encoder annotations $\{\mathbf{h}_1, \dots, \mathbf{h}_{T_x}\}$. However, because the underlying backbone remained recurrent, training was still bound to the sequential execution wall.

2. **Stage 2: Attention Is All You Need [1]**: The landmark study demonstrated that if attention provides direct $O(1)$ information flow between any pair of tokens regardless of temporal distance, **recurrence is completely redundant**. Discarding recurrent cells entirely in favor of multi-head self-attention unlocked full GPU sequence parallelization and dynamic contextualization, establishing the modern Transformer architecture.

3.7.1 Breaking the vector bottleneck: Bahdanau additive attention

In the standard Seq2Seq architecture (Section 3.6), the encoder must squeeze all grammatical, lexical, and structural information of a source sentence of arbitrary length ($T_x = 40, 80$) into the single final state $\mathbf{h}_{T_x} \in \mathbb{R}^{d_h}$. As sentence length increases, empirical translation performance drops precipitously: the fixed-size hidden vector acts as an extreme information bottleneck.

To shatter this bottleneck, **Additive (Soft) Attention** was introduced in [31], allowing the decoder to dynamically search and focus across all encoder hidden states at every generation step.

1. Mathematical formulation with explicit tensor dimensions. Let the source sequence $(x_1, \dots, x_{T_x})$ be encoded by a Bidirectional RNN, producing a sequence of source annotation vectors:

$$\mathbf{h}_j = \begin{bmatrix} \overrightarrow{\mathbf{h}}_j \\ \overleftarrow{\mathbf{h}}_j \end{bmatrix} \in \mathbb{R}^{2d_h}, \quad \forall j \in \{1, 2, \dots, T_x\}. \tag{3.26}$$

At decoder generation step i, let $\mathbf{s}_{i-1} \in \mathbb{R}^{d_{\text{dec}}}$ denote the previous decoder hidden state.

1. **Alignment Energy Score**: The relevance of source token j to target emission i is evaluated by a small feed-forward network parameterized by $\mathbf{W}_a \in \mathbb{R}^{d_a \times d_{\text{dec}}}$, $\mathbf{U}_a \in \mathbb{R}^{d_a \times 2d_h}$, and $\mathbf{v}_a \in \mathbb{R}^{d_a}$:
$$e_{ij} = \mathbf{v}_a^\top \tanh\left(\mathbf{W}_a \mathbf{s}_{i-1} + \mathbf{U}_a \mathbf{h}_j\right) \in \mathbb{R}. \tag{3.27}$$

2. **Softmax Attention Weights**: Exponentiating and normalizing across all T_x source positions yields a valid categorical distribution over input tokens:
$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^{T_x} \exp(e_{ik})}, \quad \text{where } \alpha_{ij} \in (0, 1) \text{ and } \sum_{j=1}^{T_x} \alpha_{ij} = 1. \tag{3.28}$$

   The scalar α_{ij} represents the degree to which the decoder should attend to source word x_j when generating target word y_i.

3. **Dynamic Context Vector**: The context vector $\mathbf{c}_i$ is computed as the expected annotation under distribution $\boldsymbol{\alpha}_i$:

$$\mathbf{c}_i = \sum_{j=1}^{T_x} \alpha_{ij} \mathbf{h}_j \in \mathbb{R}^{2d_h}. \tag{3.29}$$

Unlike standard Seq2Seq where context $\mathbf{h}_{T_x}$ is static, $\mathbf{c}_i$ is dynamically recomputed at every decoder step i, steering the model's focus to the relevant source phrase.

4. **Decoder State and Emission Update**: The decoder hidden state updates by conditioning on the previous token y_{i-1} and dynamic context $\mathbf{c}_i$:

$$\mathbf{s}_i = \text{RNN}_{\text{dec}}\left(\mathbf{s}_{i-1},\ [\mathbf{y}_{i-1}, \mathbf{c}_i]\right), \tag{3.30}$$

and target token probabilities are emitted via $\hat{\mathbf{y}}_i = \text{softmax}(\mathbf{W}_s[\mathbf{s}_i, \mathbf{y}_{i-1}, \mathbf{c}_i] + \mathbf{b}_s)$.

Figure 3.5 illustrates the Bahdanau Additive Attention architecture.

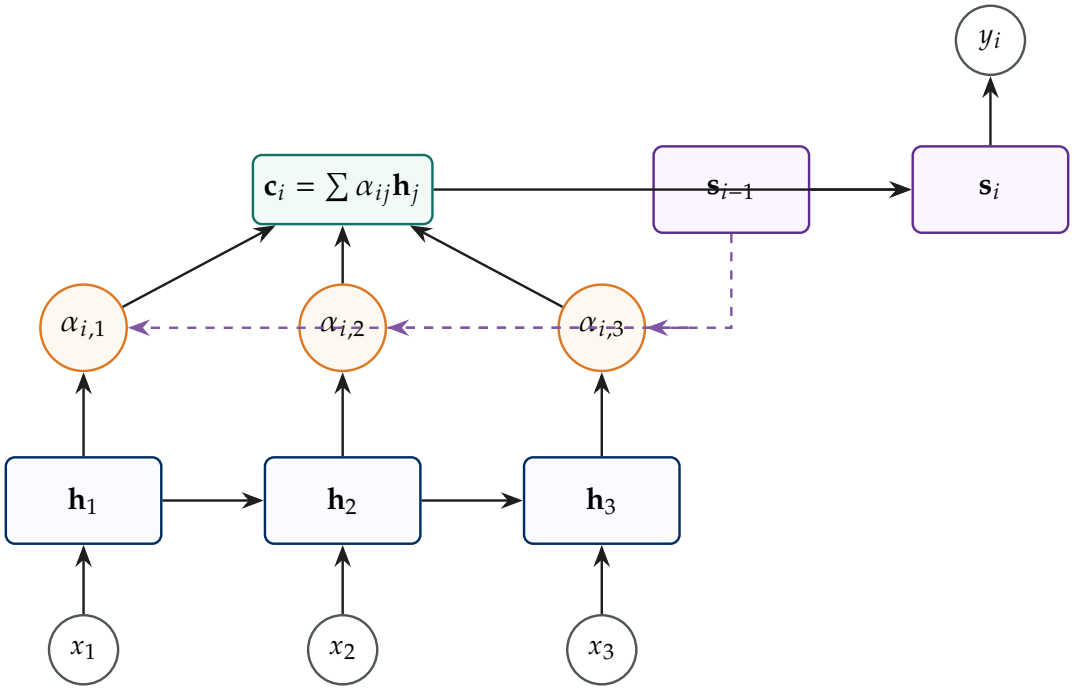


Figure 3.5: The Bahdanau Additive Attention mechanism. At decoder step i, previous decoder state $\mathbf{s}_{i-1}$ is compared against all encoder annotations $\mathbf{h}_j$ via Equation (3.27), producing attention distribution α_{ij}. The weighted sum synthesizes dynamic context vector $\mathbf{c}_i \in \mathbb{R}^{2d_h}$, directly feeding relevant source features into decoder state $\mathbf{s}_i$.

External deep dive: Historical milestone: from Schmidhuber's CEC to Sutskever's Seq2Seq

The lineage of sequence modeling represents one of the most intellectually cohesive arcs in artificial intelligence:

- In 1997, the milestone paper *"Long Short-Term Memory"* [28] solved the vanishing gradient problem via the Constant Error Carousel, proving that additive highways preserve error signals indefinitely.
- In 2014, the Gated Recurrent Unit (GRU) and RNN encoder-decoder architectures

were formulated in [29] for statistical machine translation.

- Simultaneously in 2014, the landmark study *"Sequence to Sequence Learning with Neural Networks"* [30] demonstrated that multi-layer deep LSTMs could map sentences across languages end-to-end. Crucially, it was discovered that reversing the source sentence (*"A B C → C B A"*) dramatically improved performance by shortening the initial temporal gradient distance between source and target tokens.

3.7.2 Causal bridge to Chapter 4: the representation problem and continuous word embeddings

Throughout this chapter, we have traced the linear evolutionary arc of sequential neural architectures:

1. **Vanilla RNNs** introduced temporal recurrence and weight sharing to eliminate the fixed-dimensional constraints of feedforward networks, processing elastic sequences via an evolving hidden state $\mathbf{h}_t \in \mathbb{R}^{d_h}$.
2. **LSTMs and GRUs** resolved exponential vanishing gradients along the temporal chain via the Constant Error Carousel ($\frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_{t-1}} \approx \mathbf{I}$) and multiplicative gating highways, enabling gradient propagation across extended sequences.
3. **Seq2Seq Encoder-Decoder Pipelines** overcame the causal lookahead blindness of one-to-one recurrent cells by mapping variable-length input sequences into dynamic variable-length output sequences.
4. **Bahdanau Additive Attention** shattered the single-vector bottleneck ($\mathbf{h}_N \in \mathbb{R}^{d_h}$) by granting the decoder dynamic soft addressing across all encoder annotations ($\mathbf{c}_i = \sum_j \alpha_{ij}\mathbf{h}_j$).

The unresolved input bottleneck: what is $\mathbf{x}_t$? Despite these mathematical triumphs in temporal modeling, every recurrent transition $\mathbf{h}_t = \tanh(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t + \mathbf{b}_h)$ rests upon a fundamental, unaddressed prerequisite: *the input $\mathbf{x}_t$ must be a numeric vector in $\mathbb{R}^{d_{in}}$*. In our pedagogical character casing example, we used a toy ASCII one-hot vector $\mathbf{x}_t \in \{0,1\}^{256}$. However, scaling sequence models to real-world natural language or software engineering source code reveals three catastrophic failure modes of local one-hot representations:

1. **Dimensionality Explosion**: Realistic vocabularies contain $V = |\mathcal{V}| \geq 50{,}000$ to $1{,}000{,}000$ unique words. Representing tokens as one-hot vectors $\mathbf{e}_i \in \{0,1\}^V$ forces the recurrent input projection matrix $\mathbf{W}_{xh} \in \mathbb{R}^{d_h \times V}$ to balloon into millions of parameters, causing severe memory bloat and extreme sample inefficiency.
2. **Orthogonal Metric Collapse**: Standard basis vectors are strictly mutually orthogonal ($\mathbf{e}_i^\top \mathbf{e}_j = 0$ for all $i \neq j$) and uniformly equidistant ($d_E(\mathbf{e}_i, \mathbf{e}_j) = \sqrt{2}$). The recurrent cell receives zero geometric signal that *"dog"* and *"puppy"*, or keywords `"while"` and `"for"`, share syntactic or semantic roles.
3. **Zero Gradient Parameter Sharing**: During backpropagation through time (BPTT), updating weights for one-hot vector $\mathbf{e}_i$ provides absolutely zero learning transfer to semantically similar words $\mathbf{e}_j$, requiring astronomical training data to observe every word in every temporal context.

The path forward. While recurrent networks solved the *temporal dynamics* problem across sequences, they are crippled without a dense, low-dimensional, continuous metric space $\phi : \mathcal{V} \to \mathbb{R}^d$ ($d \ll V$) where geometric proximity directly encodes syntactic and semantic similarity.

This fundamental representational challenge leads directly to Chapter 4: *Distributed Word Representations and Word2Vec*.

Chapter 4

Distributed word representations and Word2Vec

4.1 The representation problem: why continuous embeddings are mandatory

Natural language and software engineering source code are fundamentally composed of discrete, symbolic tokens. In written English or in source files, programs and sentences present themselves as strings of alphanumeric characters: words such as `"compiler"`, `"optimizer"`, `"gradient"`, and keywords such as `"class"`, `"def"`, `"return"`.

Conversely, modern artificial neural networks are continuous, differentiable mathematical operators. An L-layer deep neural network $\mathcal{F}(\mathbf{x};\Theta)$ computes outputs through compositions of parameterized linear projections and non-linear activation functions:

$$\mathcal{F}(\mathbf{x};\Theta) = \sigma_L\Big(\mathbf{W}_L\,\sigma_{L-1}\big(\cdots\sigma_1(\mathbf{W}_1\mathbf{x}+\mathbf{b}_1)\cdots\big)+\mathbf{b}_L\Big), \tag{4.1}$$

where $\mathbf{W}_l \in \mathbb{R}^{d_l\times d_{l-1}}$ and $\mathbf{b}_l \in \mathbb{R}^{d_l}$. Such architectures rely fundamentally on the continuous mathematics of vector spaces $\mathbb{R}^d$, multivariable calculus, and gradient-based backpropagation. The optimization algorithm minimizes an empirical risk loss $\mathcal{L}(\Theta)$ by iteratively updating weight tensors along the negative gradient direction:

$$\Theta \leftarrow \Theta - \eta\,\nabla_\Theta\mathcal{L}(\Theta). \tag{4.2}$$

This mathematical reality creates an immediate, fundamental impasse: **discrete text strings cannot be differentiated**. The partial derivative of a scalar loss with respect to a raw string literal, $\frac{\partial\mathcal{L}}{\partial\texttt{"cat"}}$, is mathematically undefined. Discrete symbols possess no notion of infinitesimal perturbation, tangent spaces, or linear vector combinations. One cannot add `"cat"` to `"dog"`, nor compute an inner product between `"while"` and `"for"`.

To bridge this chasm between discrete symbolic communication and continuous gradient-based computation, we must establish a formal mapping from the discrete vocabulary into a continuous vector space.

Definition 4.1: *Continuous word embedding*

Let $\mathcal{V} = \{w_1, w_2, \ldots, w_V\}$ denote a finite vocabulary of $V = |\mathcal{V}|$ distinct discrete symbolic tokens. A **continuous word embedding** is an injective mapping:

$$\phi : \mathcal{V} \longrightarrow \mathbb{R}^d, \quad \text{with } d \ll V, \tag{4.3}$$

that maps each discrete symbol $w_i \in \mathcal{V}$ to a low-dimensional, dense continuous vector $\mathbf{v}_i = \phi(w_i) \in \mathbb{R}^d$. The target vector space $(\mathbb{R}^d, \|\cdot\|, \langle\cdot,\cdot\rangle)$ is a continuous metric space equipped with an inner product, where geometric distance $D(\mathbf{v}_i, \mathbf{v}_j)$ and angular orientation $\cos(\mathbf{v}_i, \mathbf{v}_j)$ preserve semantic and syntactic relationships.

In typical natural language and software engineering applications, vocabulary sizes range from $V \approx 30{,}000$ to upwards of $V = 1{,}000{,}000$ unique tokens, while the latent embedding dimension is chosen to be compact: $d \in [100, 1024]$ (e.g., $d = 300$ in classical Word2Vec and FastText; $d = 768$ in standard Transformer baselines). By mapping discrete words into $\mathbb{R}^d$, language tokens become continuous variables that participate directly in matrix algebra, linear transformations, and gradient backpropagation.

To understand why dense continuous representations are indispensable, we must first examine the standard categorical baseline: local one-hot encoding. While intuitive and trivial to construct, one-hot vectors suffer from severe geometric and computational pathologies that fundamentally incapacitate neural learning.

4.2 The baseline approach: local one-hot encodings and why they fail

The most straightforward, historical baseline for converting discrete categorical symbols into numeric vectors is **one-hot encoding**. Under this local representational scheme, every word $w_i \in \mathcal{V}$ is mapped to a standard basis coordinate vector $\mathbf{e}_i$ in a V-dimensional Euclidean space $\mathbb{R}^V$:

$$\mathbf{e}_i = [\underbrace{0, \dots, 0}_{i-1 \text{ zeros}}, 1, \underbrace{0, \dots, 0}_{V-i \text{ zeros}}]^\top \in \{0,1\}^V, \quad \text{where } (\mathbf{e}_i)_k = \delta_{ik} = \begin{cases} 1 & \text{if } k = i, \\ 0 & \text{if } k \neq i. \end{cases} \tag{4.4}$$

For example, consider a toy vocabulary of $V = 5$ tokens from the course lecture slides:

$$\mathcal{V} = \{\texttt{"dog"}, \texttt{"cat"}, \texttt{"fish"}, \texttt{"pen"}, \texttt{"pencil"}\}. \tag{4.5}$$

Under one-hot encoding, each word is assigned an isolated Cartesian coordinate axis:

$$\mathbf{e}_{\text{dog}} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \quad \mathbf{e}_{\text{cat}} = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \quad \mathbf{e}_{\text{fish}} = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}, \quad \mathbf{e}_{\text{pen}} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}, \quad \mathbf{e}_{\text{pencil}} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}. \tag{4.6}$$

4.2.1 Algebraic equivalence: one-hot matrix multiplication as embedding lookup

In the foundational guide *Build a Large Language Model from Scratch* [32], it is proven that feeding a one-hot vector into a dense linear layer is algebraically identical to retrieving a specific row or column from a weight lookup table.

Theorem 4.1: *Embedding lookup equivalence*

Let $\mathcal{V}$ be a vocabulary of size V, and let token $w_i \in \mathcal{V}$ be encoded as the standard basis vector $\mathbf{e}_i \in \{0,1\}^V$.

Case 1: Column-Vector Orientation (Lecture Slide Convention). Let $\mathbf{W}_{\text{in}} \in \mathbb{R}^{d \times V}$ be the embedding matrix whose k-th column corresponds to the d-dimensional embedding vector $\mathbf{w}_k = (\mathbf{W}_{\text{in}})_{:,k} \in \mathbb{R}^d$:

$$\mathbf{W}_{\text{in}} = \begin{bmatrix} | & | & & | \\ \mathbf{w}_1 & \mathbf{w}_2 & \cdots & \mathbf{w}_V \\ | & | & & | \end{bmatrix}. \tag{4.7}$$

Multiplying $\mathbf{W}_{\text{in}}$ by the one-hot column vector $\mathbf{e}_i$ extracts the i-th column of $\mathbf{W}_{\text{in}}$:

$$\mathbf{h} = \mathbf{W}_{\text{in}}\mathbf{e}_i = \sum_{k=1}^{V}(\mathbf{e}_i)_k(\mathbf{W}_{\text{in}})_{:,k} = 1 \cdot (\mathbf{W}_{\text{in}})_{:,i} + \sum_{k\neq i} 0 \cdot (\mathbf{W}_{\text{in}})_{:,k} = \mathbf{w}_i \in \mathbb{R}^d. \tag{4.8}$$

Case 2: Row-Vector Orientation (PyTorch and Raschka Convention). Let $\mathbf{W} \in \mathbb{R}^{V\times d}$ be an embedding table whose k-th row contains the embedding vector $\mathbf{w}_k^\top = \mathbf{W}_{k,:} \in \mathbb{R}^{1\times d}$. Multiplying the one-hot row vector $\mathbf{e}_i^\top$ by $\mathbf{W}$ extracts the i-th row of $\mathbf{W}$:

$$\mathbf{e}_i^\top\mathbf{W} = \sum_{k=1}^{V}(\mathbf{e}_i)_k\mathbf{W}_{k,:} = 1 \cdot \mathbf{W}_{i,:} + \sum_{k\neq i} 0 \cdot \mathbf{W}_{k,:} = \mathbf{W}_{i,:} = \mathbf{w}_i^\top \in \mathbb{R}^{1\times d}. \tag{4.9}$$

Hence, matrix multiplication of an embedding weight matrix by a one-hot indicator vector is identical to an indexed memory table lookup:

$$\mathbf{W}_{\text{in}}\mathbf{e}_i \equiv \texttt{lookup}(\mathbf{W}_{\text{in}}, i). \tag{4.10}$$

4.2.2 Systems-level justification: why frameworks use dedicated embedding lookups

Although dense matrix multiplication $\mathbf{e}_i^\top\mathbf{W}$ and table indexing $\texttt{lookup}(\mathbf{W}, i)$ are algebraically equivalent, no modern deep learning framework (PyTorch, TensorFlow, JAX) ever implements word embeddings as a dense matrix multiplication. Deep learning frameworks provide specialized lookup modules, such as `torch.nn.Embedding(num_embeddings=V, embedding_dim=d)`, for three decisive systems engineering reasons:

1. **Elimination of $O(V \cdot d)$ Wasted FLOPs**: Evaluating the matrix multiplication $\mathbf{e}_i^\top\mathbf{W}$ requires executing $V \cdot d$ floating-point multiplications and $(V-1) \cdot d$ additions. In a standard language model with vocabulary $V = 50{,}257$ (GPT-2 tokenizer) and hidden dimension $d = 768$, computing the input projection for a single token via dense matrix multiplication requires:

   $$50{,}257 \times 768 \approx 3.86 \times 10^7 \text{ arithmetic operations per token.} \tag{4.11}$$

   Of these 38.6 million operations, exactly 99.998% evaluate $0 \times W_{k,j} = 0$. By contrast, `torch.nn.Embedding` accepts integer token IDs $i \in \{0, \ldots, V-1\}$ and performs direct $O(1)$ pointer arithmetic into physical GPU/CPU memory:

   $$\text{address}(\mathbf{w}_i) = \text{base_pointer} + i \times d \times \texttt{sizeof}(\text{float32}), \tag{4.12}$$

   retrieving the d-dimensional embedding slice in $O(1)$ time with exactly zero arithmetic FLOPs.

2. **Massive VRAM Savings (25,000× Memory Reduction)**: Consider a standard training batch with batch size $B = 64$ and sequence context length $T = 1{,}024$. If input sequences were materialized as dense one-hot tensors, the input tensor $\mathbf{X}_{\text{onehot}}$ would have shape $(B, T, V) = (64, 1{,}024, 50{,}257)$. Storing this tensor in single-precision floating point (`float32`, 4 bytes per element) would require:

   $$\text{VRAM}_{\text{one-hot}} = 64 \times 1{,}024 \times 50{,}257 \times 4 \text{ bytes} \approx 13{,}175{,}095{,}296 \text{ bytes} \approx 13.18\,\text{GB of VRAM,} \tag{4.13}$$

   simply to hold the input batch before computing a single activation. Conversely, storing sequences as integer token IDs produces an `int64` tensor of shape $(B, T) = (64, 1{,}024)$,

requiring:

$$\text{VRAM}_{\text{indices}} = 64 \times 1{,}024 \times 8 \text{ bytes} = 524{,}288 \text{ bytes} \approx 524\,\text{KB of VRAM}. \tag{4.14}$$

This constitutes a memory bandwidth reduction factor of $\frac{13.18\,\text{GB}}{524\,\text{KB}} \approx \mathbf{25{,}128}\times$, entirely preventing Out-Of-Memory (OOM) GPU crashes.

3. **Sparse Gradient Backpropagation**: Under dense matrix multiplication $\mathbf{y} = \mathbf{e}_i^\top \mathbf{W}$, the backpropagation adjoint with respect to the weight matrix is the outer product:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \mathbf{e}_i \left(\frac{\partial \mathcal{L}}{\partial \mathbf{y}} \right) \in \mathbb{R}^{V \times d}. \tag{4.15}$$

This gradient matrix is massive ($V \times d$) and strictly sparse: only row i contains non-zero values, while the remaining $V - 1$ rows are populated entirely by zeros. Synchronizing such dense gradient tensors across multi-GPU distributed clusters saturates PCIe and NVLink bandwidth. `torch.nn.Embedding` executes sparse indexed gradient accumulation (`index_add_`), routing the upstream vector $\frac{\partial \mathcal{L}}{\partial \mathbf{y}} \in \mathbb{R}^d$ directly into the i-th row of $\mathbf{W}$.

4.2.3 The 5 structural failure modes of one-hot representations

While the lookup table implementation rescues computational throughput, local one-hot representations fail catastrophically as semantic representations. They suffer from five fatal pathologies:

1. Extreme Sparsity and Curse of Dimensionality. As vocabulary size scales to web scale ($V \geq 50{,}000$ to 10^6), one-hot vectors become astronomical sparse vectors where 99.998% of entries are zero. The dimensionality of the vector space equals the number of distinct words. Under this curse of dimensionality, the volume of the enclosing space grows exponentially, leaving data points isolated at the extreme vertices of independent axes.

2. Metric Collapse and Semantic Orthogonality ($\cos = 0, d_E = \sqrt{2}$). Because one-hot vectors form the standard Cartesian basis of $\mathbb{R}^V$, every pair of distinct words $w_i \neq w_j$ is mutually perpendicular and equidistant:

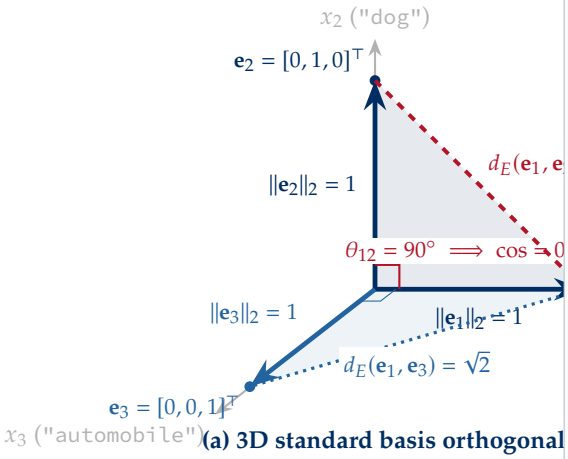
$$\cos(\mathbf{e}_i, \mathbf{e}_j) = \frac{\mathbf{e}_i^\top \mathbf{e}_j}{\|\mathbf{e}_i\|_2 \|\mathbf{e}_j\|_2} = \frac{0}{1 \cdot 1} = 0, \quad \forall i \neq j, \tag{4.16}$$

$$d_E(\mathbf{e}_i, \mathbf{e}_j) = \|\mathbf{e}_i - \mathbf{e}_j\|_2 = \sqrt{(1)^2 + (-1)^2 + \sum_{k \neq i,j} 0^2} = \sqrt{2} \approx 1.414, \quad \forall i \neq j. \tag{4.17}$$

Under one-hot encoding, every word is orthogonal and equidistant to every other word:

$$d_E(\texttt{"pen"}, \texttt{"pencil"}) = d_E(\texttt{"pen"}, \texttt{"crocodile"}) = \sqrt{2}. \tag{4.18}$$

The underlying metric space collapses into a degenerate topology where open balls $B_r(\mathbf{e}_i) = \{\mathbf{x} \in \mathcal{V} : d_E(\mathbf{e}_i, \mathbf{x}) < r\}$ contain only $\{\mathbf{e}_i\}$ for $r \leq \sqrt{2}$, and immediately engulf the entire universe of vocabulary tokens for $r > \sqrt{2}$. There is zero geometric capacity to reflect semantic proximity, synonymy, or grammatical category.



(a) 3D standard basis orthogonal

Pairwise metric equality matrices:

• **Cosine similarity matrix** (Gramian $\mathbf{E}^\top\mathbf{E} = \mathbf{I}_V$):

$$\mathbf{S}_{\cos} = \begin{pmatrix} & \text{cat} & \text{dog} & \text{car} \\ \text{cat} & 1 & \mathbf{0} & \mathbf{0} \\ \text{dog} & \mathbf{0} & 1 & \mathbf{0} \\ \text{car} & \mathbf{0} & \mathbf{0} & 1 \end{pmatrix}$$

• **Euclidean distance matrix** ($\mathbf{D}_{ij} = \sqrt{2}(1 - \delta_{ij})$):

$$\mathbf{D}_{L_2} = \begin{pmatrix} & \text{cat} & \text{dog} & \text{car} \\ \text{cat} & 0 & \mathbf{\sqrt{2}} & \mathbf{\sqrt{2}} \\ \text{dog} & \mathbf{\sqrt{2}} & 0 & \mathbf{\sqrt{2}} \\ \text{car} & \mathbf{\sqrt{2}} & \mathbf{\sqrt{2}} & 0 \end{pmatrix}$$

The geometric pathology:
dist("cat", "dog") = dist("cat", "car") = $\sqrt{2}$
cos("cat", "dog") = cos("cat", "car") = 0

(b) Semantic metric collapse

All tokens are equidistant and mutually perpendicular, completely destroying semantic metric topology.

Figure 4.1: Visual proof of pairwise orthogonality and equidistance in one-hot vector representations. (a) In V-dimensional Euclidean space, each word is an orthogonal standard basis vector of unit norm ($\|\mathbf{e}_i\|_2 = 1$). By the Pythagorean theorem, every distinct pair forms a right isosceles triangle with origin $\mathbf{0}$, yielding an identical Euclidean distance of $\sqrt{1^2 + 1^2} = \sqrt{2}$ and an inner product of 0 ($\theta = 90°$). (b) The resulting Gramian matrix is the identity $\mathbf{I}_V$, treating related concepts like `"cat"` and `"dog"` with the exact same geometric distance as totally unrelated concepts like `"cat"` and `"automobile"`.

3. Zero Gradient Parameter Sharing During Backpropagation. Because one-hot vectors activate strictly disjoint coordinates, training a neural network on an occurrence of token w_i produces gradients exclusively for the parameters associated with index i:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{k,:}} = \mathbf{0}, \quad \forall k \neq i. \tag{4.19}$$

If a model updates its weights upon observing the sentence *"The feline climbed the tree"*, the parameters associated with `"feline"` receive gradient updates, but the parameters for `"cat"`, `"kitten"`, or `"leopard"` remain completely unchanged. The model cannot transfer statistical strength across semantically synonymous or related words; it must encounter every word hundreds of times independently in every grammatical context.

4. Simplex Confinement and Absence of Latent Manifolds. One-hot vectors are confined strictly to the V extreme vertices of the standard $(V-1)$-dimensional probability simplex:

$$\Delta^{V-1} = \left\{ \mathbf{x} \in \mathbb{R}^V : \sum_{i=1}^{V} x_i = 1,\ x_i \geq 0 \right\}. \tag{4.20}$$

The interior and edges of the simplex correspond to fractional probabilistic mixtures, but distinct words themselves inhabit isolated, non-overlapping boundary corners. Words cannot reside on smooth, lower-dimensional semantic submanifolds, nor can they support linear translation arithmetic ($\mathbf{w}_{\text{king}} - \mathbf{w}_{\text{man}} + \mathbf{w}_{\text{woman}} \approx \mathbf{w}_{\text{queen}}$).

5. Software Engineering Identifier Polymorphism. In software engineering source code, developers routinely express identical semantic intents through differing lexical notations, identifier casings, and API conventions:

- **Casing variants**: `getUserById` vs. `get_user_by_id` vs. `GetUserById`.
- **API synonyms**: `push()` vs. `append()` vs. `add()` vs. `enqueue()`.
- **Syntactic equivalents**: Type references such as `ArrayList` and `LinkedList` vs. unrelated punctuation symbols like `";"`.

Under one-hot encoding, each casing variation and API synonym is assigned a totally independent, mutually orthogonal basis vector. To a one-hot model:

$$\cos(\mathbf{e}_{\texttt{get_user_by_id}}, \mathbf{e}_{\texttt{getUserById}}) = 0, \quad d_E(\mathbf{e}_{\texttt{ArrayList}}, \mathbf{e}_{\texttt{LinkedList}}) = d_E(\mathbf{e}_{\texttt{ArrayList}}, \mathbf{e}_{\texttt{";"}}) = \sqrt{2}. \tag{4.21}$$

This lexical fragmentation completely prevents code-intelligence models from transferring refactoring patterns, bug-fix templates, or type semantics across codebases.

Because local one-hot encodings confine words to isolated, orthogonal coordinate axes where semantic similarity is geometrically undefinable, we require a fundamentally different representational paradigm. This paradigm shift rests on distributing linguistic meaning across continuous latent dimensions—guided by the foundational Distributional Hypothesis.

4.3 Distributed representations and the distributional hypothesis

To overcome the orthogonality and isolation of local representations, modern deep learning adopts **distributed representations**, a concept formalized in connectionist neural networks in 1986 [21].

Definition 4.2: *Local vs. distributed representations*

- **Local Representation**: Each concept or symbol is represented by a single dedicated computational unit (e.g., one coordinate axis in $\mathbb{R}^V$). If that unit is inactive, the concept is entirely absent. There is zero overlap between distinct symbols.
- **Distributed Representation**: Each concept is represented by a continuous pattern of activation across all d dimensions of a shared vector space $\mathbb{R}^d$ ($d \ll V$). Conversely, each individual dimension participates in representing many distinct concepts.

In a distributed representation, meaning is not stored in an isolated slot; rather, it emerges from the continuous coordinates along latent dimensions. For example, latent dimensions might implicitly reflect attributes such as animacy, size, sentiment, syntactic part of speech, or software typing constructs.

4.3.1 The distributional hypothesis: context as meaning

How can an optimization algorithm discover meaningful continuous coordinates $\mathbf{v}_w \in \mathbb{R}^d$ for every word purely from raw text, without human supervision? The answer lies in the **Distributional Hypothesis**, formulated in structural linguistics in 1954 [33] and popularized in 1957 [34]:

> *"You shall know a word by the company it keeps."*
> — Distributional Hypothesis [34]

The distributional hypothesis posits that words that appear in similar linguistic contexts tend to share similar semantic and pragmatic meanings. Consider two sentences:

Sentence 1: "I used a **pencil** to write the essay."
Sentence 2: "You used my **pen** to write a letter."

Even if a machine has never encountered a dictionary definition of `"pencil"` or `"pen"`, it observes that both tokens co-occur with identical context words: `"used"`, `"write"`, `"essay"`, `"letter"`. Conversely, an unrelated word such as `"crocodile"` or `"refrigerator"` almost never occurs in the context of `"used a [...] to write"`.

Therefore, statistical co-occurrence within local contextual windows serves as an empirical, computable proxy for semantic similarity. If a learning algorithm is tasked with predicting surrounding context words from a target word (or vice versa), the learned vector coordinates must position words with similar co-occurrence profiles close together in vector space.

While the Distributional Hypothesis asserts that context defines meaning, it does not specify an optimization algorithm. To translate linguistic co-occurrence into optimized vector coordinates, we must frame a self-supervised prediction task and establish an end-to-end differentiable training loop.

4.4 The embedding training loop and semantic geometry evolution

To convert the distributional hypothesis into concrete numerical representations, we formulate a **self-supervised prediction cloze task** from unannotated text corpora.

4.4.1 The 7-step self-supervised optimization cycle

Given a raw, unannotated corpus of T running words $C = (w_1, w_2, \ldots, w_T)$, we slide a context window of half-width C across the text. At each step t, the model solves a fill-in-the-blank prediction task: estimating the probability distribution over the vocabulary for a missing token given its surrounding context:

$$\mathbb{P}(w_t = w \mid w_{t-C}, \ldots, w_{t-1}, w_{t+1}, \ldots, w_{t+C}). \tag{4.22}$$

As presented in the course lecture slides, this prediction task is executed through an iterative **7-step optimization cycle**:

1. **Initialize Parameter Matrices**: Allocate two weight matrices with small random values: the input embedding matrix $\mathbf{W}_{\text{in}} \in \mathbb{R}^{d \times V}$ and the output embedding matrix $\mathbf{W}_{\text{out}} \in \mathbb{R}^{d \times V}$.

2. **Context Lookup and Aggregation**: For the active context window, retrieve the input embedding vectors $\mathbf{v}_c = (\mathbf{W}_{\text{in}})_{:,c}$ and aggregate them into a single hidden context vector $\mathbf{h} \in \mathbb{R}^d$ (e.g., via vector summation or averaging).

3. **Output Candidate Projection**: Compute the dot product between the hidden context state $\mathbf{h}$ and every candidate output word vector $\mathbf{u}_w = (\mathbf{W}_{\text{out}})_{:,w}$ in the vocabulary:

$$z_w = \mathbf{u}_w^\top \mathbf{h}, \quad \text{yielding logit vector } \mathbf{z} = \mathbf{W}_{\text{out}}^\top \mathbf{h} \in \mathbb{R}^V. \tag{4.23}$$

4. **Probability Normalization**: Pass the similarity logits through the softmax activation function to produce a valid probability distribution $\hat{\mathbf{y}} \in \Delta^{V-1}$:

$$\hat{y}_w = \mathbb{P}(w \mid \text{context}) = \frac{\exp(z_w)}{\sum_{k=1}^{V} \exp(z_k)} = \frac{\exp(\mathbf{u}_w^\top \mathbf{h})}{\sum_{k=1}^{V} \exp(\mathbf{u}_k^\top \mathbf{h})}. \tag{4.24}$$

5. **Loss Evaluation**: Evaluate the prediction error against the true ground-truth target token $w^* = w_t$ using Categorical Cross-Entropy:

$$\mathcal{L}_{\text{CE}} = -\log \hat{y}_{w^*} = -\log \left(\frac{\exp(\mathbf{u}_{w^*}^\top \mathbf{h})}{\sum_{k=1}^{V} \exp(\mathbf{u}_k^\top \mathbf{h})} \right). \tag{4.25}$$

6. **Backpropagation Error Adjoints**: Apply reverse-mode automatic differentiation to compute error adjoints. The gradient with respect to logit vector $\mathbf{z}$ is the prediction residual $\boldsymbol{\delta} = \hat{\mathbf{y}} - \mathbf{y} \in \mathbb{R}^V$, where $\mathbf{y} = \mathbf{e}_{w^*}$. Propagating this residual backward yields parameter gradients:

$$\nabla_{\mathbf{W}_{\text{out}}} \mathcal{L} = \mathbf{h}\, \boldsymbol{\delta}^\top \in \mathbb{R}^{d \times V}, \quad \nabla_{\mathbf{h}} \mathcal{L} = \mathbf{W}_{\text{out}} \boldsymbol{\delta} \in \mathbb{R}^d, \quad \nabla_{\mathbf{W}_{\text{in}}} \mathcal{L} = (\nabla_{\mathbf{h}} \mathcal{L})\, \mathbf{e}_{\text{context}}^\top \in \mathbb{R}^{d \times V}. \tag{4.26}$$

7. **Stochastic Gradient Descent Parameter Update**: Adjust the coordinates in both embedding matrices by stepping along the negative gradient direction:

$$\mathbf{W}_{\text{out}} \leftarrow \mathbf{W}_{\text{out}} - \eta \nabla_{\mathbf{W}_{\text{out}}} \mathcal{L}, \qquad \mathbf{W}_{\text{in}} \leftarrow \mathbf{W}_{\text{in}} - \eta \nabla_{\mathbf{W}_{\text{in}}} \mathcal{L}. \tag{4.27}$$

Figure 4.2 illustrates this complete learning mechanism, juxtaposing the forward/backward computational dataflow in Panel (a) with the resulting geometric evolution of the embedding space across training epochs in Panel (b).

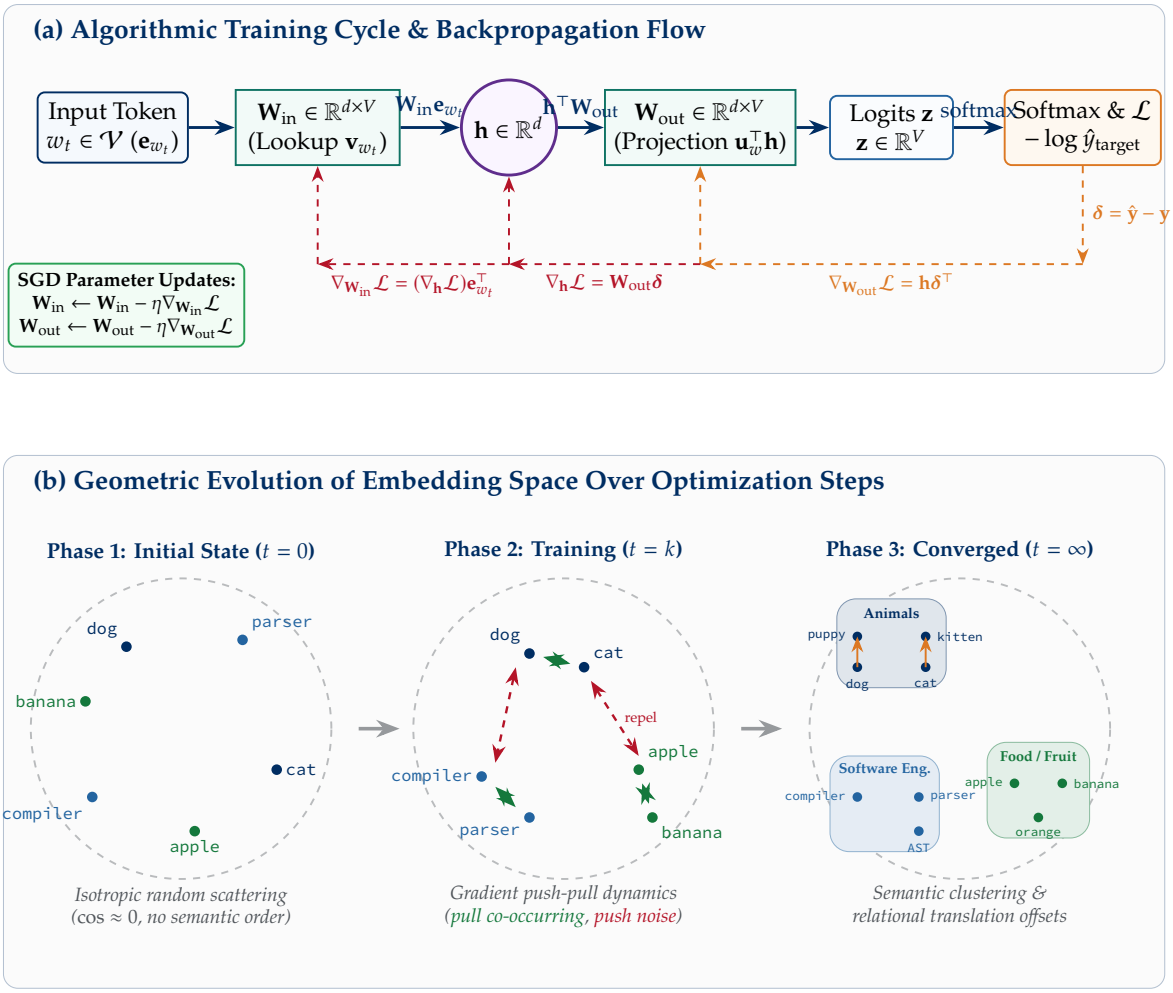


Figure 4.2: The embedding training loop and semantic space evolution. (a) Computational dataflow of the forward prediction pass and backward adjoint parameter updates. (b) Three-phase geometric evolution of embedding space from isotropic random scattering to clustered semantic topology with parallel relational offsets.

4.4.2 Geometric phase transition of the embedding space

As shown in Panel (b) of Figure 4.2, the continuous embedding space undergoes a profound topological self-organization over the course of training:

- **Phase 1: Initial State ($t = 0$).** Vectors are initialized using small Gaussian noise, $\mathbf{w} \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I}_d)$ with $\sigma \approx 0.01$ or $\sigma \approx 1/\sqrt{d}$. In high dimensions ($d \geq 100$), randomly sampled vectors are almost exactly orthogonal ($\cos \approx 0$). Tokens from disparate semantic domains—`"dog"`, `"compiler"`, `"apple"`—are scattered isotropically across the unit sphere with zero structural order.
- **Phase 2: Training Dynamics ($t = k$).** Whenever the model processes an observed sentence (e.g., *"The compiler parsed the syntax tree"*), backpropagation produces an attractive gradient force pulling the vectors of co-occurring words closer together. Concurrently, negative gradients or normalizer updates exert repulsive forces, pushing non-co-occurring words apart.
- **Phase 3: Converged Semantic Topology ($t = \infty$).** Upon convergence across millions of sentences, words coalesce into dense semantic neighborhoods (e.g., domestic animals, compiler construction terms, fruits). Remarkably, linear vector differences between related concepts become parallel across distinct pairs, giving rise to relational vector arithmetic (e.g., $\mathbf{v}_{\text{dog}} - \mathbf{v}_{\text{puppy}} \approx \mathbf{v}_{\text{cat}} - \mathbf{v}_{\text{kitten}}$).

4.4.3 Initialization dynamics and the hockey-stick loss curve

In educational analyses of neural language models and lookup tables [35], it is demonstrated that the scale of initial weights in $\mathbf{W}_{\text{in}}$ and $\mathbf{W}_{\text{out}}$ dictates optimization stability. At initialization, before the network has learned any linguistic patterns, the model should assign an approximately uniform probability to every word in the vocabulary:

$$\hat{y}_w \approx \frac{1}{V}, \quad \forall w \in \mathcal{V}. \tag{4.28}$$

Consequently, the theoretical expected initial cross-entropy loss is strictly:

$$\mathcal{L}_{\text{init}} = -\log\left(\frac{1}{V}\right) = \log V. \tag{4.29}$$

For a vocabulary of $V = 50{,}000$, the initial loss must be $\mathcal{L}_{\text{init}} = \ln(50{,}000) \approx 10.82$. If $\mathbf{W}_{\text{in}}$ or $\mathbf{W}_{\text{out}}$ are initialized with excessively large variance, logits $z_w = \mathbf{u}_w^\top \mathbf{h}$ take extreme values; the softmax distribution becomes peaky and confident on arbitrary words, causing the initial loss to spike drastically ($\mathcal{L} \gg 20$). The optimizer must then spend thousands of iterations flattening these unearned confidences—producing a pathological "hockey-stick" loss curve. Proper variance scaling ($\mathcal{N}(0, 0.01^2)$ or $\mathcal{N}(0, 1/d)$) ensures stable, well-conditioned gradient flow from step zero.

With the general self-supervised training cycle established, we must concretize the network architecture. The first foundational architecture investigated in the course is the Continuous Bag-of-Words (CBOW) model, which aggregates surrounding context words to predict an omitted target center word.

4.5 Word2Vec continuous bag-of-words (CBOW)

In 2013, **Word2Vec** was introduced in [36, 37] (covered on Slides 9–12 of the course lecture deck) as a suite of log-linear models designed to scale distributed representation learning to web-scale corpora containing billions of tokens.

The first of these architectures is the **Continuous Bag-of-Words (CBOW)** model. In CBOW, the objective is to predict the omitted center target token w_t given the symmetric surrounding context window:

$$C_t = \{w_{t-C}, \ldots, w_{t-1}, w_{t+1}, \ldots, w_{t+C}\} \setminus \{w_t\}. \tag{4.30}$$

4.5.1 Matrix formulation and tensor plumbing

The network consists of two weight matrices:

- $\mathbf{W}_{\text{in}} \in \mathbb{R}^{d \times V}$: The input embedding matrix, where each column $(\mathbf{W}_{\text{in}})_{:,k} = \mathbf{v}_k \in \mathbb{R}^d$ represents the vector of token k when acting as context.
- $\mathbf{W}_{\text{out}} \in \mathbb{R}^{d \times V}$: The output embedding matrix, where each column $(\mathbf{W}_{\text{out}})_{:,k} = \mathbf{u}_k \in \mathbb{R}^d$ represents the vector of token k when acting as the target.

Let $\mathbf{e} \in \{0,1\}^V$ denote the multi-hot context vector indicating the presence of words in C_t:

$$e_k = \sum_{j \in \{-C,\ldots,C\} \setminus \{0\}} \mathbf{1}_{\{w_{t+j} = w_k\}}. \tag{4.31}$$

The hidden state $\mathbf{h} \in \mathbb{R}^d$ is obtained by projecting the context vector through $\mathbf{W}_{\text{in}}$ and averaging over the $2C$ context positions:

$$\mathbf{h} = \frac{1}{2C}\mathbf{W}_{\text{in}}\mathbf{e} = \frac{1}{2C} \sum_{\substack{-C \le j \le C \\ j \neq 0}} \mathbf{v}_{w_{t+j}} \in \mathbb{R}^d. \tag{4.32}$$

Next, the model scores candidate target words by projecting $\mathbf{h}$ against the columns of $\mathbf{W}_{\text{out}}$:

$$\hat{\mathbf{p}} = \mathbf{h}^\top \mathbf{W}_{\text{out}} \in \mathbb{R}^{1 \times V}, \quad \text{where } \hat{p}_k = \mathbf{u}_k^\top \mathbf{h}. \tag{4.33}$$

As emphasized in geometric visual analyses [23], the inner product $\mathbf{u}_k^\top \mathbf{h} = \|\mathbf{u}_k\|_2 \|\mathbf{h}\|_2 \cos\theta$ acts as a directional alignment sensor: it evaluates how closely the target word's representation points in the same direction as the accumulated context vector.

The raw similarity logits $\hat{\mathbf{p}}$ are normalized via the softmax function to form the conditional probability distribution:

$$\mathbb{P}(w_t = w \mid C_t) = \frac{\exp(\mathbf{u}_w^\top \mathbf{h})}{\sum_{k=1}^{V} \exp(\mathbf{u}_k^\top \mathbf{h})}. \tag{4.34}$$

The model is trained by minimizing the negative log-likelihood of the ground-truth target word $w^* = w_t$:

$$\mathcal{L}_{\text{CBOW}} = -\log \mathbb{P}(w_t = w^* \mid C_t) = -\mathbf{u}_{w^*}^\top \mathbf{h} + \log \sum_{k=1}^{V} \exp(\mathbf{u}_k^\top \mathbf{h}). \tag{4.35}$$

Figure 4.3 illustrates this computational flow.

4.5.2 Comparison with earlier neural language models: the log-linear breakthrough

A neural language model learning distributed embeddings had been introduced a decade prior in 2003 [11]. However, that architecture had two major differences:

1. **Strict Causal Prediction**: In [11], language was modeled autoregressively, predicting w_t strictly from past tokens $w_{t-n+1}, \ldots, w_{t-1}$. In Word2Vec, the window was opened symmetrically to both left and right context tokens ($w_{t-C}, \ldots, w_{t+C}$), doubling the statistical signal.

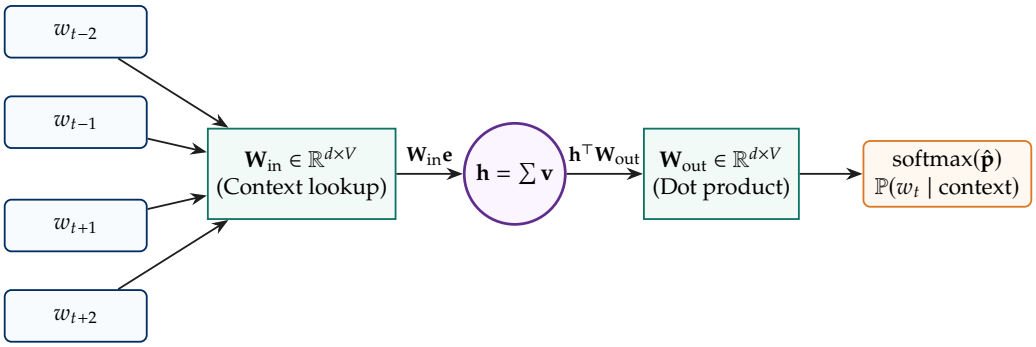


Figure 4.3: Continuous Bag-of-Words (CBOW) architecture. Context words $w_{t-c}, \ldots, w_{t+c}$ are looked up in $\mathbf{W}_{\text{in}}$ and summed to form context state $\mathbf{h} \in \mathbb{R}^d$. Inner products with output matrix $\mathbf{W}_{\text{out}}$ yield similarity logits normalized via softmax to predict the center target token w_t.

2. **Elimination of the Non-Linear Hidden Layer**: In [11], concatenated context embeddings were passed through a non-linear dense hidden layer $\tanh(\mathbf{W}_h\mathbf{x} + \mathbf{b})$. The matrix multiplication with $\mathbf{W}_h$ dominated computation time ($O(C \cdot d \cdot d_h)$). In Word2Vec, the non-linear activation was stripped away entirely, using a direct linear projection (vector averaging). This made Word2Vec a log-linear model, reducing compute costs by orders of magnitude and enabling training on billions of words in a few hours.

4.5.3 Strengths and representational bottlenecks of CBOW

Strengths:

- **Rapid Training Convergence**: Context averaging smooths the gradient signal, allowing CBOW to converge in fewer epochs over large corpora.
- **Robust Representations for Frequent Words**: For high-frequency tokens with abundant contextual observations, the smoothed averaging stabilizes coordinate estimates against noisy outliers.

Representational Bottlenecks:

- **Loss of Sequential Word Order (Syntactic Blindness)**: Vector summation in Equation (4.32) is strictly commutative: $\mathbf{v}_a + \mathbf{v}_b = \mathbf{v}_b + \mathbf{v}_a$. The model is incapable of distinguishing *"the dog bit the man"* from *"the man bit the dog"*.
- **Rare Word Dilution**: Because context words are averaged into a single vector $\mathbf{h}$, if a rare technical token appears alongside multiple high-frequency stopwords (e.g., *"the **polymorphism** in the"*), the rare token's embedding is diluted by the dominant stopwords: $\frac{1}{2C}\mathbf{v}_{\text{rare}} \approx \mathbf{0}$. The gradient update to the rare word is severely attenuated.

Although CBOW trains rapidly by averaging context representations, this very averaging creates a representational bottleneck: it washes out rare vocabulary tokens and discards syntactic word order. To learn high-fidelity representations for infrequent words, inverting the predictive formulation yields the Continuous Skip-Gram architecture.

4.6 Word2Vec continuous skip-gram and the softmax bottleneck

To preserve the distinct gradient signal of rare words, the **Continuous Skip-Gram** architecture was introduced in [36] (examined on Slide 11 of the course slides). Instead of aggregating context words to predict a single center token, Skip-Gram **inverts the predictive direction**: given a

single center target word w_t, the model independently predicts each surrounding context word within window radius C.

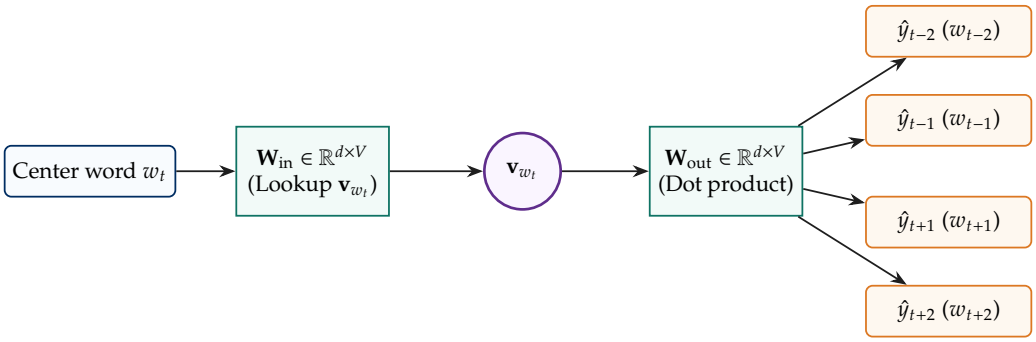


Figure 4.4: Continuous Skip-Gram architecture. A single center target word w_t is projected via $\mathbf{W}_{\text{in}}$ into vector $\mathbf{v}_{w_t} \in \mathbb{R}^d$, which predicts the probability distribution of context words across window radius c through output matrix $\mathbf{W}_{\text{out}}$.

Definition 4.3: *Skip-gram objective function*

Given a training sequence of T words $w_1, w_2, \ldots, w_T$ and a context window of half-width C, the Skip-gram model maximizes the average log-likelihood:

$$\mathcal{L}_{\text{SG}} = \frac{1}{T} \sum_{t=1}^{T} \sum_{\substack{-C \le j \le C \\ j \neq 0}} \log \mathbb{P}(w_{t+j} \mid w_t). \tag{4.36}$$

Under standard multinomial softmax, the conditional probability of context word w_O given center word w_I is:

$$\mathbb{P}(w_O \mid w_I) = \frac{\exp\left(\mathbf{u}_{w_O}^\top \mathbf{v}_{w_I}\right)}{\sum_{w=1}^{V} \exp\left(\mathbf{u}_w^\top \mathbf{v}_{w_I}\right)}, \tag{4.37}$$

where $\mathbf{v}_w \in \mathbb{R}^d$ denotes the column of $\mathbf{W}_{\text{in}}$ (input embedding) and $\mathbf{u}_w \in \mathbb{R}^d$ denotes the column of $\mathbf{W}_{\text{out}}$ (output context embedding).

4.6.1 Why Skip-gram captures rare words

In Skip-Gram, context words are never averaged together. Instead, every context-target co-occurrence (w_I, w_O) generates an independent, dedicated pairwise training instance. When an infrequent, domain-specific term (such as `"refactoring"` or `"polymorphism"`) appears in text, its vector $\mathbf{v}_{w_I}$ directly predicts each context word:

$$\nabla_{\mathbf{v}_{w_I}} \mathcal{L} = \sum_{w_O \in C} \left(\mathbf{u}_{w_O} - \sum_{k=1}^{V} \mathbb{P}(w_k \mid w_I) \mathbf{u}_k \right). \tag{4.38}$$

The gradient is never diluted by surrounding words. Consequently, Skip-Gram produces significantly higher-quality embeddings for rare words, specialized technical jargon, and nuanced semantic distinctions than CBOW.

4.6.2 The shared computational wall: the full softmax bottleneck

Despite its empirical superiority, the standard formulation of Skip-Gram crashes into an insurmountable computational wall: **the full Softmax partition function bottleneck**.

Examining the denominator of Equation (4.37):

$$Z(w_I) = \sum_{w=1}^{V} \exp\left(\mathbf{u}_w^\top \mathbf{v}_{w_I}\right). \tag{4.39}$$

Computing this normalizing denominator requires evaluating an inner product $\mathbf{u}_w^\top \mathbf{v}_{w_I}$ and an exponential $\exp(\cdot)$ for **every single token** w **in the entire vocabulary** $\mathcal{V}$.

- **Computational Complexity per Token**: Computing $Z(w_I)$ takes $O(V \cdot d)$ floating-point operations.
- **Total Corpus Cost**: For a window of radius $C = 5$, predicting $2C = 10$ context words per token over a corpus of $T = 10^9$ words with vocabulary $V = 10^5$ and dimension $d = 300$ requires:

$$10^9 \text{ tokens} \times 10 \text{ predictions} \times 10^5 \text{ words} \times 300 \text{ dims} = 3 \times 10^{17} \text{ FLOPs}. \tag{4.40}$$

- **Gradient Synchronization Bottleneck**: In backpropagation, the derivative $\frac{\partial \mathcal{L}}{\partial \mathbf{u}_w}$ touches every column $w \in \{1, \ldots, V\}$ of $\mathbf{W}_{\text{out}}$ on every single training step, forcing full-matrix memory writes.

This $O(V \cdot d)$ complexity renders standard multinomial softmax completely intractable for web-scale corpora.

While Continuous Skip-Gram excels at capturing nuanced semantic relationships and rare tokens, its standard formulation crashes into a severe computational barrier: normalizing over the full vocabulary requires an $O(V \cdot d)$ denominator calculation for every single token prediction. To scale to realistic vocabularies, we must circumvent this partition function bottleneck.

4.7 Scaling softmax: hierarchical softmax and negative sampling

To bypass the $O(V \cdot d)$ partition function barrier, two distinct scaling methods were formulated in [37]: **Hierarchical Softmax** (a structured tree-based approximation) and **Skip-Gram with Negative Sampling (SGNS)** (a sample-based binary discrimination formulation).

4.7.1 Hierarchical softmax ($O(\log_2 V)$)

First developed in 2005 [38] and adapted in [37], **Hierarchical Softmax** decomposes the flat multiclass probability distribution into a hierarchical sequence of binary routing decisions over a binary tree.

- **Binary Tree Topology**: The V vocabulary words form the leaves of a binary tree (constructed as a binary Huffman tree where frequent words have shorter code lengths).
- **Path Specification**: For any word $w \in \mathcal{V}$, let $L(w)$ denote the depth of the leaf from the root. There exists a unique path of nodes from the root node $n(w, 1) =$ root to the leaf node $n(w, L(w)) = w$.
- **Internal Node Parameters**: Each internal node $j \in \{1, \ldots, L(w)-1\}$ is assigned a continuous parameter vector $\mathbf{v}'_{n(w,j)} \in \mathbb{R}^d$.

- **Binary Branching Probability**: At each internal node, the probability of branching to the left child is parameterized by the logistic sigmoid function:

$$\mathbb{P}(\text{branch left} \mid \mathbf{v}_{w_I}) = \sigma\left(\mathbf{v}'_{n(w,j)}{}^{\top}\mathbf{v}_{w_I}\right) = \frac{1}{1+\exp\left(-\mathbf{v}'_{n(w,j)}{}^{\top}\mathbf{v}_{w_I}\right)}. \tag{4.41}$$

  The probability of branching to the right child is:

$$\mathbb{P}(\text{branch right} \mid \mathbf{v}_{w_I}) = 1-\sigma\left(\mathbf{v}'_{n(w,j)}{}^{\top}\mathbf{v}_{w_I}\right) = \sigma\left(-\mathbf{v}'_{n(w,j)}{}^{\top}\mathbf{v}_{w_I}\right). \tag{4.42}$$

- **Path Probability Product**: The conditional probability of reaching leaf word w is the product of binary decision probabilities along its unique path:

$$\mathbb{P}(w \mid w_I) = \prod_{j=1}^{L(w)-1} \sigma\left(\text{sign}(w,j)\cdot \mathbf{v}'_{n(w,j)}{}^{\top}\mathbf{v}_{w_I}\right), \tag{4.43}$$

  where $\text{sign}(w,j) = +1$ if the path branches left at node j, and -1 if it branches right.

Proof of Exact Normalization. Because probabilities sum to 1 at each binary branching step ($\sigma(z)+\sigma(-z)=1$), telescoping products guarantee that the sum of probabilities over all leaves equals 1 identically:

$$\sum_{w\in\mathcal{V}} \mathbb{P}(w \mid w_I) \equiv 1. \tag{4.44}$$

No global partition function $Z(w_I)$ ever needs to be computed!

Complexity Reduction. Evaluating $\mathbb{P}(w \mid w_I)$ and its gradients requires traversing only the $L(w)-1$ nodes along the path. In a balanced or Huffman binary tree, the average path length is $O(\log_2 V)$. For a vocabulary of $V = 100{,}000$:

$$\log_2(100{,}000) \approx 16.6 \implies \frac{100{,}000}{17} \approx \mathbf{5{,}880}\times \text{ acceleration}. \tag{4.45}$$

4.7.2 Skip-gram negative sampling (SGNS)

While Hierarchical Softmax scales to $O(\log_2 V)$, an even more efficient and empirically superior alternative was formulated in [37]: **Negative Sampling (SGNS)**, derived as a simplified approximation to Noise Contrastive Estimation (NCE).

Instead of treating context prediction as a massive multinomial classification task over V mutually exclusive categories, SGNS reformulates the problem as $1+K$ **independent binary logistic classifications**:

1. **Positive Sample ($y=1$)**: The true center-context pair (w_c, w_o) observed in the text corpus. The model maximizes the probability that this pair is a true co-occurrence.
2. **Negative Samples ($y=0$)**: K noise words $\{w_{n_1}, w_{n_2}, \ldots, w_{n_K}\}$ drawn at random from a noise distribution $P_n(w)$. The model maximizes the probability that these randomly sampled pairs are synthetic noise.

Definition 4.4: *Skip-gram negative sampling objective*

For an observed center-context token pair (w_c, w_o) and K negative samples $\{w_{n_1}, \ldots, w_{n_K}\} \sim P_n(w)$, the SGNS loss is:

$$\mathcal{L}_{\text{SGNS}} = -\log \sigma\left(\mathbf{u}_{w_o}^\top \mathbf{v}_{w_c}\right) - \sum_{k=1}^{K} \log \sigma\left(-\mathbf{u}_{w_{n_k}}^\top \mathbf{v}_{w_c}\right), \tag{4.46}$$

where $\sigma(z) = \frac{1}{1+e^{-z}}$ denotes the standard logistic sigmoid function.

4.7.3 Step-by-step display math analytical gradient derivations

To understand how SGNS shapes the geometry of the embedding space, we derive the exact analytical gradients of $\mathcal{L}_{\text{SGNS}}$ with respect to all parameter vectors.

Recall the fundamental identities for the logistic sigmoid derivative:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)), \tag{4.47}$$

$$\frac{d}{dz} \log \sigma(z) = \frac{\sigma'(z)}{\sigma(z)} = 1 - \sigma(z), \tag{4.48}$$

$$\frac{d}{dz} \log \sigma(-z) = \frac{-\sigma'(-z)}{\sigma(-z)} = \frac{-\sigma(-z)(1 - \sigma(-z))}{\sigma(-z)} = -(1 - \sigma(-z)) = -\sigma(z). \tag{4.49}$$

1. Gradient with respect to Positive Target Vector $\mathbf{u}_{w_o}$. The positive target vector $\mathbf{u}_{w_o}$ appears exclusively in the first term of Equation (4.46):

$$\begin{aligned}
\frac{\partial \mathcal{L}_{\text{SGNS}}}{\partial \mathbf{u}_{w_o}} &= -\frac{\partial}{\partial \mathbf{u}_{w_o}} \log \sigma(\mathbf{u}_{w_o}^\top \mathbf{v}_{w_c}) \\
&= -\left[1 - \sigma(\mathbf{u}_{w_o}^\top \mathbf{v}_{w_c})\right] \frac{\partial (\mathbf{u}_{w_o}^\top \mathbf{v}_{w_c})}{\partial \mathbf{u}_{w_o}} \\
&= \left[\sigma(\mathbf{u}_{w_o}^\top \mathbf{v}_{w_c}) - 1\right] \mathbf{v}_{w_c}.
\end{aligned} \tag{4.50}$$

Defining the positive prediction error as $e_{\text{pos}} = \sigma(\mathbf{u}_{w_o}^\top \mathbf{v}_{w_c}) - 1 = \hat{y}_{\text{pos}} - 1$, this simplifies to:

$$\frac{\partial \mathcal{L}_{\text{SGNS}}}{\partial \mathbf{u}_{w_o}} = (\hat{y}_{\text{pos}} - 1)\mathbf{v}_{w_c}. \tag{4.51}$$

2. Gradient with respect to Negative Noise Vector $\mathbf{u}_{w_{n_k}}$. Each negative sample vector $\mathbf{u}_{w_{n_k}}$ appears in exactly one term of the summation in Equation (4.46):

$$\begin{aligned}
\frac{\partial \mathcal{L}_{\text{SGNS}}}{\partial \mathbf{u}_{w_{n_k}}} &= -\frac{\partial}{\partial \mathbf{u}_{w_{n_k}}} \log \sigma(-\mathbf{u}_{w_{n_k}}^\top \mathbf{v}_{w_c}) \\
&= -\left[-\sigma(\mathbf{u}_{w_{n_k}}^\top \mathbf{v}_{w_c})\right] \frac{\partial (\mathbf{u}_{w_{n_k}}^\top \mathbf{v}_{w_c})}{\partial \mathbf{u}_{w_{n_k}}} \\
&= \sigma(\mathbf{u}_{w_{n_k}}^\top \mathbf{v}_{w_c})\, \mathbf{v}_{w_c}.
\end{aligned} \tag{4.52}$$

Defining the negative prediction error as $e_{\text{neg},k} = \sigma(\mathbf{u}_{w_{n_k}}^\top \mathbf{v}_{w_c}) - 0 = \hat{y}_{\text{neg},k}$, this simplifies to:

$$\frac{\partial \mathcal{L}_{\text{SGNS}}}{\partial \mathbf{u}_{w_{n_k}}} = \hat{y}_{\text{neg},k}\, \mathbf{v}_{w_c}. \tag{4.53}$$

3. Gradient with respect to Center Word Vector $\mathbf{v}_{w_c}$. The center word vector $\mathbf{v}_{w_c}$ appears in both the positive term and all K negative terms. Applying the chain rule to the complete loss:

$$\begin{aligned}\frac{\partial \mathcal{L}_{\text{SGNS}}}{\partial \mathbf{v}_{w_c}} &= -\frac{\partial}{\partial \mathbf{v}_{w_c}} \log \sigma(\mathbf{u}_{w_o}^\top \mathbf{v}_{w_c}) - \sum_{k=1}^{K} \frac{\partial}{\partial \mathbf{v}_{w_c}} \log \sigma(-\mathbf{u}_{w_{n_k}}^\top \mathbf{v}_{w_c}) \\ &= \underbrace{\left[\sigma(\mathbf{u}_{w_o}^\top \mathbf{v}_{w_c}) - 1\right] \mathbf{u}_{w_o}}_{\text{Attractive Component (Pull Context)}} + \underbrace{\sum_{k=1}^{K} \sigma(\mathbf{u}_{w_{n_k}}^\top \mathbf{v}_{w_c})\, \mathbf{u}_{w_{n_k}}}_{\text{Repulsive Component (Push Noise)}} . \end{aligned} \tag{4.54}$$

4.7.4 Physical vector mechanics interpretation

Under gradient descent, parameter updates subtract the gradient: $\mathbf{v}_{w_c} \leftarrow \mathbf{v}_{w_c} - \eta \nabla_{\mathbf{v}_{w_c}} \mathcal{L}$:

$$\mathbf{v}_{w_c} \leftarrow \mathbf{v}_{w_c} + \eta \left(1 - \hat{y}_{\text{pos}}\right) \mathbf{u}_{w_o} - \eta \sum_{k=1}^{K} \hat{y}_{\text{neg},k}\, \mathbf{u}_{w_{n_k}} . \tag{4.55}$$

This update formula admits a clean, intuitive **physical vector mechanics** interpretation:

- **The Attractive Pull Force**: Since $\hat{y}_{\text{pos}} = \sigma(\mathbf{u}_{w_o}^\top \mathbf{v}_{w_c}) \in (0, 1)$, the coefficient $(1 - \hat{y}_{\text{pos}})$ is strictly positive. The update adds a vector component in the direction $+\mathbf{u}_{w_o}$, **pulling the center word vector toward the true context vector**. If the model is already confident ($\hat{y}_{\text{pos}} \approx 1$), the force vanishes; if the model is surprised ($\hat{y}_{\text{pos}} \approx 0$), the force reaches maximum strength $+1 \cdot \mathbf{u}_{w_o}$.
- **The Repulsive Push Force**: For every negative sample k, $\hat{y}_{\text{neg},k} = \sigma(\mathbf{u}_{w_{n_k}}^\top \mathbf{v}_{w_c}) \in (0, 1)$ is strictly positive. The update subtracts a vector component along each noise vector $-\mathbf{u}_{w_{n_k}}$, **pushing the center word vector away from noise words**. If the noise word is far away ($\hat{y}_{\text{neg},k} \approx 0$), no force is exerted; if the noise word is erroneously close ($\hat{y}_{\text{neg},k} \approx 1$), a strong repulsive kick $-\mathbf{u}_{w_{n_k}}$ repels the points.

Crucially, the computational complexity of evaluating Equation (4.55) is $O(K \cdot d)$, which is **entirely independent of vocabulary size** V. In practice, choosing $K \in [5, 20]$ for small corpora and $K \in [2, 5]$ for web-scale corpora provides sufficient negative contrast, reducing training time from days to hours.

4.7.5 Theoretical justification of the $3/4$ unigram noise exponent

How should negative samples be drawn? In [37], the noise distribution $P_n(w)$ is defined as a smoothed unigram distribution raised to the power of $\alpha = 3/4 = 0.75$:

$$P_n(w) = \frac{f(w)^{3/4}}{\sum_{w' \in \mathcal{V}} f(w')^{3/4}}, \tag{4.56}$$

where $f(w)$ denotes the raw unigram occurrence frequency of token w in the corpus.

Why $\alpha = 0.75$ instead of $\alpha = 1.0$ (Raw Frequency) or $\alpha = 0.0$ (Uniform)?

1. **Failure of Raw Frequency ($\alpha = 1.0$)**: Natural language word frequencies follow Zipf's law ($f(r) \propto 1/r$). A tiny fraction of stopwords (`"the"`, `"of"`, `"and"`, `"to"`) account for over 50% of all token occurrences in a typical corpus. If $\alpha = 1.0$, more than half of all negative samples drawn are uninformative stopwords. The model wastes its entire discriminative budget separating words from `"the"`, while rare content words (`"polymorphism"`, `"compiler"`) have near-zero probability of ever being selected as negative samples, receiving virtually zero negative gradient updates.

2. **Failure of Uniform Sampling ($\alpha = 0.0$)**: Under a uniform distribution $P_n(w) = 1/V$, every vocabulary word has equal probability. Because the vast majority of words in any natural vocabulary reside in the long tail of rare words, sampling uniformly selects almost exclusively rare words as negatives. The network rarely practices distinguishing moderately frequent content words from one another.

3. **The Sub-linear Compromise ($\alpha = 0.75$)**: The exponent 3/4 acts as a concave, sub-linear dampener. It compresses the gap between ultra-frequent stopwords and rare content words.

Numerical Demonstration of the Rare-Word Boost. Consider a frequent stopword w_{freq} with corpus frequency $f(w_{\text{freq}}) = 0.01$ (e.g., 10^6 occurrences in 10^8 words) and a rare technical term w_{rare} with corpus frequency $f(w_{\text{rare}}) = 10^{-6}$ (e.g., 100 occurrences in 10^8 words):

- Under raw unigram frequency ($\alpha = 1.0$), the sampling ratio is:

$$\frac{f(w_{\text{freq}})}{f(w_{\text{rare}})} = \frac{10^{-2}}{10^{-6}} = 10{,}000. \tag{4.57}$$

  The stopword is sampled 10,000 times more frequently than the rare word.

- Under the smoothed distribution ($\alpha = 0.75$), the ratio becomes:

$$\frac{f(w_{\text{freq}})^{0.75}}{f(w_{\text{rare}})^{0.75}} = \left(\frac{10^{-2}}{10^{-6}}\right)^{0.75} = (10{,}000)^{0.75} = (10^4)^{3/4} = 10^3 = 1{,}000. \tag{4.58}$$

By applying the 3/4 power, the relative probability of sampling the rare word is amplified by a factor of:

$$\frac{10{,}000}{1{,}000} = \mathbf{10}\times. \tag{4.59}$$

This 10× boost provides rare words with sufficient negative gradient pushes to carve out accurate decision boundaries, while still reflecting broad corpus statistics.

4.7.6 Global count matrix factorization: GloVe

Around the same time, **GloVe (Global Vectors for Word Representation)** was introduced in [39]. Whereas Word2Vec is a local window-based predictive model that processes streaming word pairs, GloVe demonstrated that distributed embeddings could be trained by directly factorizing a **global co-occurrence matrix** $\mathbf{X} \in \mathbb{R}^{V \times V}$, where X_{ij} counts the number of times word j appears in the context window of word i.

GloVe's core insight is that the semantic distinction between two words i and j relative to a probe word k is encoded not by raw probabilities $P_{ik} = X_{ik}/X_i$, but by the **ratio of co-occurrence probabilities**:

$$\frac{P_{ik}}{P_{jk}} = \frac{P(k \mid i)}{P(k \mid j)}. \tag{4.60}$$

Table 4.1 reproduces the seminal demonstration from [39] comparing i = `"ice"` and j = `"steam"` across probe words k.

When k = `"solid"` is strongly related to `"ice"` but not `"steam"`, the ratio is large ($8.9 \gg 1$). When k = `"gas"` is related to `"steam"` but not `"ice"`, the ratio is tiny ($0.085 \ll 1$). When k = `"water"` is equally related to both (or k = `"fashion"` is related to neither), the ratio hovers near 1.0.

Table 4.1: Co-occurrence probability ratios illustrating semantic discrimination in GloVe [39].

Probability & Ratio	k = "solid"	k = "gas"	k = "water"	k = "fashion"
$P(k \mid$ "ice"$)$	1.9×10^{-4}	6.6×10^{-5}	3.0×10^{-3}	1.7×10^{-5}
$P(k \mid$ "steam"$)$	2.2×10^{-5}	7.8×10^{-4}	2.2×10^{-3}	1.8×10^{-5}
$P(k \mid$ "ice"$)/P(k \mid$ "steam"$)$	**8.9**	**0.085**	1.36	0.96

GloVe enforces this relationship by fitting a log-bilinear model where dot products approximate log co-occurrence counts, minimized via weighted least squares:

$$J = \sum_{i=1}^{V} \sum_{j=1}^{V} f(X_{ij}) \left(\mathbf{w}_i^\top \tilde{\mathbf{w}}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2, \tag{4.61}$$

where $f(X_{ij}) = \min(1, (X_{ij}/x_{\max})^\alpha)$ is a sub-linear weighting function preventing frequent stopwords from dominating the loss. Both Word2Vec and GloVe construct semantically structured vector spaces, with SGNS operating on streaming text and GloVe operating on aggregated count statistics.

Skip-Gram Negative Sampling and GloVe resolve the computational bottleneck, making large-scale distributed representation learning feasible. However, Word2Vec operates under an implicit architectural assumption: that the word is the minimal, indivisible atomic unit of language. In real-world deployment, this assumption creates severe structural vulnerabilities.

4.8 Limitations of Word2Vec and the out-of-vocabulary (OOV) barrier

Despite their revolutionary impact, classical Word2Vec and GloVe architectures suffer from fundamental linguistic and software engineering limitations.

4.8.1 The atomic word token assumption and out-of-vocabulary (OOV) failure

Word2Vec models assume a closed, fixed vocabulary $\mathcal{V} = \{w_1, \ldots, w_V\}$ established during training. Each word is treated as an indivisible, atomic entity: its character composition is completely ignored. This creates the **Out-of-Vocabulary (OOV) catastrophe**:

- **Unseen Words at Test Time**: In open-domain text and evolving codebases, users constantly produce novel tokens: technical neologisms, composite identifiers (e.g., `AsyncHttpClientFactory`), misspelled words (e.g., `"recusive"`), and inflected stems.
- **Mapping to `<UNK>`**: When a trained Word2Vec model encounters an OOV word at inference, it has no entry in $\mathbf{W}_{\text{in}}$ or $\mathbf{W}_{\text{out}}$. Deep learning pipelines are forced to map all OOV tokens to a generic unknown token placeholder:

$$w_{\text{oov}} \longmapsto \texttt{<UNK>} \implies \mathbf{v}_{w_{\text{oov}}} = \mathbf{v}_{\texttt{<UNK>}}. \tag{4.62}$$

  This discards all semantic identity. To Word2Vec, a typo like `"algorithmm"` is as completely devoid of meaning as an alien string.

4.8.2 Morphological blindness

Word2Vec is entirely blind to internal word morphology. In natural languages with rich inflectional systems (e.g., Italian, German, Turkish) as well as programming languages, words

share sub-morphemic stems, prefixes, and suffixes:

Natural language: `"develop"`, `"developer"`, `"developing"`, `"undeveloped"`,
Software identifiers: `"parseFile"`, `"parseTokens"`, `"parseTree"`.

Under Word2Vec, each of these variants is assigned a totally disjoint, independent column in $\mathbf{W}_{\text{in}}$. The model possesses zero architectural mechanism to share parameters across words containing the common root `"develop"` or `"parse"`. If `"undeveloped"` occurs only twice in the corpus, its vector remains poorly trained, unable to inherit semantic properties from the frequent token `"develop"`.

4.8.3 Polysemy collapse: static context invariance

Word2Vec assigns exactly **one static vector $\mathbf{v}_w \in \mathbb{R}^d$ to each word string** w, regardless of the surrounding runtime context in which it appears. This results in **polysemy collapse**.

Definition 4.5: *Polysemy collapse*

Let word w possess M distinct semantic senses $\mathcal{S}_1, \ldots, \mathcal{S}_M$ with respective corpus mixture proportions $\pi_1, \ldots, \pi_M$ ($\sum \pi_m = 1$). Because Word2Vec minimizes prediction error across all occurrences using a single vector $\mathbf{v}_w$, the optimal vector converges to the convex combination of distinct sense centroids:

$$\mathbf{v}_w^* \approx \sum_{m=1}^{M} \pi_m \mathbf{v}_{\mathcal{S}_m}. \tag{4.63}$$

For example, consider the polysemous word `"bat"`:

Sense 1 (Zoological): "A **bat** is a nocturnal flying mammal."
Sense 2 (Sports Equipment): "The baseball player swung the wooden **bat**."

In Word2Vec, $\mathbf{v}_{\text{bat}}$ is pulled toward biological animals on some iterations, and toward sports equipment on others. The resulting point converges to an intermediate centroid in $\mathbb{R}^d$ that represents neither animal nor sports gear accurately, degrading cosine similarity to both senses.

In software engineering, polysemy is ubiquitous:

- `"Python"`: A biological serpent vs. a high-level dynamic programming language.
- `"Spring"`: A season of the year vs. a popular Java enterprise application framework.
- `"rust"`: Iron oxide corrosion vs. a memory-safe systems programming language.

A static Word2Vec model maps `"Rust"` to a compromise vector that conflates oxidation chemistry with memory safety guarantees.

While contextual ambiguity requires sequence models that dynamically modulate representations based on full sentence context, the Out-of-Vocabulary barrier and morphological blindness can be solved at the token representation level. By abandoning the word as an atomic unit and descending into subword character n-grams, FastText establishes a morphologically aware embedding paradigm.

4.9 FastText: subword character n-gram embeddings

To conquer the OOV barrier and enable morphological parameter sharing, **FastText** was introduced in [40] (and covered on Slides 13–14 of the course lecture deck).

4.9.1 Character n-gram decomposition and boundary markers

Rather than treating a word as an atomic token, FastText represents each word as a **bag of character n-grams**.

1. **Boundary Enclosure**: To distinguish prefixes and suffixes from character sequences occurring in the interior of a word, each word w is bracketed by special boundary symbols "<" and ">":
$$\tilde{w} = \texttt{"<"} \circ w \circ \texttt{">"}. \tag{4.64}$$
This ensures that the prefix sequence "<re" (as in "<refactor>") is mathematically distinct from the internal substring "re" (as in "<where>").

2. **Subword Extraction**: All character n-grams of length $n \in [n_{\min}, n_{\max}]$ (typically $n \in [3, 6]$) are extracted, along with the special sequence containing the entire word itself $\tilde{w}$.

Course Slide Example: ⟨where⟩. Following Slide 13 of the course lecture deck, consider the word "where" with character trigrams ($n = 3$):

$$\texttt{"<where>"} \implies \mathcal{G}_{\texttt{where}} = \Big\{\texttt{"<wh"}, \texttt{"whe"}, \texttt{"her"}, \texttt{"ere"}, \texttt{"re>"}, \texttt{"<where>"}\Big\}. \tag{4.65}$$

4.9.2 Vector sum composition

Each subword n-gram g is assigned a latent continuous embedding vector $\mathbf{z}_g \in \mathbb{R}^d$. The final representation of word w is defined as the **sum of its constituent subword vectors**:

$$\mathbf{v}_w = \sum_{g \in \mathcal{G}_w} \mathbf{z}_g \in \mathbb{R}^d. \tag{4.66}$$

Under the Skip-Gram Negative Sampling objective, the similarity score between center word w and context word c becomes:

$$s(w, c) = \mathbf{v}_w^\top \mathbf{u}_c = \sum_{g \in \mathcal{G}_w} \mathbf{z}_g^\top \mathbf{u}_c. \tag{4.67}$$

During backpropagation, the prediction error with respect to $\mathbf{v}_w$ distributes gradients to every constituent subword vector $\mathbf{z}_g$.

4.9.3 Resolving OOV tokens dynamically and morphological sharing in code

This simple compositional formulation yields two decisive capabilities:

1. **Zero-Shot Synthesis of Out-of-Vocabulary Tokens**: When an unseen word w_{oov} appears at test time (e.g., a compound identifier like "refactorable" or a code typo like "recusive"), even if the full token was never seen during training, its constituent character n-grams ("<re", "ref", "fact", "able>") were updated across other vocabulary words. FastText synthesizes an accurate embedding vector on the fly by summing the vectors of its known subwords:
$$\mathbf{v}_{w_{\text{oov}}} = \sum_{g \in \mathcal{G}_{w_{\text{oov}}}} \mathbf{z}_g. \tag{4.68}$$
The OOV barrier is completely eliminated!

2. **Morphological Parameter Sharing**: In natural language, prefixes like `"un-"`, suffixes like `"-tion"`, and inflections like `"-ing"` accumulate shared vector representations across hundreds of distinct words. In software engineering, identifier substrings automatically share semantics:

$$\texttt{"parseXmlDocument"} \quad \text{and} \quad \texttt{"parseJsonDocument"} \tag{4.69}$$

   share the character subwords for `"parse"` and `"Document"`. The model immediately recognizes that both identifiers describe parsing operations on structured documents.

Memory Bounding via the Fowler-Noll-Vo (FNV-1a) Hash Trick. Because the universe of all character n-grams ($n \in [3, 6]$) across large vocabularies exceeds tens of millions of unique strings, allocating an unbounded matrix would cause memory blowout. FastText bounds the embedding table to $K = 2 \times 10^6$ rows using the Fowler-Noll-Vo (FNV-1a) hashing function:

$$\text{index}(g) = \text{FNV-1a}(g) \pmod{K}. \tag{4.70}$$

Colliding n-grams share bucket rows; due to the high dimensionality ($d = 300$) and independent sum aggregation, hash collisions act as benign regularizing noise that does not disrupt semantic retrieval.

4.9.4 Connection to modern subword tokenizers: BPE and WordPiece

As highlighted in pedagogical breakdowns of language models [41], the subword philosophy pioneered by FastText directly inspired the tokenization mechanisms powering modern Large Language Models:

- **Byte Pair Encoding (BPE)** [12, 42]: Iteratively merges the most frequent pairs of adjacent characters or bytes to form a deterministic subword vocabulary (used in GPT-2, GPT-4, and LLaMA).
- **SentencePiece** [43]: Treats input text as a raw byte stream without whitespace pre-segmentation, training unigram language model segmentations.

FastText operates continuous character n-gram summations at the embedding layer, whereas BPE operates discrete token mergers during preprocessing. Both architectures fundamentally exist to dismantle the brittle, isolated word-level assumption and establish robust subword parameter sharing.

By decomposing words into character n-grams, FastText conquers the Out-of-Vocabulary barrier and enables morphological parameter sharing. With our embedding models fully trained, we can now step back and examine the remarkable mathematical geometry that spontaneously emerges within these dense vector spaces.

4.10 Emergent vector space geometry: clustering, metrics and relational analogies

When Word2Vec and FastText models are trained over large text corpora, the resulting continuous vector spaces exhibit profound emergent mathematical properties: high-dimensional semantic clustering, metric equivalence, and linear relational translations.

4.10.1 Semantic clustering and dimensionality reduction

In high-dimensional embedding space ($d = 300$), words with similar functional roles organize into distinct, tightly bound clusters.

Following Slide 14 of the course lecture deck, projecting 300-dimensional FastText vectors into two dimensions via Principal Component Analysis (PCA) reveals distinct semantic clusters:

- **Household Items**: `"table"`, `"chair"`, `"desk"`, `"sofa"`.
- **Mammals**: `"dog"`, `"cat"`, `"tiger"`, `"elephant"`.
- **Birds**: `"eagle"`, `"sparrow"`, `"parrot"`, `"pigeon"`.

The leading principal components preserve categorical variance: domestic entities, animal taxonomy, and biological sub-branches form linearly separable clusters in the compressed 2D projection as well as in the original 300-dimensional manifold.

4.10.2 Linear relational translations (vector analogy arithmetic)

The most celebrated emergent property of continuous word embeddings is that semantic and syntactic relationships manifest as **linear vector translation offsets**:

$$\mathbf{v}_{w_2} - \mathbf{v}_{w_1} \approx \vec{d}_{\text{relation}}. \tag{4.71}$$

Country-to-Capital Analogy (Course Slide 15). In the course lecture deck (Slide 15), subtracting the vector for `"Berlin"` from `"Germany"` produces a directional displacement vector $\vec{d}_{\text{capital_of}}$:

$$\mathbf{v}_{\text{Germany}} - \mathbf{v}_{\text{Berlin}} \approx \vec{d}_{\text{capital_of}}. \tag{4.72}$$

Applying this learned offset vector to another country predicts its capital city via simple vector addition:

$$\mathbf{v}_{\text{Spain}} + (\mathbf{v}_{\text{Germany}} - \mathbf{v}_{\text{Berlin}}) \approx \mathbf{v}_{\text{Madrid}}. \tag{4.73}$$

The Gender-Royalty Parallelogram. The canonical demonstration established in [37] is the gender-royalty parallelogram:

$$\mathbf{v}_{\text{king}} - \mathbf{v}_{\text{man}} + \mathbf{v}_{\text{woman}} \approx \mathbf{v}_{\text{queen}}. \tag{4.74}$$

Figure 4.5 illustrates this parallelogram in two dimensions.

Software Engineering Analogy Equivalents. In code representation learning (e.g., CodeBERT [44], OpenAI Codex embeddings [45]), relational vector arithmetic reflects runtime architectures and language toolchains:

$$\mathbf{v}_{\text{Java}} - \mathbf{v}_{\text{JVM}} + \mathbf{v}_{\text{Python}} \approx \mathbf{v}_{\text{CPython}}, \tag{4.75}$$

$$\mathbf{v}_{\text{C++}} - \mathbf{v}_{\text{g++}} + \mathbf{v}_{\text{Rust}} \approx \mathbf{v}_{\text{rustc}}. \tag{4.76}$$

4.10.3 Metric geometry of embedding spaces: cosine vs. Euclidean distance

When querying nearest neighbors or evaluating semantic similarity between two continuous vectors $\mathbf{u}, \mathbf{v} \in \mathbb{R}^d$, three metrics are commonly considered:

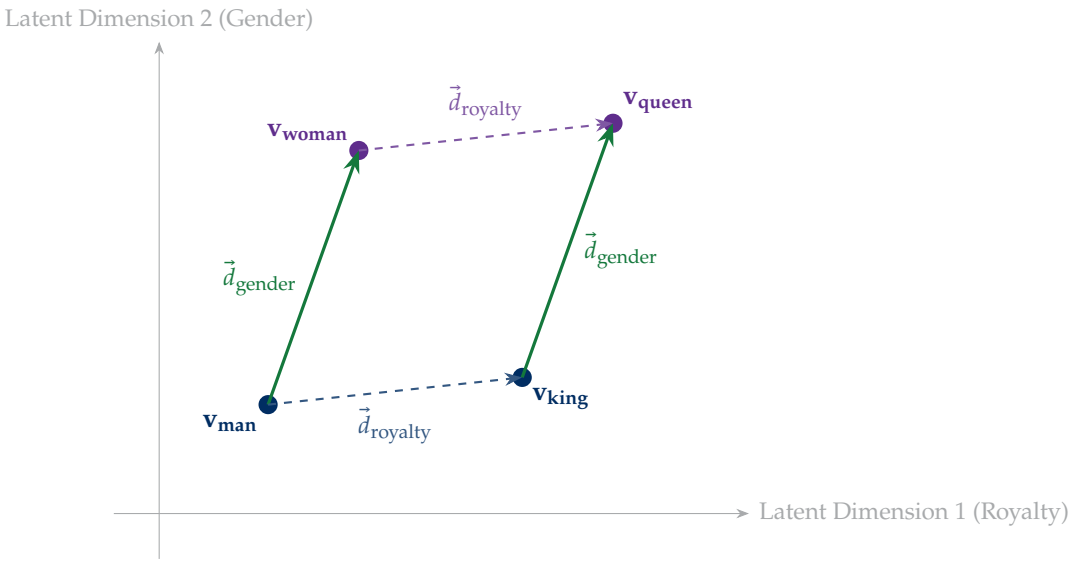


Figure 4.5: Geometric parallelogram in continuous embedding space demonstrating relational vector translations: $\mathbf{v}_{\text{King}} - \mathbf{v}_{\text{Man}} \approx \mathbf{v}_{\text{Queen}} - \mathbf{v}_{\text{Woman}}$. The directional displacement $\vec{d}_{\text{gender}}$ isolates the feminine gender transformation, while $\vec{d}_{\text{royalty}}$ isolates the monarchical status offset.

1. **Euclidean (L_2) Distance**:

$$D_{L_2}(\mathbf{u}, \mathbf{v}) = \|\mathbf{u} - \mathbf{v}\|_2 = \sqrt{\sum_{i=1}^{d} (u_i - v_i)^2}. \tag{4.77}$$

2. **Dot Product (Inner Product)**:

$$\langle \mathbf{u}, \mathbf{v} \rangle = \mathbf{u}^\top \mathbf{v} = \sum_{i=1}^{d} u_i v_i = \|\mathbf{u}\|_2 \|\mathbf{v}\|_2 \cos\theta. \tag{4.78}$$

3. **Cosine Distance**:

$$D_{\cos}(\mathbf{u}, \mathbf{v}) = 1 - \cos(\mathbf{u}, \mathbf{v}) = 1 - \frac{\mathbf{u}^\top \mathbf{v}}{\|\mathbf{u}\|_2 \|\mathbf{v}\|_2} \in [0, 2]. \tag{4.79}$$

Theorem 4.2: *Monotonic equivalence under L_2 normalization*

Let $\mathbf{u}, \mathbf{v} \in \mathbb{R}^d$ be normalized to unit Euclidean length ($\|\mathbf{u}\|_2 = \|\mathbf{v}\|_2 = 1$). Then squared Euclidean distance is strictly proportional to Cosine distance:

$$\|\mathbf{u} - \mathbf{v}\|_2^2 = \|\mathbf{u}\|_2^2 + \|\mathbf{v}\|_2^2 - 2\mathbf{u}^\top \mathbf{v} = 1 + 1 - 2\cos(\mathbf{u}, \mathbf{v}) = 2(1 - \cos(\mathbf{u}, \mathbf{v})) = 2D_{\cos}(\mathbf{u}, \mathbf{v}). \tag{4.80}$$

Consequently, nearest neighbor rankings under Euclidean distance, Cosine distance, and Dot Product are mathematically identical on the unit hypersphere $\mathbb{S}^{d-1}$.

Why Cosine Distance Dominates Natural Language Processing. As demonstrated in educational analyses [41], when word embeddings are trained without explicit length regularization, the Euclidean norm $\|\mathbf{v}_w\|_2$ strongly correlates with unigram token frequency. Highly frequent words undergo millions of gradient pushes, systematically expanding their vector magnitude, whereas rare terms remain close to the origin. Cosine distance normalizes out vector magnitude, projecting all embeddings onto $\mathbb{S}^{d-1}$. It evaluates **pure angular orientation** (conceptual topic) rather than **vector length** (frequency artifact).

4.10.4 Dimensionality trade-offs and the Johnson-Lindenstrauss lemma

Why do word embeddings operate effectively in dimensions $d \in [100, 300]$, and modern LLMs in $d \in [768, 4096]$?

- **Under-parameterization ($d < 50$)**: Severe geometric crowding occurs. The space lacks enough orthogonal degrees of freedom to separate independent semantic concepts, forcing unrelated topics into spurious proximity.
- **Over-parameterization ($d \gg 10{,}000$)**: The space suffers from the curse of dimensionality. The volume of the unit ball concentrates entirely in a thin outer shell, distance distributions become uniform, and memory bandwidth demands explode.

The mathematical guarantee underwriting this stability is the **Johnson-Lindenstrauss Lemma**:

Theorem 4.3: *Johnson-Lindenstrauss Lemma*

Given $0 < \epsilon < 1$, any set of N points in arbitrary Euclidean space can be embedded into a Euclidean space of dimension:

$$d = O\left(\frac{\log N}{\epsilon^2}\right), \tag{4.81}$$

such that all pairwise Euclidean distances are preserved within a distortion factor of $1 \pm \epsilon$:

$$(1 - \epsilon)\|\mathbf{u} - \mathbf{v}\|_2^2 \leq \|f(\mathbf{u}) - f(\mathbf{v})\|_2^2 \leq (1 + \epsilon)\|\mathbf{u} - \mathbf{v}\|_2^2. \tag{4.82}$$

For a vocabulary of $N = 100{,}000$ tokens and distortion tolerance $\epsilon = 0.2$, $\log(10^5)/0.04 \approx 287$, precisely justifying why $d = 300$ serves as the golden standard for dense static word vectors.

4.10.5 Vector databases and Approximate Nearest Neighbor (ANN) search

In modern software engineering, distributed embeddings power semantic code search and Retrieval-Augmented Generation (RAG) across millions of functions and documentation files. Evaluating brute-force cosine comparisons across $N = 10^7$ stored vectors requires $O(N \cdot d)$ operations, resulting in unacceptable search latencies (> 500 ms).

To achieve sub-millisecond retrieval, modern vector databases (e.g., Chroma, FAISS, Milvus, Qdrant) implement **Approximate Nearest Neighbor (ANN)** indexing algorithms [41]:

1. **Hierarchical Navigable Small World (HNSW) Graphs**: Constructs a multi-layer graph topology inspired by the "six degrees of separation" principle. The top layers contain sparse long-range highway edges that skip across semantic clusters, while the bottom layer contains dense local nearest-neighbor connections. Search begins at the top layer with greedy routing and transitions downward, achieving logarithmic $O(\log N)$ query complexity with $> 98\%$ recall.
2. **Inverted File with Product Quantization (IVF-PQ)**: Partitions the vector space into K Voronoi cells using k-means clustering, while Product Quantization divides each d-dimensional vector into M sub-vectors and quantizes them into compact discrete codebooks. This compresses embeddings by $8\times$ to $32\times$, allowing millions of code embeddings to reside entirely in high-speed RAM with symmetric distance lookup tables.

4.10.6 Python implementation and the query filtering caveat

In the course demonstration on Word2Vec and FastText semantic arithmetic [46], a clean Python implementation is presented for answering relational analogy queries:

```python
import numpy as np

def analogy_query(model, word_a, word_b, word_c, top_k=5):
    """
    Computes: word_a is to word_b as word_c is to ?
    Target vector: v = v_b - v_a + v_c
    """
    vec_a = model.get_vector(word_a)
    vec_b = model.get_vector(word_b)
    vec_c = model.get_vector(word_c)

    # Linear relational translation
    target_vec = vec_b - vec_a + vec_c

    # Query nearest neighbors by cosine similarity
    candidates = model.most_similar(positive=[target_vec], topn=top_k + 3)

    # CRITICAL PRACTICAL RULE: Exclude input query words
    query_tokens = {word_a.lower(), word_b.lower(), word_c.lower()}
    filtered_results = [
        (word, score) for word, score in candidates
        if word.lower() not in query_tokens
    ]
    return filtered_results[:top_k]

# Example: Germany is to Berlin as Spain is to ?
# target = v_Berlin - v_Germany + v_Spain -> Madrid
results = analogy_query(model, "germany", "berlin", "spain")
print("Top prediction:", results[0][0])  # Output: 'madrid'
```

Listing 4.1: Implementation of relational vector arithmetic and candidate filtering in Python [46].

Exam insights: The query token exclusion caveat in vector arithmetic

A recurrent practical and examination trap involves candidate ranking in vector arithmetic:

$$\vec{v}_{\text{target}} = \vec{v}_{\text{Berlin}} - \vec{v}_{\text{Germany}} + \vec{v}_{\text{Spain}}. \tag{4.83}$$

Because words naturally have significant baseline norm and self-correlation, if an unconstrained nearest neighbor search is performed across the entire vocabulary $\mathcal{W}$, the query token $\vec{v}_{\text{Spain}}$ or $\vec{v}_{\text{Berlin}}$ will frequently have the highest dot product with $\vec{v}_{\text{target}}$! Practitioners must explicitly filter out the input prompt words ($\mathcal{W} \setminus \{A, B, C\}$), forcing the cosine argmax to search exclusively among genuine unconditioned candidates:

$$w^* = \underset{w \in \mathcal{W} \setminus \{A,B,C\}}{\arg\max} \cos(\mathbf{v}_w, \vec{v}_{\text{target}}). \tag{4.84}$$

4.11 Modern subword tokenization and input pipelines for foundation models

In Sections 4.1 through 4.10, we formulated the mathematical foundations of classical distributed word representations (Word2Vec, GloVe, FastText). While continuous embeddings solved the differentiability barrier and geometric similarity, classical architectures operated under a fragmented, static paradigm: input text was naively pre-split on whitespace into word-level units, vocabulary sizes ballooned into hundreds of thousands of entries, rare or misspelled tokens collapsed into the uninformative <UNK> placeholder, and input sequences were treated as unordered bags or static lookup tables.

Modern Large Language Models (LLMs) such as GPT-2, GPT-3, GPT-4, LLaMA, and Claude discard this isolated design in favor of an integrated, end-to-end **data preprocessing, tokenization, and input embedding pipeline**. As formalized in *Build a Large Language Model from Scratch* [32], preparing raw text for foundation model pretraining requires an indivisible four-stage architectural pipeline:

1. **Subword Tokenization**: Segmenting raw text into atomic subword units using Byte Pair Encoding (BPE), resolving punctuation and whitespace losslessly.
2. **Token ID Mapping**: Converting discrete subwords into numerical indices via an invertible vocabulary lookup table, enriched with special operational context markers.
3. **Autoregressive Sliding Window Sampling**: Constructing shifted input-target training pairs $(\mathbf{X}, \mathbf{Y})$ with causal next-token alignment.
4. **Composite Embedding Synthesis**: Injecting order into permutation-invariant neural layers by additively combining continuous token embeddings with position embeddings: $\mathbf{X}_{\text{in}} = \mathbf{E}_{\text{tok}} + \mathbf{E}_{\text{pos}}$.

Figure 4.6 illustrates this end-to-end data pipeline from raw textual strings to the final continuous tensor ingested by Transformer architectures.

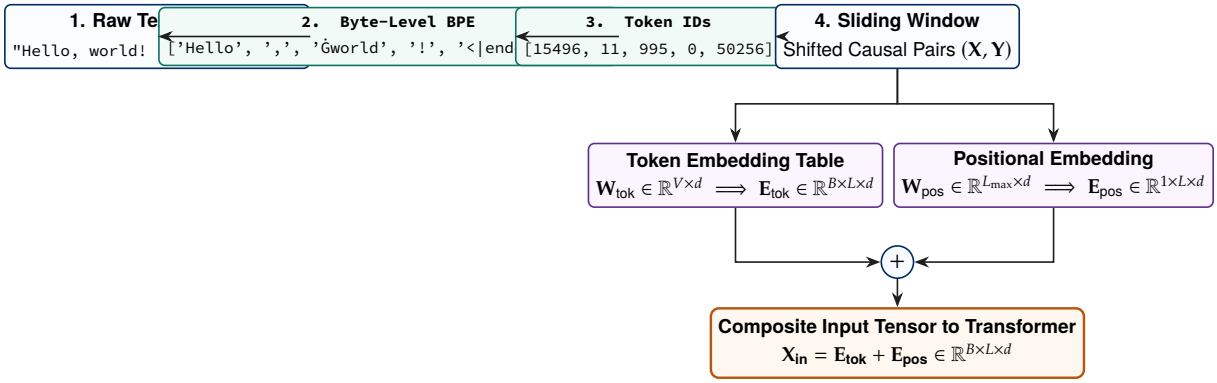


Figure 4.6: The complete modern foundation model tokenization and input preparation pipeline. Raw text is segmented into byte-level BPE subwords, mapped to unique token IDs, sliced into autoregressive input-target pairs via a sliding window, and mapped into continuous latent space where token embeddings and learned positional embeddings combine additively.

4.11.1 The tokenization spectrum: character, word, and subword trade-offs

Converting raw natural language or software code into discrete units reveals a fundamental trade-off across three representational granularities: character-level, word-level, and subword tokenization.

Table 4.2: Comprehensive taxonomy and trade-off analysis of tokenization paradigms in natural language processing and modern foundation models.

Paradigm	Vocab (V)	Length (L)	OOV	Linguistic & Systems Limitations	Representative Models
Character-Level	100 – 256	Massive (4 – 6×)	Zero	High sequence length explodes computational complexity $O(L^2)$ in self-attention; lacks word-level semantic priors.	Char-RNN, Canine, ByT5
Word-Level	50k – 1M+	Minimal (1×)	Severe	Out-of-vocabulary words collapse to `<UNK>`; morphologically blind; massive embedding matrix $\mathbf{W} \in \mathbb{R}^{V \times d}$.	Word2Vec, GloVe, N-gram LMs
Subword-Level	32k – 100k	Moderate (1.2 – 1.4×)	Zero	Requires statistical offline pre-tokenization merge training; optimal Pareto frontier of length and vocabulary.	BPE (GPT-2/3/4, LLaMA), WordPiece (BERT)

Why whitespace splitting fails. A naive baseline approach to text tokenization splits raw strings on whitespace characters ('\s'). However, whitespace splitting fails catastrophically on punctuation:

$$\texttt{"Hello, world! This is a test."} \overset{\text{whitespace split}}{\Longrightarrow} \texttt{['Hello,', 'world!', 'This', 'is', 'a', 'test.']}$$

The comma is fused to `"Hello"`, and periods are fused to `"world"` and `"test"`. Consequently, the model assigns distinct vocabulary indices to `"world"`, `"world!"`, `"world,"`, and `"world?"`. This fragments training counts, bloats the vocabulary with redundant punctuation variants, and destroys syntactic generalization.

Regex-based tokenization with punctuation isolation. To isolate punctuation while preserving them as distinct semantic tokens, Sebastian Raschka formulates a regular expression splitting rule that captures punctuation symbols, em-dashes, and whitespaces:

```python
import re

class SimpleTokenizerV1:
    def __init__(self, vocab):
        self.str_to_int = vocab
        self.int_to_str = {i: s for s, i in vocab.items()}

    def encode(self, text):
        # Split on whitespace, punctuation marks, and special dash patterns
        tokens = re.split(r'([,.:;?_!"()\']|--|\s)', text)
        tokens = [t.strip() for t in tokens if t.strip()]
        token_ids = [self.str_to_int[t] for t in tokens]
        return token_ids

```

```
    def decode(self, token_ids):
        text = " ".join([self.int_to_str[i] for i in token_ids])
        # Clean up spaces before punctuation marks
        text = re.sub(r'\s+([,.?!])', r'\1', text)
        return text

# Building the vocabulary from unique preprocessed tokens
text = "Hello, world. Is this-- a test?"
raw_tokens = re.split(r'([,.:;?_!"()\']|--|\s)', text)
tokens = [t.strip() for t in raw_tokens if t.strip()]
vocab = {token: idx for idx, token in enumerate(sorted(set(tokens)))}
tokenizer = SimpleTokenizerV1(vocab)
encoded = tokenizer.encode("Hello, world.")
print("Encoded IDs:", encoded)
print("Decoded text:", tokenizer.decode(encoded))
```

Listing 4.2: Basic regular expression text tokenization with punctuation isolation and vocabulary construction [32].

4.11.2 Special context tokens: document boundaries, padding, and masked batching

In practical machine learning and software engineering workflows, a vocabulary cannot consist solely of natural language words. The model requires **special context tokens** to encode structural and operational metadata:

1. **Unknown Word Token** (<|unk|>): In fixed-vocabulary systems, words unseen during training cannot be mapped to an integer index. Such tokens are mapped to <|unk|>. While this prevents code execution errors, it irreversibly destroys semantic information: a technical identifier or typo is flattened into a generic void.

2. **Document Boundary Delimiter** (<|endoftext|>): In large-scale autoregressive pretraining, billions of tokens spanning Wikipedia articles, books, and GitHub repositories are concatenated into continuous sequential streams. Without an explicit separator, the causal attention mechanism would assume that the first sentence of an article is the causal continuation of an unrelated previous document! The token <|endoftext|> signals that all preceding context is decoupled from subsequent tokens, allowing clean document demarcation.

3. **Padding Token** (<|pad|>): Mini-batch processing on GPU hardware requires rectangular tensors $\mathbf{X} \in \mathbb{Z}^{B \times L}$. When batching sequences of varying lengths ($L_1 \neq L_2$), shorter sequences are padded with <|pad|> tokens up to maximum length L. Crucially, the model's self-attention mechanism is configured with an **attention mask**:

$$M_{ij} = \begin{cases} 0 & \text{if token } j \text{ is a genuine text token,} \\ -\infty & \text{if token } j \text{ is a <|pad|> token.} \end{cases} \tag{4.85}$$

   Adding M_{ij} to attention logits before Softmax drives pad attention weights to $\frac{e^{-\infty}}{\sum e^z} = 0$, completely preventing padded positions from corrupting token representations.

4.11.3 Byte Pair Encoding (BPE): algorithmic mechanics and merge trees

To conquer the Out-of-Vocabulary barrier without blowing up sequence length, modern LLMs deploy **Byte Pair Encoding (BPE)**. Originally invented for data compression in [47], BPE was adapted to natural language processing in [42].

Algorithmic formalization. BPE constructs a subword vocabulary through an iterative bottom-up statistical merge procedure:

1. **Initialization**: Initialize vocabulary $\mathcal{V}_0$ with all unique characters occurring in the training corpus C. Represent each word in C as a sequence of individual character symbols followed by an end-of-word boundary marker.
2. **Pair Counting**: At iteration k, count the frequency of all adjacent symbol pairs (p_1, p_2) across all words in corpus C:

$$(p_1^*, p_2^*) = \arg\max_{(p_1,p_2)} \sum_{w \in C} \text{count}(w) \cdot \mathbb{I}\{(p_1, p_2) \subseteq w\}. \tag{4.86}$$

3. **Symbol Merge**: Merge the most frequent pair (p_1^*, p_2^*) into a new atomic symbol $p_{\text{new}} = p_1^* \circ p_2^*$.
4. **Vocabulary Expansion**: Update the vocabulary: $\mathcal{V}_k = \mathcal{V}_{k-1} \cup \{p_{\text{new}}\}$, and replace all occurrences of the contiguous pair (p_1^*, p_2^*) across corpus C with p_{new}.
5. **Termination**: Repeat Steps 2–4 for M iterations until the target vocabulary size $|\mathcal{V}| = |\mathcal{V}_0| + M$ is reached.

Table 4.3: Step-by-step Byte Pair Encoding (BPE) merge walkthrough on a toy corpus: `{"low": 5, "lower": 2, "newest": 6, "widest": 3}`.

Iter (k)	Top Pair (p_1^*, p_2^*)	Freq	New Merged Symbol	Corpus Representation After Merge
0	—	—	Base: `{d, e, i, l, n, o, r, s, t, w}`	`l o w : 5, l o w e r : 2, n e w e s t : 6, w i d e s t : 3`
1	`('e', 's')`	9	`'es'`	`l o w : 5, l o w e r : 2, n e w es t : 6, w i d es t : 3`
2	`('es', 't')`	9	`'est'`	`l o w : 5, l o w e r : 2, n e w est : 6, w i d est : 3`
3	`('l', 'o')`	7	`'lo'`	`lo w : 5, lo w e r : 2, n e w est : 6, w i d est : 3`
4	`('lo', 'w')`	7	`'low'`	`low : 5, low e r : 2, n e w est : 6, w i d est : 3`
5	`('n', 'e')`	6	`'ne'`	`low : 5, low e r : 2, ne w est : 6, w i d est : 3`

As demonstrated in Table 4.3, frequent words such as `"low"` coalesce into atomic single tokens, while rare words like `"widest"` decompose into common constituent subwords (`"w"`, `"i"`, `"d"`, `"est"`).

4.11.4 Byte-level BPE: the 256 UTF-8 byte foundation and elimination of unknown tokens

Despite the success of character-level BPE, applying it directly to Unicode strings faces a severe scaling barrier: the Unicode standard contains over 150,000 unique characters spanning hundreds of languages and symbol sets. Initializing $\mathcal{V}_0$ with every conceivable character creates an unmanageably large base alphabet, yet still fails when a user inputs rare historical glyphs or newly released emojis.

The 256 UTF-8 base byte breakthrough. In GPT-2, this dilemma was solved in [12] by introducing **Byte-Level Byte Pair Encoding (BBPE)**. Instead of characters, the base alphabet $\mathcal{V}_0$ is initialized strictly with the **256 distinct byte values** ($0x00$ through $0xFF$) of the UTF-8 encoding standard:

$$\mathcal{V}_0 = \{0x00, 0x01, \ldots, 0xFF\}, \quad |\mathcal{V}_0| = 256. \tag{4.87}$$

Theorem 4.4: *Total elimination of unknown tokens in byte-level BPE*

Under Byte-Level BPE with base alphabet $\mathcal{V}_0 = \{0x00, \ldots, 0xFF\}$, the probability of an Out-of-Vocabulary (OOV) token is strictly zero:

$$\mathbb{P}(w_{\text{oov}}) = 0 \quad \forall S \in \Sigma^*. \tag{4.88}$$

Proof. By definition of the Unicode transformation format (UTF-8), any arbitrary character or sequence of symbols in the Unicode code space ($U+0000$ to $U+10FFFF$) encodes deterministically into a sequence of between 1 and 4 raw bytes in $\{0x00, \ldots, 0xFF\}$. Because the base alphabet $\mathcal{V}_0$ contains all 256 possible 8-bit byte values, every byte sequence can be parsed. If an input string contains an unseen compound word, a programming syntax construct, a severe typo, or an unknown emoji, the tokenizer attempts to apply merged subword rules; if no higher-level merge rule applies, the tokenizer simply falls back to emitting individual byte tokens from $\mathcal{V}_0$. Because no input byte can ever fall outside $\{0x00, \ldots, 0xFF\}$, no token can ever be unmapped. The token `<|unk|>` is completely eliminated. ■

Lossless whitespace preservation. Standard natural language processing pipelines strip whitespace or split on spaces, making exact detokenization ambiguous. In Byte-Level BPE, whitespaces are treated as regular characters/bytes. In GPT-2 and Hugging Face tokenizers, a leading whitespace is mapped to a special character (e.g., 'Ġ' or byte $0x20$), binding the space directly to the succeeding word:

$$\texttt{"Hello world"} \overset{\text{BBPE}}{\Longrightarrow} \texttt{['Hello', 'Ġworld']}$$

This guarantees that tokenization is **strictly lossless and fully invertible**:

$$\texttt{decode}(\texttt{encode}(S)) \equiv S \quad \forall S \in \Sigma^*. \tag{4.89}$$

Production implementations: OpenAI's `tiktoken`. In modern production LLM engineering, tokenization must process terabytes of training data without creating CPU bottlenecks. OpenAI's open-source library `tiktoken` implements byte-level BPE in pure Rust, executing tokenization orders of magnitude faster than Python regular expressions:

- **GPT-2 Tokenizer** (`gpt2`): Vocabulary size $V = 50{,}257$ (composed of 256 base byte tokens +50,000 learned BPE merges +1 special `<|endoftext|>` token).

- **GPT-4 Tokenizer** (`cl100k_base`): Vocabulary size $V = 100{,}277$. Expanding vocabulary from 50k to 100k reduces the total token count of natural language and code by $15 - 20\%$, directly reducing sequence length L, decreasing GPU memory footprint, and slashing API inference costs.

```python
import tiktoken

# Instantiate the GPT-2 tokenizer (vocab size 50,257)
tokenizer = tiktoken.get_encoding("gpt2")

text = "Hello, do you like tea? <|endoftext|> In the sunlit terraces of
    someunknownPlace."
# Disallow special tokens check or explicitly allow specific ones
integers = tokenizer.encode(text, allowed_special={"<|endoftext|>"})
print("Token IDs:", integers)

# Zero-shot handling of unknown words:
# 'someunknownPlace' is decomposed into subwords: ['some', 'unknown', 'Place']
for token_id in integers:
    print(f"{token_id:>5} -> {tokenizer.decode([token_id])!r}")

# Lossless roundtrip reconstruction
decoded_text = tokenizer.decode(integers)
assert decoded_text == text, "Decoded text must match original string identically!"
print("Lossless decoding confirmed!")
```

Listing 4.3: Byte-level BPE tokenization using OpenAI's `tiktoken` library [32].

4.11.5 Regex pre-tokenization boundaries and LLM tokenization quirks

While Byte-Level BPE ensures that any Unicode byte stream can be mapped without out-of-vocabulary errors, executing BPE naively on raw text introduces severe structural pathologies. As dissected in foundational tokenizer analyses and the companion repository `minbpe` [48], production foundation models never execute pair merging directly across an unconstrained document. Instead, they enforce strict category isolation using **regular expression pre-tokenization**.

The naive BPE category-bleed dilemma. Under unconstrained Byte-Level BPE, the merge algorithm scans adjacent byte pairs across the entire text corpus and greedily merges whichever pair exhibits the highest co-occurrence frequency. Because punctuation marks (such as periods, commas, colons, and quotation marks) frequently terminate common English words, naive BPE readily merges letters and punctuation marks together. For instance, if the word `"dog"` frequently occurs at the end of sentences, naive BPE creates the merged token `"dog."`. Similarly, tokens like `"world!"`, `"user123"`, `"return;"`, and `"fn()"` coalesce into monolithic single tokens.

This induces three catastrophic failure modes:

1. **Combinatorial vocabulary bloat**: Rather than maintaining independent vocabulary entries for words and punctuation marks, the vocabulary $\mathcal{V}$ becomes clogged with multiple affixed variants of every common root (`"dog"`, `"dog."`, `"dog,"`, `"dog!"`, `"dog?"`). With a fixed vocabulary budget (e.g., 50,000 tokens), this wastes thousands of token slots on redundant combinations.

2. **Representation fragmentation and broken parameter sharing**: In the token embedding matrix $\mathbf{W}_{\text{in}} \in \mathbb{R}^{V \times d}$, the vector row for `"dog."` is parameterized completely independently of the vector row for `"dog"`. During backpropagation, error gradients for the sentence-final token

`"dog."` do not update the base representation of `"dog"`, fragmenting semantic knowledge and degrading out-of-distribution generalization.

3. **Whitespace and indentation collapse**: If whitespace characters merge indiscriminately across word boundaries (e.g., merging `"in"` and `" the"` into `"in the"`), the tokenizer struggles to parse non-standard spacing, unusual code indentation, or structured YAML/JSON data.

The regex pre-tokenization invariant. To eliminate category bleed, a mandatory pre-tokenization stage was introduced in [12] (GPT-2) and analyzed in [48] (`minbpe`). The input document $S \in \Sigma^*$ is first partitioned by a deterministic regular expression $\mathcal{R}$ into an ordered sequence of M disjoint text chunks:

$$S = c_1 \circ c_2 \circ \cdots \circ c_M, \quad c_m \in \mathcal{L}(\mathcal{R}). \tag{4.90}$$

The fundamental architectural invariant governing regex pre-tokenization is formulated as follows:

The Boundary Invariant: *BPE merges are strictly forbidden from crossing the boundary between adjacent regex chunks.*

Formally, let $\mathcal{M} : \mathcal{V} \times \mathcal{V} \to \mathcal{V}$ denote the learned BPE merge dictionary. A candidate pair (a, b) is eligible for replacement if and only if both subwords originate within the same chunk:

$$\text{eligible}(a, b) \iff \exists m \in \{1, \ldots, M\} \quad \text{such that } (a \circ b) \sqsubseteq c_m. \tag{4.91}$$

Consequently, cross-chunk merges are strictly forbidden:

$$\forall a \in c_m,\ b \in c_{m+1} \quad \mathcal{M}(a, b) = \emptyset. \tag{4.92}$$

Every chunk c_m is tokenized into subwords independently, and the final sequence of token IDs is formed by concatenating the chunk-level token arrays:

$$\mathsf{encode}(S) = \mathsf{encode}_{\text{BPE}}(c_1) \,\|\, \mathsf{encode}_{\text{BPE}}(c_2) \,\|\, \ldots \,\|\, \mathsf{encode}_{\text{BPE}}(c_M). \tag{4.93}$$

Because the punctuation mark in `"dog."` is partitioned into a separate chunk c_2 = `"."`, the pair (`"dog"`, `"."`) is never recorded in the pair frequency histogram during tokenizer training, guaranteeing that words and punctuation remain strictly separated.

Dissecting the GPT-2 and GPT-4 regex patterns. The regular expression used in GPT-2 is formulated across its disjunctive categories as:

$$\begin{aligned}\mathcal{R}_{\text{GPT-2}} = {} & \texttt{'s|'t|'re|'ve|'m|'ll|'d} \\ & \vee\ \texttt{ ?\p\{L\}+} \vee\ \texttt{ ?\p\{N\}+} \vee\ \texttt{ ?[\^\s\p\{L\}\p\{N\}]+} \\ & \vee\ \texttt{\s+(?!\S)} \vee\ \texttt{\s+}\end{aligned} \tag{4.94}$$

where \p{L} and \p{N} denote the Unicode categories for Letters and Numbers, respectively. In GPT-4, OpenAI refined the pre-tokenization pattern into:

$$\begin{aligned}\mathcal{R}_{\text{GPT-4}} = {} & \texttt{'(?i:[sdmt]|ll|ve|re)} \vee\ \texttt{[\^\r\n\p\{L\}\p\{N\}]?+\p\{L\}+} \\ & \vee\ \texttt{\p\{N\}\{1,3\}} \vee\ \texttt{ ?[\^\s\p\{L\}\p\{N\}]++[\r\n]*} \\ & \vee\ \texttt{\s*[\r\n]} \vee\ \texttt{\s+(?!\S)} \vee\ \texttt{\s+}\end{aligned} \tag{4.95}$$

Table 4.4 details the linguistic and algorithmic function of each regex clause.

Table 4.4: Anatomy of regex pre-tokenization patterns in GPT-2 and GPT-4 [12, 48].

Regex clause	Linguistic category	Example matches	Structural invariant preserved
`'s|'t|'re|...`	English contractions	`'s`, `'t`, `'re`, `'ll`	Isolates clitic suffixes so roots and contractions tokenize independently.
`?\p{L}+`	Unicode letters	`"hello"`, `" world"`	Keeps alphabetic words intact while preventing fusion with punctuation.
`?\p{N}+` (GPT-2)	Digits (unbounded)	`" 1"`, `" 42"`, `" 123456"`	Prevents letters and digits from merging; allows multi-digit merges.
`\p{N}{1,3}` (GPT-4)	Digits (capped at 3)	`"1"`, `"234"`, `"567"`	Restricts numbers to ≤ 3 digits, aligning tokens with decimal place values.
`?[^\s\p{L}\p{N}]+`	Punctuation and symbols	`"."`, `"!"`, `"=="`, `"->"`	Groups compound operators/symbols without bleeding into words.
`\s+(?!\S)`	Trailing whitespaces	`" "`, `"\t"`	Preserves indentation blocks and prevents trailing space fusion.

The GPT-4 digit capping refinement and arithmetic reasoning. A profound insight emphasized in [48] lies in the contrast between GPT-2's number clause `?\p{N}+` and GPT-4's `\p{N}{1,3}`: In GPT-2, the unconstrained quantifier + allowed numbers of arbitrary lengths to form a single regex chunk. Consequently, the tokenizer merged frequent multi-digit combinations unpredictably based on dataset frequency. For example, `"123456"` might be tokenized as `["12", "3456"]`, while `"923456"` might be tokenized as `["923", "456"]`. Because digit slice boundaries varied unpredictably across numbers, Transformer attention heads could not consistently align corresponding place values (ones, tens, hundreds, thousands) during multi-digit addition or multiplication, inducing frequent arithmetic failures.

In GPT-4, the quantifier `{1,3}` caps every numeric segment to **at most 3 digits**. A number like 1,234,567 is deterministically split into:

$$\texttt{"1234567"} \xrightarrow{\mathcal{R}_{\text{GPT-4}}} \texttt{["1", "234", "567"]}. \tag{4.96}$$

Because merges cannot cross chunk boundaries, no token can ever represent more than 3 decimal digits. Notice that these chunks align precisely with standard thousands separators in human mathematics $(1,234,567)$. This ensures that numeric tokens reflect consistent orders of magnitude (10^0 to 10^2, 10^3 to 10^5, etc.), providing uniform structural representations that significantly enhance the model's arithmetic and algorithmic computation capabilities.

```python
import regex as re

# GPT-2 pre-tokenization regex pattern
GPT2_SPLIT_PATTERN = r"""'s|'t|'re|'ve|'m|'ll|'d| ?\p{L}+| ?\p{N}+|
    ?[^\s\p{L}\p{N}]+|\s+(?!\S)|\s+"""

# GPT-4 pre-tokenization regex pattern (enforces 3-digit number capping)
GPT4_SPLIT_PATTERN = r"""'(?i:[sdmt]|ll|ve|re)|[^\r\n\p{L}\p{N}]?+\p{L}+|\p{N}{1,3}|
    ?[^\s\p{L}\p{N}]++[\r\n]*|\s*[\r\n]|\s+(?!\S)|\s+"""

sample_text = "Hello, world! 1234567 don't"

# Extract regex-partitioned text chunks
gpt2_chunks = re.findall(GPT2_SPLIT_PATTERN, sample_text)
gpt4_chunks = re.findall(GPT4_SPLIT_PATTERN, sample_text)
```

```python

print("GPT-2 chunks:", gpt2_chunks)
# Output: ['Hello', ',', ' world', '!', ' 1234567', ' don', "'t"]

print("GPT-4 chunks:", gpt4_chunks)
# Output: ['Hello', ',', ' world', '!', ' 123', '456', '7', ' don', "'t"]
# Notice: ' 1234567' is broken into 3-digit chunks: [' 123', '456', '7']
# Crucial Invariant: BPE merges are never permitted to cross chunk boundaries!
```

Listing 4.4: Regex pre-tokenization boundary splitting under GPT-2 and GPT-4 patterns [48].

Intuition & analogy: Why LLMs behave weirdly: demystifying tokenization quirks

Many counter-intuitive behaviors exhibited by large language models are not deep cognitive failures of neural attention, but direct downstream artifacts of subword tokenization [48]:

1. **Character spelling blindness and letter counting**: In natural conversations, models famously stumble when asked *"How many 'r's are in strawberry?"*. The tokenizer maps the string `"strawberry"` to two atomic subword token IDs: [`straw`, `berry`] (e.g., tokens 496 and 675 in GPT-4). The Transformer's attention heads process two continuous vectors; the individual letters $s, t, r, a, w, b, e, r, r, y$ are completely opaque to the model unless it has explicitly memorized the character decomposition of each subword token during pretraining.

2. **String reversal**: Requesting a model to reverse a word (e.g., `"hello"` $\rightarrow$ `"olleh"`) is difficult because the model observes a single token for `"hello"`. Generating the reversed text requires projecting to a completely different sequence of subword or byte tokens, rather than simply reversing an input array.

3. **The non-English token fertility tax**: While English text achieves an average compression ratio of ≈ 1.2 tokens per word, non-Latin scripts (e.g., Japanese, Arabic, Cyrillic, Hindi) require 3 to 4 raw bytes per character in UTF-8. Because pretraining corpora are overwhelmingly English ($> 80\%$), the tokenizer acquires few subword merges for non-Latin scripts. Consequently, non-English text is fragmented into raw bytes, requiring $3\times$ to $8\times$ more tokens per sentence. This consumes context window capacity $4-8\times$ faster, increases autoregressive generation latency, and scales inference costs proportionately.

4. **Glitch tokens and uninitialized embeddings (*SolidGoldMagikarp*)**: In early foundation models, prompt strings like `"SolidGoldMagikarp"`, `"StreamerBot"`, or `"TheNitromeFan"` provoked bizarre hallucinations, hostile replies, or model crashes. This anomaly arose because the BPE tokenizer was trained on an unfiltered web crawl containing frequent Reddit usernames. However, the subsequent pretraining corpus for the neural network was filtered, stripping out Reddit user lists. Consequently, those token IDs remained in the vocabulary $\mathcal{V}$, but their corresponding rows in the embedding matrix $\mathbf{W}_{\text{in}}$ received **zero gradient updates** during pretraining, stranding them at their random Gaussian initialization coordinates ($\mathbf{w} \sim \mathcal{N}(0, \sigma^2)$). When queried, these unnormalized outlier vectors produced anomalous internal activation spikes that disrupted layer normalization and softmax distributions.

5. **Special token leakage and delimiter prompt injection**: Control delimiters like `<|endoftext|>` or `<|fim_middle|>` serve as internal signals. If user-submitted

prompts are passed to tokenizers without escaping special tokens, an adversary can inject delimiter strings to prematurely close prompt contexts or bypass safety guardrails. Modern tokenizers disallow special tokens in user inputs by default (`allowed_special=set()`), requiring explicit developer opt-in.

Connecting token streams to training objectives. Having established how byte-level BPE and regular expression pre-tokenization transform arbitrary text into a clean, category-isolated stream of subword integer tokens $\mathbf{T} = [t_0, t_1, \ldots, t_{N-1}]$, we must now address how this linear token stream is formatted for neural training. We proceed to the autoregressive sliding window formulation that constructs the input-target pairs $(\mathbf{X}, \mathbf{Y})$ powering modern foundation models.

4.11.6 Autoregressive data preparation: the sliding window input-target formulation

After a raw text corpus is converted into a 1D sequence of token IDs $\mathbf{T} = [t_0, t_1, \ldots, t_{N-1}] \in \mathbb{Z}^N$, how does an LLM learn to generate language? Foundation models are trained on the self-supervised objective of **next-token prediction** (autoregressive language modeling). Preparing training data requires chunking continuous token streams into input-target pairs $(\mathbf{X}, \mathbf{Y})$ using a **sliding window**.

The shifted input-target alignment. For a context window length L (often termed '`max_length`' or context size, e.g., $L = 1024$ in GPT-2, $L = 8192$ in LLaMA-3) starting at token index k:

$$\text{Input Sequence:} \quad \mathbf{x} = [t_k,\ t_{k+1},\ t_{k+2},\ \ldots,\ t_{k+L-1}] \in \mathbb{Z}^L, \tag{4.97}$$

$$\text{Target Sequence:} \quad \mathbf{y} = [t_{k+1},\ t_{k+2},\ t_{k+3},\ \ldots,\ t_{k+L}] \in \mathbb{Z}^L. \tag{4.98}$$

The target sequence $\mathbf{y}$ is identically the input sequence $\mathbf{x}$ **shifted forward by one position**.

Why target shifting maximizes training efficiency. In causal language models, a triangular causal attention mask ensures that when computing the output at position i, the model can only attend to tokens $x_0, \ldots, x_i$. Consequently, a single input-target tensor pair of length L simultaneously optimizes L distinct autoregressive prediction tasks in parallel during a single forward-backward pass:

$$\begin{aligned}
\text{Step 1:} \quad & [t_k] \longrightarrow \text{predicts } t_{k+1}, \\
\text{Step 2:} \quad & [t_k, t_{k+1}] \longrightarrow \text{predicts } t_{k+2}, \\
\text{Step 3:} \quad & [t_k, t_{k+1}, t_{k+2}] \longrightarrow \text{predicts } t_{k+3}, \\
& \vdots \\
\text{Step } L: \quad & [t_k, t_{k+1}, \ldots, t_{k+L-1}] \longrightarrow \text{predicts } t_{k+L}.
\end{aligned}$$

Context size and sliding stride. To extract all available training samples from a corpus of N tokens, the window slides by a configurable **stride** s:

$$N_{\text{samples}} = \left\lfloor \frac{N - L}{s} \right\rfloor + 1. \tag{4.99}$$

- **Non-Overlapping Chunks** ($s = L$): Maximizes computational throughput and avoids duplicate token evaluations.

- **Overlapping Sliding Window** ($s < L$, e.g., $s = L/2$ or $s = 1$): Maximizes data efficiency on small datasets and prevents contextual boundary blindness (ensuring tokens near window edges are also evaluated with rich preceding context).

```python
import torch
from torch.utils.data import Dataset, DataLoader

class GPTDatasetV1(Dataset):
    def __init__(self, txt, tokenizer, max_length, stride):
        self.input_ids = []
        self.target_ids = []

        # Tokenize entire corpus into a 1D token ID sequence
        token_ids = tokenizer.encode(txt, allowed_special={"<|endoftext|>"})

        # Slide window across token_ids with stride s
        for i in range(0, len(token_ids) - max_length, stride):
            input_chunk = token_ids[i:i + max_length]
            target_chunk = token_ids[i + 1:i + max_length + 1]
            self.input_ids.append(torch.tensor(input_chunk, dtype=torch.long))
            self.target_ids.append(torch.tensor(target_chunk, dtype=torch.long))

    def __len__(self):
        return len(self.input_ids)

    def __getitem__(self, idx):
        return self.input_ids[idx], self.target_ids[idx]

def create_dataloader_v1(txt, batch_size=4, max_length=256, stride=128, shuffle=True,
    drop_last=True):
    tokenizer = tiktoken.get_encoding("gpt2")
    dataset = GPTDatasetV1(txt, tokenizer, max_length, stride)
    dataloader = DataLoader(
        dataset, batch_size=batch_size, shuffle=shuffle, drop_last=drop_last
    )
    return dataloader

# Example batch extraction
sample_text = ("In software engineering, continuous integration and continuous "
               "delivery automate the testing and deployment pipeline.") * 20
dataloader = create_dataloader_v1(sample_text, batch_size=2, max_length=8, stride=4)
data_iter = iter(dataloader)
inputs, targets = next(data_iter)
print("Batch Input Tensor (X)  shape:", inputs.shape)   # [2, 8]
print("Batch Target Tensor (Y) shape:", targets.shape)  # [2, 8]
print("Input  row 0:", inputs[0].tolist())
print("Target row 0:", targets[0].tolist())
```

Listing 4.5: PyTorch custom `Dataset` and `DataLoader` implementing sliding window causal data sampling [32].

4.11.7 From token IDs to composite inputs: token embeddings and positional encoding synthesis

Having transformed raw text into mini-batch integer tensors $\mathbf{X} \in \mathbb{Z}^{B \times L}$, we arrive at the final computational phase: mapping discrete integer token IDs into continuous, differentiable vector representations equipped with sequential order.

1. The token embedding layer. In PyTorch, continuous token representations are instantiated via:

```
token_embedding_layer = torch.nn.Embedding(num_embeddings=vocab_size, embedding_dim=d)
```

Passing batch tensor $\mathbf{X} \in \mathbb{Z}^{B\times L}$ retrieves the corresponding continuous embedding vectors:

$$\mathbf{E}_{\text{tok}} = \texttt{Embedding}(\mathbf{X}) \in \mathbb{R}^{B\times L\times d}, \tag{4.100}$$

where slice $(\mathbf{E}_{\text{tok}})_{b,t,:} = \mathbf{w}_{X_{b,t}} \in \mathbb{R}^d$ is the d-dimensional embedding vector for the token at position t in batch item b.

2. The permutation invariance of self-attention. However, feeding $\mathbf{E}_{\text{tok}}$ alone into a Transformer causes a fundamental failure. Unlike Recurrent Neural Networks (which process tokens sequentially step-by-step) or Convolutional Neural Networks (which slide localized spatial kernels), the core mathematical operator of Transformers—**Self-Attention**—is strictly **permutation-invariant**:

Theorem 4.5: *Permutation invariance of self-attention*

Let $\mathbf{X} \in \mathbb{R}^{L\times d}$ be an input sequence of L vectors, and let $\mathbf{P} \in \{0,1\}^{L\times L}$ be an arbitrary permutation matrix. Under standard dot-product self-attention $\texttt{Attention}(\mathbf{X}) = \text{softmax}\left(\frac{\mathbf{X}\mathbf{W}_Q(\mathbf{X}\mathbf{W}_K)^\top}{\sqrt{d_k}}\right)\mathbf{X}\mathbf{W}_V$, the operator is equivariant to permutation:

$$\texttt{Attention}(\mathbf{P}\mathbf{X}) = \mathbf{P}\,\texttt{Attention}(\mathbf{X}). \tag{4.101}$$

In the absence of causal masking or positional markers, self-attention treats input tokens as an *unordered set*. To an unaugmented Transformer, the sentences:

"The compiler parsed the code" and *"The code parsed the compiler"*

would generate identical contextual representations! Positional awareness is mandatory.

3. Absolute learned positional embeddings (GPT-2 convention). To inform the model of sequential order, modern autoregressive models (including GPT-2, GPT-3, and original Transformer variants) allocate a dedicated, trainable **positional embedding table**:

$$\mathbf{W}_{\text{pos}} \in \mathbb{R}^{L_{\max}\times d}. \tag{4.102}$$

Each integer position index $p \in \{0, 1, \ldots, L-1\}$ is mapped to a trainable d-dimensional vector $\mathbf{p}_p = (\mathbf{W}_{\text{pos}})_{p,:} \in \mathbb{R}^d$. A position index tensor $\mathbf{p} = [0, 1, \ldots, L-1]^\top$ is queried:

$$\mathbf{E}_{\text{pos}} = \texttt{pos_embedding_layer}(\texttt{torch.arange}(L)) \in \mathbb{R}^{L\times d}. \tag{4.103}$$

4. Additive synthesis of the composite Transformer input. The continuous token representations and positional embeddings are combined via **elementwise addition**:

$$\mathbf{X}_{\text{in}} = \mathbf{E}_{\text{tok}} + \mathbf{E}_{\text{pos}} \in \mathbb{R}^{B\times L\times d}, \tag{4.104}$$

where for each batch sample b and temporal position p:

$$(\mathbf{X}_{\text{in}})_{b,p,:} = \mathbf{w}_{X_{b,p}} + \mathbf{p}_p \in \mathbb{R}^d. \tag{4.105}$$

Because vector addition in $\mathbb{R}^d$ is elementwise, the resulting vector $(\mathbf{X}_{\text{in}})_{b,p,:}$ encodes both *what* token was observed (semantic identity $\mathbf{w}_{X_{b,p}}$) and *where* it appeared in the sequence (temporal coordinate $\mathbf{p}_p$).

```python
import torch

vocab_size = 50257     # GPT-2 BPE vocabulary size
output_dim = 256       # Latent embedding dimension d
context_length = 4     # Sequence length L

# 1. Allocate trainable token embedding layer
token_embedding_layer = torch.nn.Embedding(vocab_size, output_dim)

# 2. Allocate trainable absolute positional embedding layer
pos_embedding_layer = torch.nn.Embedding(context_length, output_dim)

# Example batch of token IDs: [batch_size=2, context_length=4]
inputs = torch.tensor([[40, 367, 2885, 1464],
                       [1807, 3619, 402, 271]])

# Extract token embeddings: [2, 4, 256]
token_embeddings = token_embedding_layer(inputs)

# Generate position indices: tensor([0, 1, 2, 3]) -> [4, 256]
pos_indices = torch.arange(context_length)
pos_embeddings = pos_embedding_layer(pos_indices)

# 3. Additive composite synthesis with PyTorch tensor broadcasting
# token_embeddings: [2, 4, 256] + pos_embeddings: [4, 256] -> [2, 4, 256]
input_embeddings = token_embeddings + pos_embeddings

print("Token Embeddings Shape:", token_embeddings.shape)  # torch.Size([2, 4, 256])
print("Pos Embeddings Shape:  ", pos_embeddings.shape)    # torch.Size([4, 256])
print("Final Composite Shape: ", input_embeddings.shape)  # torch.Size([2, 4, 256])
```

Listing 4.6: PyTorch synthesis of token embeddings and learned positional embeddings [32].

4.11.8 Causal bridge to Chapter 5: from static token lookups to the Transformer attention revolution

Throughout this chapter, we have traced the evolution of vector space representations:

1. **Local Encodings Failed**: One-hot vectors $\mathbf{e}_i \in \{0,1\}^V$ suffered from orthogonality ($d_E = \sqrt{2}, \cos = 0$), memory explosion, and zero parameter sharing.
2. **Continuous Dense Embeddings**: Word2Vec and FastText learned dense vector manifolds $\mathbb{R}^d$ where geometric proximity encodes semantic clustering and linear relational translations.
3. **Byte-Level BPE**: Solved the Out-of-Vocabulary catastrophe by decomposing arbitrary text into subword units down to 256 UTF-8 base bytes, guaranteeing $\mathbb{P}(\text{OOV}) = 0$ with lossless roundtrip detokenization.
4. **Autoregressive Sliding Windows & Positional Injection**: Formulated causal next-token prediction targets and synthesized composite position-aware input tensors $\mathbf{X}_{\text{in}} \in \mathbb{R}^{B \times L \times d}$.

The remaining representational impasse: static isolation. Despite these remarkable advances in data preparation, a profound theoretical limitation remains:

The input tensor $\mathbf{X}_{\text{in}}$ is still a collection of context-independent static lookups.

In $\mathbf{X}_{\text{in}}$, the vector representation for a word like `"bank"` or code keyword `"count"` is identical whether it appears in a financial document, a geography textbook, or an algorithmic loop counter. It contains zero dynamic information about the surrounding tokens in the sentence!

In Chapter 3, we saw that Recurrent Neural Networks (Elman RNNs, LSTMs, GRUs) attempted to build dynamic context sequentially by recurrently propagating state $\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t)$. However, sequential recurrence suffered from an insurmountable physical bottleneck: **the sequential execution wall**, requiring $O(L)$ sequential steps that prevented parallel GPU tensor execution during training.

This impasse sets the stage for the single most consequential breakthrough in modern artificial intelligence: *Can we design an architecture that allows every token in $\mathbf{X}_{in}$ to attend to every other token dynamically across the entire sequence in constant $O(1)$ time, with full GPU sequence parallelization?*

In Chapter 5 (*The Transformer Architecture*), we answer this question by introducing the architecture that discarded recurrence entirely: the **Transformer** [1], powered by Multi-Head Self-Attention.

Bibliography

[1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, pages 5998–6008, 2017.

[2] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 1877–1901, 2020.

[3] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. Training compute-optimal large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 30016–30030, 2022.

[4] Claude E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(3):379–423, 1948.

[5] Daniel Jurafsky and James H. Martin. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition*. Prentice Hall / Online Draft, 3rd draft edition, 2024. URL https://web.stanford.edu/~jurafsky/slp3/.

[6] I. J. Good. The population frequencies of species and the estimation of population parameters. *Biometrika*, 40(3-4):237–264, 1953.

[7] Christopher D. Manning and Hinrich Schütze. *Foundations of Statistical Natural Language Processing*. MIT Press, 1999.

[8] Slava M. Katz. Estimation of probabilities from sparse data for the language model component of a speech recognizer. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 35(3):400–401, 1987.

[9] Reinhard Kneser and Hermann Ney. Improved backing-off for m-gram language modeling. In *IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*, pages 181–184, 1995.

[10] Gerasimos Potamianos and Frederick Jelinek. A study of n-gram and decision tree letter language modeling methods. *Speech Communication*, 24(2):171–192, 1998.

[11] Yoshua Bengio, Réjean Ducharme, Pascal Vincent, and Christian Jauvin. A neural probabilistic language model. *Journal of Machine Learning Research*, 3:1137–1155, 2003.

[12] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019.

[13] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

[14] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

[15] Warren S. McCulloch and Walter Pitts. A logical calculus of the ideas immanent in nervous activity. *The Bulletin of Mathematical Biophysics*, 5(4):115–133, 1943.

[16] Frank Rosenblatt. The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6):386–408, 1958.

[17] George Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals, and Systems*, 2(4):303–314, 1989.

[18] Kurt Hornik, Maxwell Stinchcombe, and Halbert White. Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5):359–366, 1989.

[19] Moshe Leshno, Vladimir Ya. Lin, Allan Pinkus, and Shimon Schocken. Multilayer feedforward networks with a nonpolynomial activation function can approximate any function. *Neural Networks*, 6(6):861–867, 1993.

[20] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.

[21] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986.

[22] Jeffrey L. Elman. Finding structure in time. *Cognitive Science*, 14(2):179–211, 1990.

[23] Grant Sanderson. Visualizing attention, embeddings and neural network trajectories. 3Blue1Brown Educational Video Series, 2024. URL https://www.3blue1brown.com/lessons/gpt.

[24] Paul J. Werbos. Backpropagation through time: what it does and how to do it. *Proceedings of the IEEE*, 78(10):1550–1560, 1990.

[25] Yoshua Bengio, Patrice Simard, and Paolo Frasconi. Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, 5(2):157–166, 1994.

[26] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. On the difficulty of training recurrent neural networks. In *International Conference on Machine Learning (ICML)*, pages 1310–1318, 2013.

[27] Andrej Karpathy. The unreasonable effectiveness of recurrent neural networks. Blog post, 2015. URL https://karpathy.github.io/2015/05/21/rnn-effectiveness/.

[28] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9 (8):1735–1780, 1997.

[29] Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using RNN encoder–decoder for statistical machine translation. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1724–1734, 2014.

[30] Ilya Sutskever, Oriol Vinyals, and Quoc V. Le. Sequence to sequence learning with neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 27, pages 3104–3112, 2014.

[31] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. In *International Conference on Learning Representations (ICLR)*, 2015.

[32] Sebastian Raschka. *Build a Large Language Model (From Scratch)*. Manning Publications, 2024.

[33] Zellig S. Harris. Distributional structure. *Word*, 10(2-3):146–162, 1954.

[34] John R. Firth. A synopsis of linguistic theory 1930–1955. *Studies in Linguistic Analysis*, pages 1–32, 1957.

[35] Andrej Karpathy. Building makemore part 2: MLP language model. YouTube Video Tutorial and GitHub Repository, 2022. URL https://github.com/karpathy/makemore.

[36] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. In *International Conference on Learning Representations (ICLR) Workshop Track*, 2013.

[37] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S. Corrado, and Jeffrey Dean. Distributed representations of words and phrases and their compositionality. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 26, pages 3111–3119, 2013.

[38] Frederic Morin and Yoshua Bengio. Hierarchical probabilistic neural network language model. In *Proceedings of the Tenth International Workshop on Artificial Intelligence and Statistics (AISTATS)*, pages 246–252, 2005.

[39] Jeffrey Pennington, Richard Socher, and Christopher D. Manning. GloVe: Global vectors for word representation. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1532–1543, 2014.

[40] Piotr Bojanowski, Edouard Grave, Armand Joulin, and Tomas Mikolov. Enriching word vectors with subword information. *Transactions of the Association for Computational Linguistics (TACL)*, 5:135–146, 2017.

[41] Enkk. Cosa sono gli embeddings: Geometria semantica, distanza coseno e vector databases. YouTube Educational Lecture, 2023. URL https://www.youtube.com/watch?v=5sYkC8aVv1k.

[42] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 1715–1725, 2016.

[43] Taku Kudo and John Richardson. Sentencepiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 66–71, 2018.

[44] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. CodeBERT: A pre-trained model for programming and natural languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 1536–1547, 2020.

[45] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[46] Flavio Giobergia. Word embeddings and vector space relationships demo. GitHub Gist, 2024. URL https://gist.github.com/fgiobergia/b3a20e097f9b697d0a02fb17685cfd5a.

[47] Philip Gage. A new algorithm for data compression. *C Users Journal*, 12(2):23–38, 1994.

[48] Andrej Karpathy. Let's build the GPT tokenizer. YouTube Educational Lecture and GitHub Repository (minbpe), 2024. URL https://www.youtube.com/watch?v=zduSFxRajkE.